LOVE LAUNDRY

# System Manual

*Laundry, Shop, Hotel and Linen Room Operations*



RASINDU DULSHAN SIRIWARDHANA

*BSC. (HONS) ENGINEERING (COMPUTER ENGINEERING) · SECOND CLASS UPPER DIVISION*

*FACULTY OF ENGINEERING, UNIVERSITY OF RUHUNA*

FIRST EDITION · OCTOBER 2026

# LOVELAUNDRY

SYSTEM MANUAL



FRONTISPIECE · THE HOUSE AND ITS LINEN, ENGRAVED FOR THIS MANUAL.

LOVELAUNDRY

# LoveLaundry

## System Manual

Laundry, Shop, Hotel and Linen Room Operations

Rasindu Dulshan Siriwardhana

BSc. (Hons) Engineering (Computer Engineering) — Second Class Upper Division

Faculty of Engineering, University of Ruhuna

ROLE: DEVELOPER AND SYSTEMS ARCHITECT

VERSION 1.0 · 24 OCTOBER 2026 · LL-SM-001

COPYRIGHT AND DOCUMENT CONTROL

# Document Control

## Copyright

Copyright © 2026 Rasindu Dulshan Siriwardhana. All rights reserved. This manual and the LoveLaundry platform it describes are proprietary works. No part of this document may be reproduced, stored in a retrieval system, transmitted in any form or by any means — electronic, mechanical, photocopying, recording or otherwise — or disclosed to any third party without the prior written consent of the copyright holder, except as expressly permitted in the Licence on the following page.

## Identification

TABLE 0.1 — DOCUMENT IDENTIFICATION RECORD

| | |
|---|---|
| **Document title** | LoveLaundry — System Manual |
| **Document identifier** | LL-SM-001 |
| **Version** | 1.0 — first complete issue |
| **Issue date** | 24 October 2026 |
| **Status** | Issued for operational use |
| **Classification** | Internal — Proprietary |
| **Document owner** | Rasindu Dulshan Siriwardhana, Developer and Systems Architect |
| **Applies to** | LoveLaundry platform release 1.x — six FastAPI services and four React front ends |
| **Review cycle** | Reviewed at each minor release and after every Severity 1 incident |
| **Retention** | Retained for the life of the platform plus seven years |
| **Total pages** | 105, including cover and appendices |

COPYRIGHT AND DOCUMENT CONTROL · CONTINUED

## Author

**Rasindu Dulshan Siriwardhana**

BSc. (Hons) Engineering (Computer Engineering) — Second Class Upper Division
Faculty of Engineering, University of Ruhuna

**Role:** Developer and Systems Architect

## Revision summary

TABLE 0.2 — REVISION HISTORY SUMMARY (FULL HISTORY AT APPENDIX E)

| VERSION | DATE | SUMMARY OF CHANGE | AUTHOR |
|---|---|---|---|
| **1.0** | 24 Oct 2026 | First complete issue. Covers architecture, security, four application manuals, administration, disaster management, backup and recovery, resilience, testing, operations and appendices. | R. D. Siriwardhana |

## How this manual is organised

Chapters 1 to 3 establish the common ground: purpose, architecture and security. Chapters 4 to 7 are task-oriented manuals for the four front ends. Chapters 8 to 13 cover operating discipline — administration, disaster management, backup, resilience, testing and operations. The appendices carry reference material that is consulted rather than read.

CONVENTIONS USED IN THIS MANUAL

Procedures are numbered. Interface elements are shown in **bold**. Literal values, field names, environment variables and endpoints appear in `monospace`. Warnings describe actions that can lose or expose data; notes describe behaviour that is correct but easy to misread. Every figure and table carries a chapter-based number, and every page carries the document title and page number in the footer.

LICENCE

LEGAL

# Proprietary Software Licence

*Terms governing use of the LoveLaundry platform and of this manual.*

**Copyright © 2026 Rasindu Dulshan Siriwardhana. All Rights Reserved.**

The LoveLaundry platform — its services, front ends, schemas, deployment configuration — is proprietary software, licensed and not sold. No ownership or intellectual-property right is transferred; all rights remain with the copyright holder.

## 1. Permitted use

- The copyright holder, and any person expressly authorised in writing, may install, run, configure and maintain the platform for the operation of LoveLaundry laundry, shop, hotel-desk and linen-room services, and may keep one copy of this manual per licensed deployment.
- Hotel client users may use the Hotel Desk application to view only the gate passes, deliveries, invoices and statements belonging to their own `client_name` within the laundry that issued their account.

## 2. Restricted use

- No copying, redistribution, sublicensing, resale, publication or public display of the software or this manual, in whole or in part, without prior written consent.
- No reverse engineering, decompilation or derivation of source code or database structure, except where such restriction is prohibited by law, and no removal of copyright, licence or attribution notices.
- No access to the data of a tenant other than the one granted, and no attempt to bypass tenant scoping, authentication or authorisation controls.

## 3. Confidentiality, warranty, liability and term

**3.1 Confidentiality.** The software, its architecture, its security design and this manual contain confidential information and trade secrets of the copyright holder. The licensee shall protect them with at least the care applied to its own confidential information and shall not disclose them to any third party without prior written consent. This obligation survives termination.

**3.2 No warranty.** The software and this manual are provided **"as is"**, without warranty of any kind, express, implied or statutory, including the implied warranties of merchantability, fitness for a particular purpose, title and non-infringement. The copyright holder does not warrant uninterrupted or error-free operation, or that the software will meet the licensee's requirements.

**3.3 Limitation of liability.** To the maximum extent permitted by law, the copyright holder shall not be liable for indirect, incidental, special, consequential or punitive damages, or for loss of profit, revenue, business, goodwill, data or anticipated savings, arising from the software, its use or inability to be used, or reliance on this manual, even if advised of such damages.

**3.4 Term.** The licence remains in force until terminated and terminates automatically on breach. On termination the licensee shall cease all use, remove all copies and certify removal in writing if requested. Clauses 3.1 to 3.3 survive termination.

LICENCE · CONTINUED

## 4. Third-party components

The platform is built on open-source components that remain under their own licences and copyright holders. Those licences govern the components themselves and nothing here restricts rights granted under them.

TABLE 0.3 — PRINCIPAL THIRD-PARTY COMPONENTS AND THEIR LICENCES

| COMPONENT | ROLE IN THE PLATFORM | LICENCE |
|---|---|---|
| **React 19** | User-interface library for all four front ends | MIT License |
| **Vite 8** | Build tool and development server | MIT License |
| **FastAPI** | Python HTTP service framework | MIT License |
| **MongoDB** | Document database and replication | Server Side Public License (SSPL) v1 |
| **Tailwind CSS 4** | Utility-first styling framework | MIT License |

TypeScript (Apache-2.0), Python (PSF), Flutter (BSD-3-Clause) and the transitive npm and PyPI trees carry their own licences, reproduced in the repository's third-party notices and with the deployed artefacts. The SSPL applies to MongoDB as a database program and does not alter the licence of the LoveLaundry application code.

# Contents

PART ONE · FOUNDATION

**1 · Introduction** 14
1.1 · Purpose 14
1.2 · Scope 14
1.3 · Who the system serves 15
1.4 · Design principles 16
1.5 · Glossary of domain terms 17

**2 · System Overview and Architecture** 19
2.1 · Service map 19
2.2 · Service catalogue 20
2.3 · Front-end applications 22
2.4 · Request lifecycle 23
2.5 · The three-database model 24
2.6 · Data ownership 26
2.7 · JWT and tenancy flow 27
2.8 · Hotel client-account isolation 28
2.9 · Deployment topology 29
2.10 · Technology stack 30

**3 · Security Model** 35
3.1 · Authentication 35
3.2 · Roles and capabilities 36
3.3 · Tenant isolation rules 38
3.4 · Encryption at rest 39
3.5 · Transport, audit and rate limiting 41
3.6 · Secrets management 42

# Contents — continued

PART TWO · THE APPLICATIONS

**4 · User Manual — Laundry App** 46
4.1 · Sign-in 46
4.2 · Dashboards 47
4.3 · Gate passes 48
4.4 · Creating a gate pass 48
4.5 · Deliveries 49
4.6 · Bills 50

**5 · User Manual — Shop Counter App** 58
5.1 · Overview and sign-in 58

**6 · User Manual — Hotel Desk App** 66
6.1 · Hotel client sign-in 66

**7 · User Manual — Linen Room App** 75
7.1 · Overview and scanning 75

# Contents — continued

PART THREE · OPERATING THE BUSINESS

**8 · Administration and Configuration** 84
8.1 · Creating laundries 84

**9 · Disaster Management** 89
9.1 · Framework and risk method 89
9.2 · Risk register 89
9.6 · Communication plan 95
9.7 · Post-incident review 96

**10 · Backup and Recovery** 98
10.1 · What is backed up 98

**11 · Fail-Safes and Resilience** 105
11.1 · Fail-closed tenancy 105

PART FOUR · ASSURANCE AND REFERENCE

**12 · Testing and Quality Assurance** 113
12.1 · Test strategy 113

**13 · Operations Runbook and Monitoring** 119
13.1 · Health checks 119

**· Appendices** 124

# List of Figures

**Figure 1** · Service map. Five client applications call six FastAPI services; three MongoDB databases hold identity, operations and accounting data respectively. The operational core, bill_service, is outlined in the accent colour. 18

**Figure 2** · Request lifecycle for an authenticated write. Tenancy resolution precedes authorisation, and no query is built before both have succeeded. 22

**Figure 3** · JWT issuance and tenancy resolution. The token is issued once, verified on every request, and is the sole carrier of laundry_id. 26

**Figure 4** · Reference deployment topology: proxy at the edge, services on loopback, replica set and object storage behind a private network. 28

**Figure 5** · Service dependency graph, drawn as a line diagram: authenticated HTTP between services, per-service credentials to each store. 31

**Figure 6** · The tenancy decision chain. Each failure returns before the next check runs. 32

**Figure 7** · Failure domains in the reference deployment: application host, database replica set, object storage. 32

**Figure 8** · Envelope encryption at rest, with a separate deterministic path for equality search. 38

**Figure 9** · Claim-by-claim comparison of staff and hotel client tokens. 42

**Figure 10** · Laundry app sign-in with field annotations. 44

**Figure 11** · Gate-pass register with annotations. 46

**Figure 12** · Gate-pass editor, showing header fields and item lines. 46

**Figure 13** · Day-close sequence, including the blocked branch when unbilled deliveries remain. 54

**Figure 14** · Shop Counter today view. 57

**Figure 15** · Quick bill pad: item list left, one-touch services and tender right. 58

**Figure 16** · Hotel Desk overview for a single property. 65

**Figure 17** · Request path from a client token to a scoped query, with the rejected paths shown. 71

**Figure 18** · Linen state machine. Exceptions (mark_missing, mark_damaged, retire) branch from the active state. 74

**Figure 19** · Scan station: scan field, current tag card and legal next actions. 75

**Figure 20** · The linen state machine with its exception branches. Transitions are written by scan events only. 80

**Figure 21** · Incident roles. In a small team one person may hold two roles, but the scribe role is never dropped. 91

**Figure 22** · Severity bands with response targets and the people they summon. 93

**Figure 23** · Backup, retention and point-in-time recovery timeline. 96

**Figure 24** · Daily dumps, continuous oplog and the off-host copy, against the recovery-point target. 101

**Figure 25** · Idempotent write path. The key and the record are committed together, so a crash cannot leave one without the other. 103

**Figure 26** · Layered defences against double billing, from the data model to the day-close review. 107

**Figure 27** · Test strategy pyramid: broad service-level coverage, thin top layer. 109

**Figure 28** · Measured coverage by layer. 111

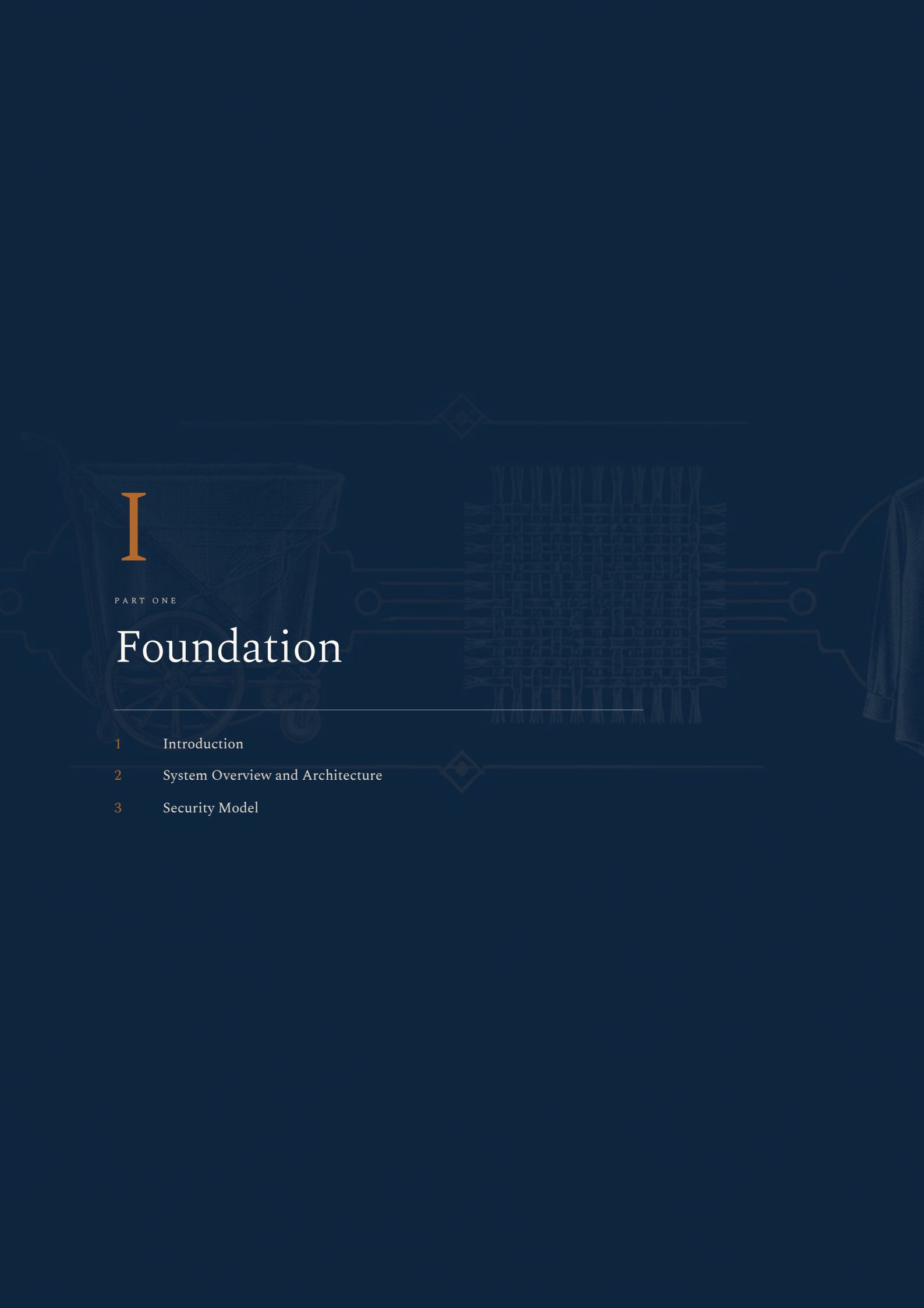
**Figure 29** · Health dependency chain. The proxy routes only to services whose readiness probe passes. 115

**Figure 30** · Maintenance calendar by cadence. 118

**Figure 31** · Alert lifecycle and the escalation loop when acknowledgement does not happen within target. 118

# List of Tables

**Table 1** · Scope boundaries by capability area 13
**Table 2** · Design principles and their practical consequences 14
**Table 3** · Core domain vocabulary 16
**Table 4** · Services, responsibilities and interfaces 19
**Table 5** · Service-to-service dependencies 19
**Table 6** · Front-end applications, sessions and audiences 21
**Table 7** · Front-end toolchain 21
**Table 8** · Stage-by-stage failure behaviour 22
**Table 9** · Store characteristics 23
**Table 10** · Collection ownership and key fields 24
**Table 11** · Token shapes compared 27
**Table 12** · Technology stack 29
**Table 13** · Runtime and tooling versions 29
**Table 14** · Authentication paths 34
**Table 15** · Roles, scope and intent 35
**Table 16** · Capability reference 35
**Table 17** · Tenancy invariants 37
**Table 18** · Audited events 40
**Table 19** · Rate-limit classes 40
**Table 20** · Dashboard tiles and what each one means 45
**Table 21** · Typical use of the dashboard through the day 45
**Table 22** · Delivery rules 47
**Table 23** · Bill numbering and identity 48
**Table 24** · Payment methods and their effect 50
**Table 25** · Quotation states 50
**Table 26** · Worker record 51
**Table 27** · Available reports 52
**Table 28** · What the close freezes 53
**Table 29** · Common errors and what to do 54
**Table 30** · Shop Counter at a glance 56
**Table 31** · Shop bill states 58
**Table 32** · Reconciliation items 59
**Table 33** · Shop roles 60
**Table 34** · Capability mapping 60
**Table 35** · Handover checklist 62
**Table 36** · Common errors at the counter 62
**Table 37** · Hotel Desk at a glance 64
**Table 38** · Columns in the hotel gate-pass list 66
**Table 39** · Delivery information visible to a hotel 66
**Table 40** · Invoice states visible to the hotel 67
**Table 41** · Statement contents 68
**Table 42** · Hotel permissions 69
**Table 43** · Errors a hotel user may see 70
**Table 44** · Linen Room at a glance 73
**Table 45** · Actions and their meaning 74
**Table 46** · Inventory views 76
**Table 47** · Discrepancy types 77
**Table 48** · Tag printing rules 77
**Table 49** · Common problems at the station 79
**Table 50** · What a new laundry requires 81
**Table 51** · Role assignment guidance 82
**Table 52** · Client account record 83
**Table 53** · Required configuration 84
**Table 54** · Rollback decision points 84
**Table 55** · Scoring scales 86
**Table 56** · Risk register 86
**Table 57** · Supplier failure fallbacks 90
**Table 58** · Severity levels, examples and targets 91
**Table 59** · Who is told what, and when 92
**Table 60** · Review agenda 92
**Table 61** · Backup inventory 95
**Table 62** · Targets per component 96
**Table 63** · Schedule and retention 97
**Table 64** · Post-restore checks 99
**Table 65** · Choosing the recovery point 99
**Table 66** · Drill checklist 100
**Table 67** · Failure conditions and outcomes 102
**Table 68** · Concurrency outcomes 103
**Table 69** · Atomic units 104
**Table 70** · Capability by failed component 105
**Table 71** · Four layers of defence 106
**Table 72** · Test layers and their scope 109
**Table 73** · Measured test results 110
**Table 74** · Coverage by area 111
**Table 75** · Troubleshooting a suite run 112
**Table 76** · Known limitations 113
**Table 77** · Health probes 115
**Table 78** · Log fields 116
**Table 79** · Alert catalogue 117
**Table 80** · Maintenance schedule 118
**Table 81** · user-service endpoints (port 8002) 120
**Table 82** · Client and hotel account endpoints 120
**Table 83** · Operational endpoints (port 8001) 121
**Table 84** · Shop and reporting endpoints 121
**Table 85** · Linen endpoints 122
**Table 86** · Client portal endpoints (bill_service) 122
**Table 87** · Supporting services 122
**Table 88** · Capabilities by role 123
**Table 89** · Status values by document 124
**Table 90** · Glossary of terms 125
**Table 91** · Revision history 126
**Table 92** · Typographic and structural conventions 126
**Table 93** · A – L 127
**Table 94** · M – Z 127



I

PART ONE

# Foundation


1 Introduction

2 System Overview and Architecture

3 Security Model

PART ONE · INTRODUCTION

# 1 Introduction

*What LoveLaundry is, the operation it was built for, and the vocabulary its users already speak.*

## 1.1 Purpose

This manual documents the LoveLaundry platform: a multi-tenant operations system for a commercial laundry that also serves a retail shop counter, hotel clients, and its own linen room. It exists so that the people who run, support and extend the platform have one authoritative description of how it is put together and how it is used — not a marketing overview, and not a substitute for reading the code, but the document that makes the code and the operation legible to each other.

The manual serves three audiences at once. Operators — counter staff, supervisors, linen-room attendants, hotel front-office users — need precise, numbered procedures and a clear statement of what the system will and will not let them do. Administrators need to know how tenants, users, memberships and hotel client accounts are created, and what the security model enforces on their behalf. Engineers and support staff need the architecture, the failure modes, the recovery paths and the measured state of the test suites, so that an incident can be diagnosed without rediscovering the design under pressure.

Every statement in this manual is either derived from the implementation as it stands or is explicitly marked as a target, a policy or a recommendation. Where the platform does not yet do something, the manual says so. Chapter 12 in particular reports measured test results rather than aspirational ones, and names the gaps — no browser-driven end-to-end suite, no load testing — instead of implying coverage that does not exist.

## 1.2 Scope

The manual covers the platform as deployed for a single laundry business operating four front ends against a shared service estate. The boundary is set out below so that readers know when they have left the documented surface.

**In scope**

- Authentication, tenancy and membership for staff, hotel client and linen-room users.
- Gate passes, deliveries, bills, shop bills, payments, quotations and returns.
- Linen tagging, scanning and the full wash lifecycle, including discrepancies.
- The monthly matrix, dispatch, dashboards and day close.
- Deployment, configuration, backup, recovery, monitoring and incident response for the platform itself.

**Out of scope**

- Accounting practice and statutory reporting outside the platform's own ledgers.
- The internal design of third-party components (MongoDB, React, FastAPI and their dependencies).
- Payment-provider contracts, bank settlement and card-acquirer operations.
- Mobile client (LoveMobi) feature documentation beyond its role in the service map.
- Hardware procurement, printer models and network cabling.

TABLE 1 — SCOPE BOUNDARIES BY CAPABILITY AREA

| AREA | DOCUMENTED HERE | BOUNDARY NOTE |
|---|---|---|
| **Gate passes and deliveries** | Yes — full | Creation, editing, status, dispatch and the linen lines attached to each pass. |
| **Bills and payments** | Yes — full | Bill generation from deliveries, payment recording, balances and day close totals. |
| **Shop counter sales** | Yes — full | Counter bills, quick bill, shift totals. Stock valuation is outside the platform. |
| **Hotel client portal** | Yes — full | Read-only operational and financial view scoped to one `client_name`. |
| **Linen room** | Yes — full | Tag lifecycle, scanning, inventory and discrepancy handling. |
| **Infrastructure** | Partial | Topology, backup and recovery are documented; provider-specific console steps are not. |

## 1.3 Who the system serves

LoveLaundry is one platform with four working surfaces. Each exists because a different group of people does a different job and at a different speed. The design keeps them on one dataset while giving each a purpose-built interface.

**The laundry business**

The core operation: collecting soiled linen and garments by gate pass, washing and finishing them, delivering them back, and billing for the work. The laundry app (quotations-ui) is the full-surface application — gate passes, deliveries, bills, quotations, workers, reports, the monthly matrix, dispatch and day close. It is where supervisors and managers spend their day.

### The shop counter

A retail and walk-in service point. Customers arrive with a bag, not an account. The Shop Counter app is deliberately narrow: a today view, a bill list, a quick bill pad and a day close.

### Hotel clients

Hotels are the laundry's largest customers and its most demanding readers of records. A hotel is *not* a separate entity in the platform: it is the `client_name` carried on gate passes, deliveries, bills and quotations. Hotel staff sign in with a client account and see only their own property's records — never another hotel's, and never the laundry's internal costs.

### The linen room

The linen room is where physical items are counted, tagged, washed and reconciled. The Linen Room app turns that physical work into a digital trail: each tag is scanned at every stage, from collection through wash, press and readiness to delivery, with explicit actions for items that go missing, arrive damaged or are retired from service. Its users work one-handed with a scanner, so the interface is a single scan field and a short list of valid next actions.

## 1.4 Design principles

Seven principles explain most of the decisions in the codebase. They are stated here because they are also the grounds on which a change should be rejected.

TABLE 2 — DESIGN PRINCIPLES AND THEIR PRACTICAL CONSEQUENCES

| PRINCIPLE | WHAT IT MEANS IN PRACTICE |
|---|---|
| **Tenant from the token** | The `laundry_id` is read from verified JWT claims. No request parameter, header or body field can set or override it. Missing context fails closed rather than falling back to an unscoped query. |
| **One operation, one record** | Gate passes, deliveries and bills are separate documents with explicit links, so each can be corrected without rewriting history. A bill is derived from deliveries; it does not absorb them. |
| **Money is additive and reversible** | Payments are records, not field mutations. Balances are computed. Corrections are new records that reference the original, so an audit trail survives every mistake. |
| **Physical truth is scanned** | The linen lifecycle advances only through explicit scan actions. No status is inferred from time passing, and no bulk update silently skips a stage. |
| **Interfaces match the job** | Counter staff get speed; linen staff get one scan field; hotels get read-only clarity; managers get the full surface. No single screen tries to serve all four. |
| **Degrade, do not disappear** | When a dependency is unavailable the affected feature stops and says so, while independent features continue. Chapter 11 states exactly what remains usable. |
| **Recoverable by design** | Backups, point-in-time recovery and restore drills are part of the product, not an afterthought. Chapter 10 defines the targets and the procedure. |

> WARNING
>
> Any change that resolves a tenant from a request body or header is a security defect, not a feature — as is any change that lets a hotel client token reach a staff endpoint.

## 1.5 Glossary of domain terms

The platform uses the vocabulary of the laundry trade. These terms recur throughout the manual and in the interface, and they do not always mean what a software reader would expect.

TABLE 3 — CORE DOMAIN VOCABULARY

| TERM | MEANING IN LOVELAUNDRY |
|---|---|
| **Gate pass** | The document that accompanies goods leaving or entering the premises. It records the client, the date, the item lines and quantities, and it is the origin of every downstream delivery and bill. A gate pass is the unit of operational truth: if it is not on a pass, it did not happen. |
| **Delivery** | The act and record of returning finished goods to the client. A delivery references the gate pass lines it fulfils, carries the delivered quantities, and is the input to billing. Partial deliveries against one pass are normal. |
| **Bill** | A financial document raised for one client over a period or a set of deliveries. It states quantities, rates, charges, discounts and tax, and it holds the balance outstanding. A bill is not edited once paid; corrections are made by credit or adjustment records. |
| **Linen tag** | A physical label with a unique identifier attached to an item or bundle. The tag is the join between the physical item and its digital record; scanning it advances the lifecycle. Tags are printed from the Linen Room app and retired when an item leaves service. |
| **Day close** | The end-of-day reconciliation. It freezes the day's totals — bills raised, payments received, shop sales, deliveries — and marks the business date closed so that later edits cannot silently change reported figures. Each front end has its own close, and the laundry close is the authoritative one. |
| **Monthly matrix** | A grid of clients against days of the month showing daily quantities and values, used to reconcile contract volumes and to spot missing or duplicated entries. It is a reporting artefact, generated from gate passes and deliveries, not a separately maintained ledger. |
| **Quotation** | A priced offer to a prospective or existing client, with line items, rates and validity. A quotation can be converted into operational work but does not itself create a bill. |
| **Shop bill** | A counter sale to a walk-in customer, recorded in the shop-app rather than the laundry ledger. Shop bills carry their own numbering and are reconciled at shop day close. |
| **Dispatch** | The release of finished goods to the delivery route. Dispatch records who released what, to which vehicle or driver, and at what time, closing the loop between the linen room and the client. |
| **Return** | Goods coming back from a client — rejected items, unused linen, or items sent in error. A return references the original delivery where one exists. |
| **Client account** | A login belonging to a hotel client organisation, stored in the `client_accounts` collection of user-service. It carries a `client_scope` of one laundry and one `client_name`; it is not a membership and grants no staff capability. |
| **Tenant / laundry** | The laundry business that owns a set of records. Represented by `laundry_id`. Every business document carries it and every query filters on it. |
| **Membership** | The link between a user and a laundry, carrying the user's role in that laundry. A user may hold memberships in more than one laundry; each session acts in exactly one. |
| **Capability** | A named permission such as `bill:write` or `linen:scan`. Roles are bundles of capabilities; endpoints check capabilities, not role names. |
| **Discrepancy** | A mismatch between expected and scanned linen quantities or statuses. Discrepancies are recorded, not corrected silently, so that loss patterns remain visible. |

NOTE ON "HOTEL"

Throughout this manual, "hotel" refers to a client organisation identified by the `client_name` string. It is never a database entity, never a tenant, and never a login domain of its own. Chapter 2.8 explains why this matters for isolation.

2

# System Overview and Architecture

*Six services, four front ends, three databases, one tenancy rule.*

## 2.1 Service map

LoveLaundry is a service-oriented system built from Python/FastAPI services, each owning a bounded part of the business, behind four React front ends. There is no shared application server and no shared in-process state: a front end calls a service over HTTPS with a bearer token, and the service answers from the database it owns. Figure 1 shows the estate as deployed.

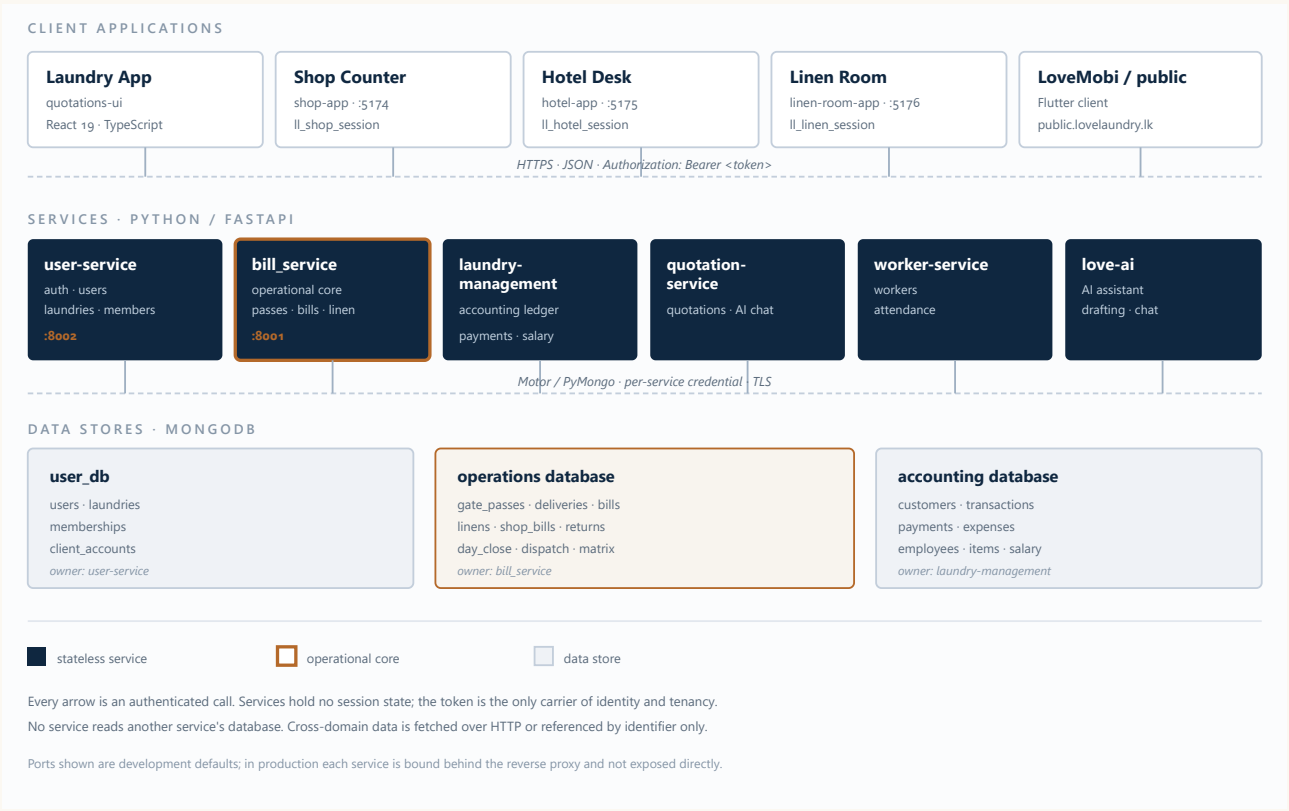



FIGURE 1 — SERVICE MAP. FIVE CLIENT APPLICATIONS CALL SIX FASTAPI SERVICES; THREE MONGODB DATABASES HOLD IDENTITY, OPERATIONS AND ACCOUNTING DATA RESPECTIVELY. THE OPERATIONAL CORE, BILL_SERVICE, IS OUTLINED IN THE ACCENT COLOUR.

> WHY THIS SHAPE
>
> The split follows ownership, not technology. Identity and tenancy change rarely and must be trusted by everyone, so they live in user-service. Operational documents change constantly and are read by all four front ends, so they live in bill_service. Accounting follows its own reporting calendar and retention rules, so it is isolated in laundry-management.

## 2.2 Service catalogue

Each service is independently deployable and owns its collections. The table below is the authoritative list of services, their responsibilities and their development ports.

TABLE 4 — SERVICES, RESPONSIBILITIES AND INTERFACES

| SERVICE | PORT | RESPONSIBILITY | OWNS |
|---|---|---|---|
| **user-service** | 8002 | Authentication for staff and hotel clients, user records, laundries (tenants), memberships, hotel client accounts, role and capability resolution, token issuance. | `users`, `laundries`, `memberships`, `client_accounts` |
| **bill_service** | 8001 | The operational core: gate passes, deliveries, bills, shop bills, linens and tag lifecycles, returns, monthly matrix, dispatch, day close and dashboards. | Operational collections in the operations database |
| **laundry-management** | – | Accounting: customers, transactions, payments, expenses, salary, employees and priced items. Feeds financial reporting and the payment ledger. | Accounting database |
| **quotation-service** | – | Quotation drafting, pricing line items and the AI chat surface used to assemble offers. | Quotation collections |
| **worker-service** | – | Worker records, assignment and attendance. Supplies worker lists to the laundry app. | Worker collections |
| **love-ai** | – | AI assistance: drafting, summarisation and conversational help over permitted data. Invoked by the other services and by the laundry app. | No business data of record |

## Division of responsibility

**user-service** is the only issuer of tokens and the only authority on who a caller is. It answers three questions for every other service: is this token genuine, which laundry is this session acting in, and what may this caller do. It stores the membership snapshot that is copied into the token, so downstream services do not need a database lookup to resolve tenancy.

**bill_service** is where the business actually happens. Gate passes are created here, linen lines are attached here, deliveries are recorded here, and bills are raised here. It is also the service the shop counter and hotel desk call, and the one that must remain available for the business to trade.

**laundry-management** holds the money in its accounting form: customers, transactions, payments, expenses, salaries, employees and items. It is separate from the operational ledger because accounting records have different retention, correction rules and audience. A payment recorded operationally is mirrored into accounting, never edited in place in either system.

**quotation-service** owns the pre-sale conversation. A quotation is a priced proposal with line items and validity; it can become work, but it never becomes a bill by itself. **worker-service** owns people and attendance. **love-ai** holds no data of record: it reads through the services that do, and anything it drafts must be saved through a normal write path with a normal capability check.

## Ports and addressing

The ports above are development defaults. In a deployed environment the reverse proxy terminates TLS and routes by path prefix, and the services listen on the loopback interface only. A service port is never published to the internet, and no front end hard-codes one: service base URLs are supplied at build time through Vite environment variables.

TABLE 5 — SERVICE-TO-SERVICE DEPENDENCIES

| CALLER | CALLEE | PURPOSE | FAILURE MODE |
|---|---|---|---|
| **All services** | user-service | Token verification and capability resolution | Fail closed |
| **shop-app, hotel-app, linen-room-app** | bill_service | Operational reads and writes | Surface unavailable |
| **bill_service** | laundry-management | Mirror payment and customer records | Queue and retry |

> WARNING
>
> Service-to-service calls are authenticated with the calling user's token, not with a privileged backdoor credential. If the user could not perform the action directly, the platform does not perform it on their behalf through an internal call.

## 2.3 Front-end applications

Four React applications share a toolchain and a design language but not a codebase. Each is built with React 19, TypeScript, Vite 8 and Tailwind CSS 4, and each is deployed as static assets behind the same reverse proxy. Two additional surfaces — the guest-billing print view and the public marketing site — are served alongside them.

TABLE 6 — FRONT-END APPLICATIONS, SESSIONS AND AUDIENCES

| APPLICATION | DEV PORT | SIGN-IN | SESSION KEY | PRIMARY AUDIENCE |
|---|---|---|---|---|
| **quotations-ui** LAUNDRY APP | 5173 | Staff sign-in; existing application, extended for the new modules | – | Owners, managers, supervisors, office staff |
| **shop-app** SHOP COUNTER | 5174 | `POST /auth/login` against user-service | `ll_shop_session` | Counter staff and shop supervisors |
| **hotel-app** HOTEL DESK | 5175 | `POST /client/auth/login` against user-service | `ll_hotel_session` | Hotel front-office and housekeeping managers |
| **linen-room-app** LINEN ROOM | 5176 | Staff sign-in with linen-room roles | `ll_linen_session` | Linen supervisors and housekeepers |
| **public.lovelaundry.lk** | – | None — public marketing site | – | Prospective clients |
| **LoveMobi** | – | Staff sign-in (Flutter client) | – | Owners and managers on the move |

## Shared conventions

- **Session storage.** Each application stores its token under its own key — `ll_shop_session`, `ll_hotel_session`, `ll_linen_session` — so that two applications open in the same browser cannot overwrite each other's session, and so that a token issued to one audience is never silently reused by another.
- **Token in the header.** Every request attaches `Authorization: Bearer <token>`. No application puts a token in a query string, and none stores a token in a cookie readable by scripting.
- **No client-side authorisation.** Buttons are hidden when a capability is absent, but the check that matters is on the server. Hiding a control is a usability measure, not a security control.
- **Fail visible.** When a call fails with 401 the application clears its session and returns to sign-in; when it fails with 403 it shows a permission message and leaves the session intact; when it fails with 5xx it shows a retry affordance and does not silently retry a write.

### Build and release

Each application builds to a static bundle with `vite build` and is served from the reverse proxy under its own path. Build-time configuration is injected through Vite environment variables (`VITE_*`), which means an artefact is environment-specific: the same commit built for staging and for production produces different bundles. Release therefore always involves a build, never a file copy between environments.

> NOTE
> The three new applications — shop-app, hotel-app and linen-room-app — share a common scaffolding and authentication pattern but are separate deployments. A defect in one does not take down the others, and an upgrade can be rolled out one counter at a time.

TABLE 7 — FRONT-END TOOLCHAIN

| CONCERN | CHOICE |
|---|---|
| **UI library** | React 19 with function components and hooks |
| **Language** | TypeScript, strict mode |
| **Build tool** | Vite 8 — development server with HMR, production static bundle |
| **Styling** | Tailwind CSS 4 with a shared colour and spacing token set |
| **HTTP** | Fetch API with a shared wrapper for auth headers, error normalisation and timeouts |
| **State** | Local component state plus a small session context; no global store |
| **Printing** | Dedicated print routes with print-only stylesheets; bills and PDFs via guest-billing |

## 2.4 Request lifecycle

Every authenticated request follows the same seven-stage path. The order matters: identity is established before tenancy, tenancy before authorisation, and authorisation before any database access. A request that fails at any stage never reaches the next one.

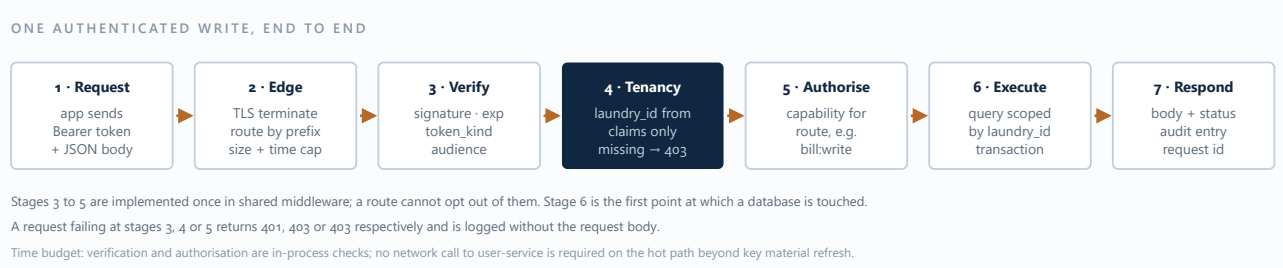



FIGURE 2 — REQUEST LIFECYCLE FOR AN AUTHENTICATED WRITE. TENANCY RESOLUTION PRECEDES AUTHORISATION, AND NO QUERY IS BUILT BEFORE BOTH HAVE SUCCEEDED.

TABLE 8 — STAGE-BY-STAGE FAILURE BEHAVIOUR

| STAGE | ENFORCED BY | ON FAILURE |
|---|---|---|
| **1 Request** | Client HTTP wrapper | No token → the app redirects to sign-in without calling the API. |
| **2 Edge** | Reverse proxy | Oversized body → 413; TLS failure → connection refused; timeout → 504. |
| **3 Verify** | Auth middleware | Bad signature, expired token or wrong `token_kind` → 401 with a generic message. |
| **4 Tenancy** | Tenant dependency | No `laundry_id` in claims → 403. There is no unscoped fallback and no default tenant. |
| **5 Authorise** | Capability dependency | Capability not held for this laundry → 403, logged with the capability name. |
| **6 Execute** | Repository layer | Cross-tenant reference, stale version or duplicate key → 409 or 422; nothing is written. |
| **7 Respond** | Serialiser and logger | Serialisation error → 500 with a request id; the audit entry records the failure. |

The practical consequence is that a handler never has to ask “which laundry is this?”. It receives a resolved tenant context, or it is never called. That single decision removes an entire class of defect — the query that forgets its filter — because there is no filter to forget: the repository takes the tenant as a mandatory argument.

## 2.5 The three-database model

The platform separates its data into three MongoDB databases along ownership lines. The split is not cosmetic: it constrains what each service can see, it lets each store be backed up and restored on its own schedule, and it keeps a reporting query from ever competing with a counter sale for the same lock or connection pool.

### Identity and tenancy — user_db

Holds `users`, `laundries`, `memberships` and `client_accounts`. It is small, changes rarely, and is read on every sign-in. Its integrity is a precondition for everything else: if identity is wrong, every downstream decision is wrong. It is written only by user-service.

### Operations — the bill_service database

Holds `gate_passes`, `deliveries`, `bills`, `shop_bills`, `linens`, `returns`, day-close records, dispatch records and the materialised monthly matrix. This is the largest and busiest store, written by every front end and read by dashboards, and it is the one whose loss stops the business trading.

**Accounting — laundry-management**

Holds `customers`, `transactions`, `payments`, `expenses`, `employees`, `items` and salary records. It follows accounting conventions: records are posted, not edited; periods are closed; and the ledger is the source of financial truth even when an operational figure disagrees.

## Rules that hold the model together

- **One writer per collection.** A service writes only its own collections. There is no shared collection and no "just this once" cross-write, because the second writer is always the one that skips a validation.
- **No cross-database joins.** Where a document needs data from another domain, it stores the identifier and the small denormalised fragment it needs at write time — a client name, a rate, a worker name — and treats that fragment as a snapshot, not a live reference.
- **References are validated at the boundary.** Creating a delivery that references a gate pass checks that the pass exists *in the same tenant*; a valid identifier from another laundry is rejected exactly like a fabricated one.
- **Restore order is fixed.** user_db first, operations second, accounting third — because operations records reference identity and accounting records reference operations. Chapter 10 uses this order in every restore procedure.
- **Backups are per database.** Each store has its own RPO and retention, reflecting how much history each one must be able to reconstruct.

## Why not one database?

A single database would be simpler to operate and harder to reason about. Three concerns drive the separation. First, blast radius: a runaway reporting aggregation in the operations store must not degrade sign-in. Second, retention: accounting records must survive longer and be restorable to a specific date without replaying operational churn. Third, authority: when an operational balance and an accounting balance disagree, the platform needs an unambiguous rule about which one is corrected — and that rule is only credible if the two live in different stores.

> NOTE
> The three databases may run on the same MongoDB replica set in a small deployment. Separation is logical first and physical second; the ownership rules apply either way, and a future move to separate clusters requires no application change.

TABLE 9 — STORE CHARACTERISTICS

| STORE | WRITE RATE | GROWTH DRIVER | RESTORE PRIORITY | LOSS IMPACT |
|---|---|---|---|---|
| **user_db** | Low | Users and memberships | 1 — first | Nobody can sign in; total outage |
| **Operations** | High | Daily passes, deliveries, scans | 2 — second | Trading stops; unbilled work |
| **Accounting** | Medium | Transactions and payroll | 3 — third | Reporting and settlement impaired |

## 2.6 Data ownership

Ownership is the answer to two questions: which service may write this data, and which service must be consulted when it changes? The table below is the reference used during design reviews and incident triage.

TABLE 10 — COLLECTION OWNERSHIP AND KEY FIELDS

| COLLECTION | OWNER | TENANT KEY | KEY FIELDS AND NOTES |
|---|---|---|---|
| **users** | user-service | – | Identity: email, password hash, display name, status. Global to the platform, not tenant-scoped; authorisation comes from memberships. |
| **laundries** | user-service | `_id` | The tenant record: name, status, plan, contact details. Its `_id` is the `laundry_id` used everywhere else. |
| **memberships** | user-service | `laundry_id` | User ↔ laundry link with role and capability set. Copied as a snapshot into the token at sign-in. |
| **client_accounts** | user-service | `laundry_id` | Hotel logins: email, password hash, `client_name`, status. Carries no staff role and no membership. |
| **gate_passes** | bill_service | `laundry_id` | Client name, pass number, dates, item lines and quantities, status, linen tag references. |
| **deliveries** | bill_service | `laundry_id` | References gate pass lines; delivered quantities, delivery date, receiver, status. |
| **bills** | bill_service | `laundry_id` | Client name, period, line items, totals, balance, status. Immutable once issued; corrected by adjustment records. |
| **shop_bills** | bill_service | `laundry_id` | Counter sales: number, lines, tender type, cashier, day-close reference. |
| **linens** | bill_service | `laundry_id` | Tag identifier, item type, current status, lifecycle events, last scan time and user. |
| **returns** | bill_service | `laundry_id` | Returned quantities and reasons, referencing the original delivery where one exists. |
| **quotations** | quotation-service | `laundry_id` | Client name, line items, rates, validity, status. Converts to work, never directly to a bill. |
| **workers** | worker-service | `laundry_id` | Worker identity, assignment, attendance. Names are denormalised onto operational records at write time. |
| **customers, transactions, payments** | laundry-management | `laundry_id` | Accounting ledger. Payments posted here mirror operational payments; neither copy is edited in place. |
| **expenses, salary, employees, items** | laundry-management | `laundry_id` | Cost side of the business and the priced item catalogue used when raising bills. |

### Denormalisation, and why it is deliberate

Operational documents copy small fragments of other domains at write time: a gate pass stores the `client_name` string, a delivery line stores the item description and rate, a bill stores the client's billing name. This is not a violation of ownership: it is how the platform avoids cross-database joins and keeps historical documents readable after the source record changes.

The rule is that a copied fragment is a *snapshot of what was true when the document was written*. Renaming a client does not rewrite last month's invoices; it changes what future documents carry. The same applies to rates: a bill raised at an old rate remains correct at that rate, and a re-price is a new document, not an edit.

### What each service may read

- A service reads its own collections freely, always with the tenant filter applied.
- A service reads another domain's data only through that domain's HTTP interface, with the caller's own token.
- Reporting reads are the exception that proves the rule: dashboards and the monthly matrix are computed inside bill_service from its own collections, not by reaching into accounting.

> WARNING
>
> A direct database read across domains — for example, joining `bills` to `payments` inside a single aggregation — couples two services' schemas and breaks the restore-order guarantees in Chapter 10. Use the owning service's API.

## 2.7 JWT and tenancy flow

Tenancy is established once, at sign-in, and carried in the token thereafter. The sequence below shows a staff sign-in and the first authenticated read that follows it.

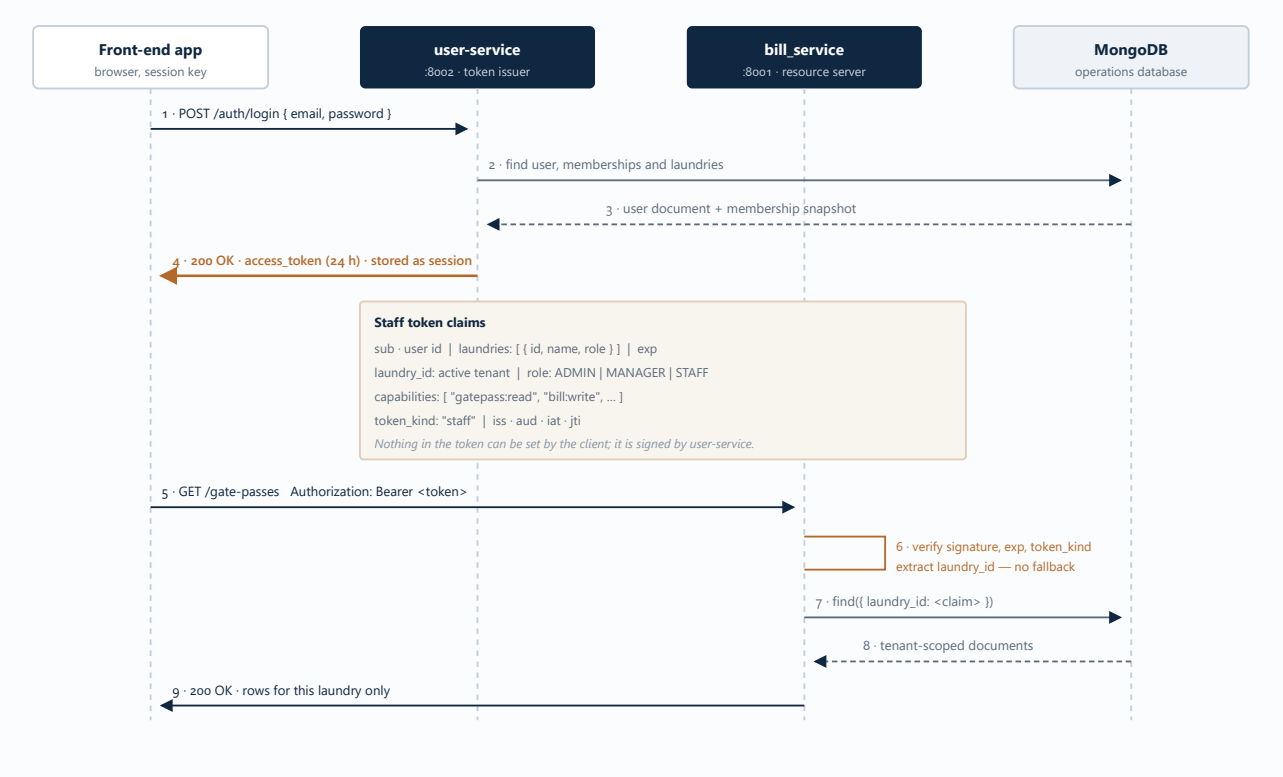



FIGURE 3 — JWT ISSUANCE AND TENANCY RESOLUTION. THE TOKEN IS ISSUED ONCE, VERIFIED ON EVERY REQUEST, AND IS THE SOLE CARRIER OF `LAUNDRY_ID`.

### What the token is, and is not

The token is a signed statement of identity, tenant and capability. It is not a session record: the services keep no session table, which is why a stolen token remains usable until it expires and why expiry is kept short — 24 hours for staff, 12 hours for hotel clients. Revocation is therefore a matter of disabling the user or the client account and rotating the signing key, not of deleting a row; Chapter 3.6 describes the procedure.

The `laundries` claim is a snapshot of the user's memberships at sign-in. It exists so that downstream services can resolve a laundry name without calling user-service, and so that a session can switch tenants only by signing in again — which is the intended behaviour, because a silent tenant switch is exactly the mistake the design is trying to prevent.

> NOTE
>
> A user with memberships in two laundries has two valid tenants, but a token names exactly one. Switching laundry re-authenticates. This is a deliberate trade of convenience for certainty.

## 2.8 Hotel client-account isolation

Hotels are the platform's most sensitive external readers, and the design that protects them is deliberately unusual. **A hotel is not an entity.** There is no `hotels` collection and no hotel tenant. A hotel is the `client_name` string carried on gate passes, deliveries, bills and quotations — a value that already existed in the operational data before the hotel portal was built.

Hotel logins live in a separate `client_accounts` collection in user-service. A client account records an email, a password hash, a status, the laundry it belongs to and the `client_name` it may see. It holds no role, no membership and no capability list, because a hotel user is not staff and must never be granted a staff permission by accident.

### How the isolation actually works

When a hotel signs in through `POST /client/auth/login`, user-service issues a token with a deliberately different shape. It carries **no `laundry_id`** and **no `laundries` claim**. Instead it carries:

```
{
  "sub": "<client account id>",
  "token_kind": "client",
  "client_scope": { "laundry_id": "<tenant>", "client_name": "<hotel name>" },
  "exp": <issued + 12 hours>
}
```

Staff endpoints resolve the tenant from `laundry_id` or from the `laundries` membership snapshot. A client token has neither, so those endpoints reject it automatically — not because someone remembered to add a check, but because the tenant dependency cannot resolve a context and therefore fails closed. There is no path by which a hotel token can reach a staff handler and be treated as a member of staff.

Hotel portal endpoints live under `/client-portal` and read `client_scope` explicitly. Every query filters on **both** `laundry_id` and `client_name`. Filtering on the name alone would be unsafe — two laundries could serve hotels with the same name — and filtering on the laundry alone would expose every hotel's data to every other hotel. Requiring both makes the pair the effective scope.

TABLE 11 — TOKEN SHAPES COMPARED

| CLAIM | STAFF TOKEN | HOTEL CLIENT TOKEN |
|---|---|---|
| **token_kind** | `"staff"` | `"client"` |
| **laundry_id** | Present — the active tenant | **Absent by design** |
| **laundries** | Membership snapshot with roles | **Absent by design** |
| **client_scope** | Absent | { laundry_id, client_name } |
| **role / capabilities** | Role plus resolved capability list | None — portal endpoints check scope only |
| **Lifetime** | 24 hours | 12 hours |
| **Accepted by** | Staff routes on all services | `/client-portal` routes only |
| **Rejected by** | `/client-portal` routes | Every staff route, automatically |

### Why not give hotels a tenant?

Making a hotel a tenant would multiply the identity surface: every hotel would need a laundry, a membership, a role and a capability set, and every operational query would need a second scoping rule to decide whether the caller was staff or client. It would also make the existing data wrong, because the operational documents already carry `client_name` and no hotel identifier.

The chosen design keeps one tenant per laundry and one extra dimension — `client_name` — that is already present in the data. The cost is that hotel accounts are managed separately from staff accounts, which Chapter 8.3 covers. The benefit is that the two classes of token cannot be confused, and that a defect in hotel-facing code cannot escalate into staff authority.

> WARNING
>
> Adding `laundry_id` to a client token to "simplify" a query destroys this isolation: the token would then satisfy the staff tenant dependency. Hotel portal code must read `client_scope`, never a top-level tenant claim.

## 2.9 Deployment topology

The reference deployment is a single application host behind a TLS-terminating reverse proxy, with MongoDB on a separate host and backups written to object storage. This is the topology the recovery procedures in Chapter 10 assume.

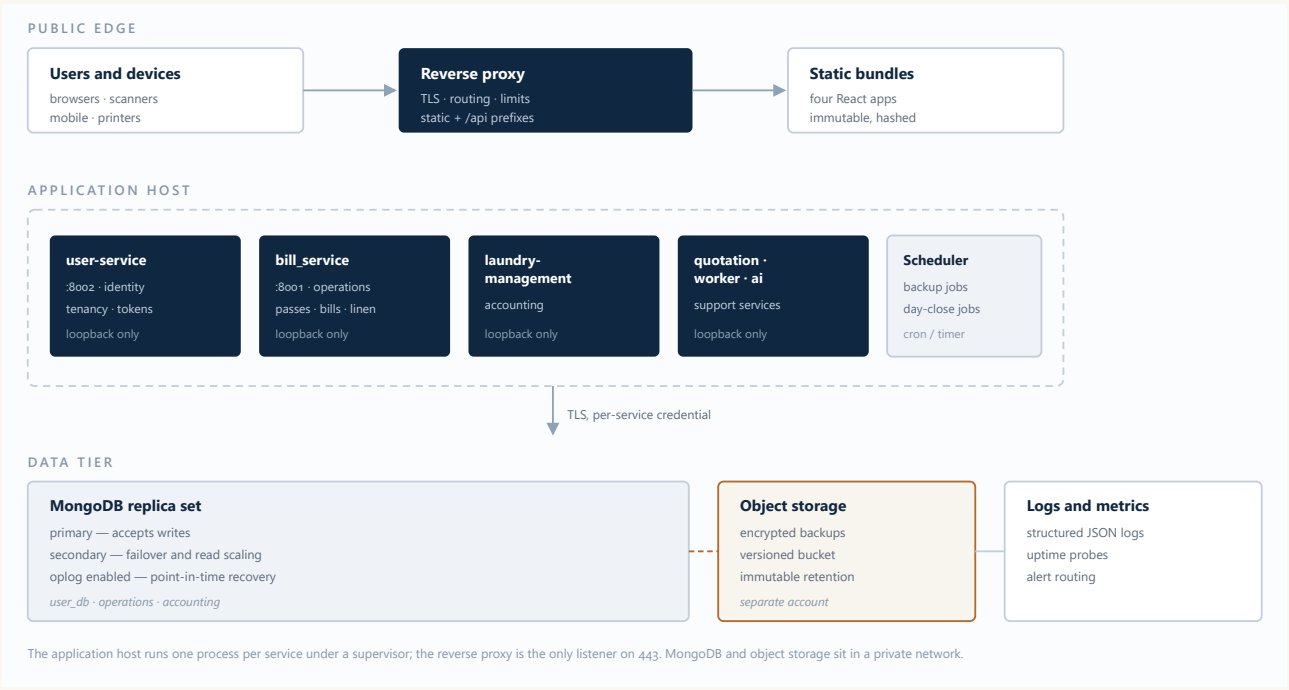



FIGURE 4 — REFERENCE DEPLOYMENT TOPOLOGY: PROXY AT THE EDGE, SERVICES ON LOOPBACK, REPLICA SET AND OBJECT STORAGE BEHIND A PRIVATE NETWORK.

## Operating assumptions

- **Single application host.** The reference deployment accepts a single host as a single point of failure for compute, mitigated by documented rebuild steps and by the fact that all state lives in MongoDB and object storage.
- **MongoDB replica set.** Three members are recommended: one primary, one secondary for failover, and one arbiter or a third data-bearing member. The oplog is what makes point-in-time recovery possible.
- **Backups off-host.** Backup files are written to an object-storage bucket in a separate account with immutability enabled, so that a compromised application host cannot delete its own history.
- **Static assets are disposable.** Front-end bundles can be rebuilt from source at any time; they are not backed up as data, only as release artefacts.

## 2.10 Technology stack

The stack is deliberately conventional. Every component is widely used, well documented and replaceable, because a laundry platform is maintained by a small team and must remain legible to whoever inherits it.

TABLE 12 — TECHNOLOGY STACK

| LAYER | TECHNOLOGY | WHY IT WAS CHOSEN |
|---|---|---|
| **Front-end framework** | React 19 | Component model that suits form-heavy operational screens; the largest available hiring and support pool. |
| **Language (front end)** | TypeScript | Types on API payloads catch the field-name mistakes that otherwise reach production as blank columns. |
| **Build tool** | Vite 8 | Fast development server, simple static output, environment injection at build time. |
| **Styling** | Tailwind CSS 4 | Consistent spacing and colour without a bespoke component library; small CSS payload. |
| **Service framework** | FastAPI (Python) | Typed request models, automatic OpenAPI schema, dependency injection that suits auth and tenancy middleware. |
| **Database** | MongoDB | Document model matches the shape of gate passes and bills, whose line items vary by client; flexible schema for a business that evolves. |
| **Auth** | JWT (signed bearer tokens) | Stateless verification at every service; claims carry tenancy so no lookup is needed on the hot path. |
| **Mobile** | Flutter (LoveMobi) | One codebase for Android and iOS for owners and managers. |
| **Hosting** | Linux host, reverse proxy, object storage | Simple, reproducible, and recoverable without provider-specific tooling. |

## Version policy

Front-end dependencies are pinned in the lockfile and upgraded deliberately, one minor version at a time, with a build and a smoke test after each step. Major upgrades — such as the move to React 19 and Tailwind 4 — are treated as projects: they are scheduled, tested against the four applications and released together, because a split across applications would leave staff switching between two visual languages.

Python service dependencies are pinned in the requirements file per service. A service may be upgraded independently, which is one of the practical benefits of the split: bill_service can stay on a proven dependency set while a lower-risk service takes a newer one.

## What the stack deliberately excludes

- **No client-side database.** The applications cache responses for resilience (Chapter 11.4) but never treat the cache as a source of truth.
- **No message broker in the reference deployment.** Cross-service side effects use durable outbox records and retries rather than introducing an operational dependency the team would have to run.
- **No server-side rendering.** The applications are internal tools behind authentication; search-engine visibility is irrelevant and a static bundle is simpler to secure.
- **No micro-front-end framework.** Four separate deployments achieve independence without a runtime composition layer.

TABLE 13 — RUNTIME AND TOOLING VERSIONS

| COMPONENT | VERSION LINE | NOTES |
|---|---|---|
| **Node.js (build)** | Current LTS | Build-time only; the runtime artefact is static files. |
| **Python** | 3.11+ | Service runtime; virtual environment per service. |
| **MongoDB** | 6.0+ | Replica set required for transactions and oplog-based recovery. |
| **React / Vite / Tailwind** | 19 / 8 / 4 | Aligned across all four applications. |

NOTE

Exact pinned versions are recorded in the lockfiles and in each service's requirements file. This manual states version lines, not patch numbers, because patch numbers change with every security update and a manual that quotes them is wrong within weeks.

2.11 · SERVICE DEPENDENCIES

## Service dependency graph

*Which service calls which, and what fails when an arrow breaks.*

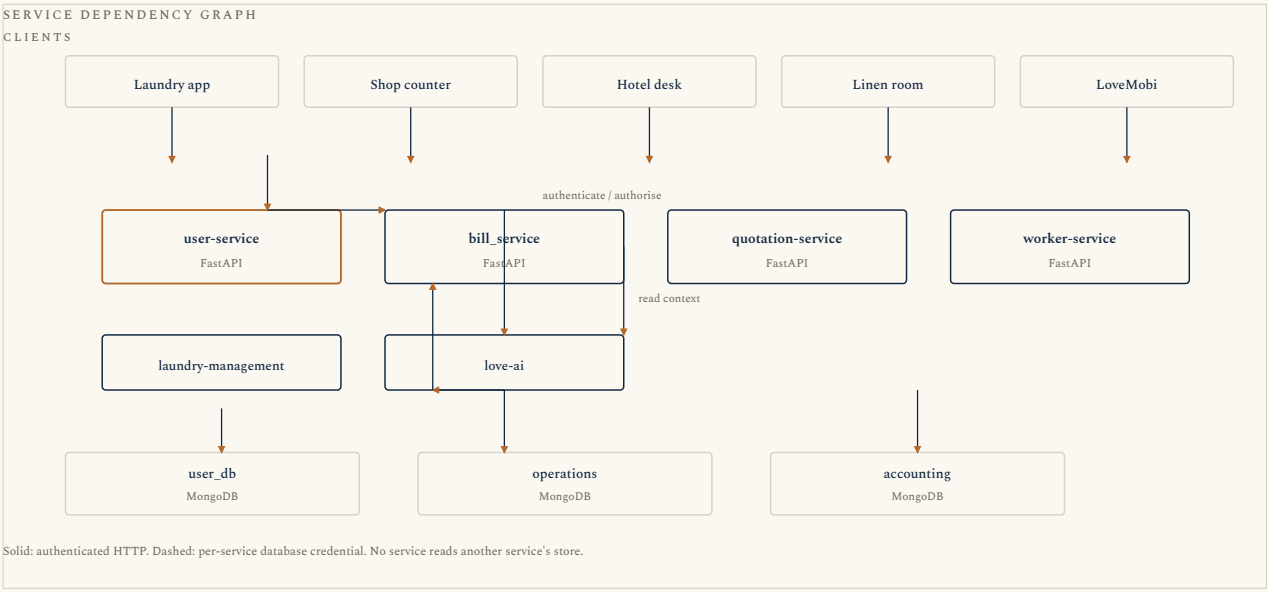



FIGURE 5 — SERVICE DEPENDENCY GRAPH, DRAWN AS A LINE DIAGRAM: AUTHENTICATED HTTP BETWEEN SERVICES, PER-SERVICE CREDENTIALS TO EACH STORE.

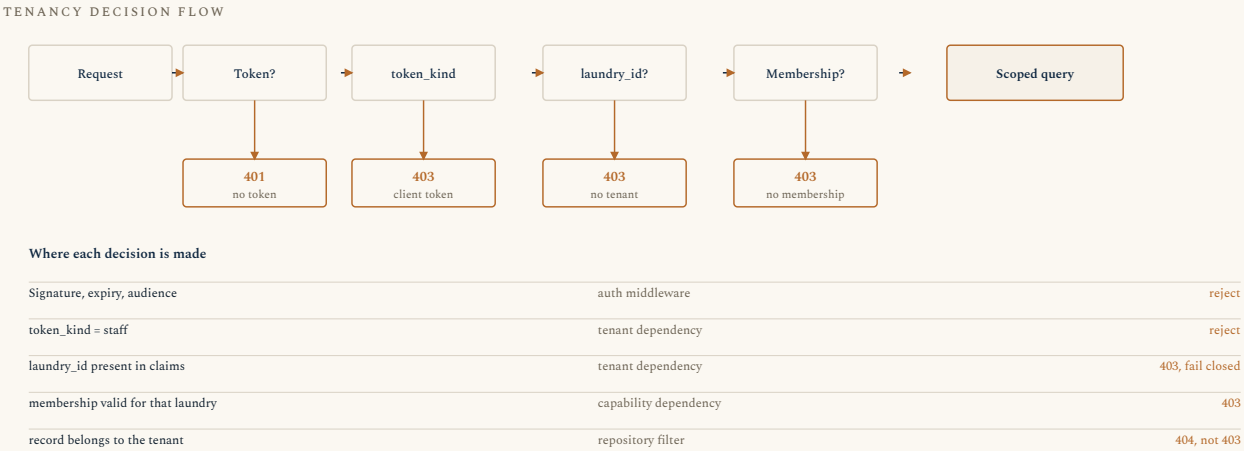
The graph is deliberately shallow. Clients talk to services; services talk to user-service for identity and to each other only where a business operation genuinely spans two domains. There is no service that every other service depends on except the identity authority, and that is the single point the platform accepts.

Reading the graph during an incident is a matter of following the arrows backwards from the symptom. A shop counter that cannot take payment is not a shop problem: the arrow from shop-app to bill_service is broken, and the first question is whether bill_service is reachable or merely slow.

2.12 · TENANCY DECISION FLOW

# Tenancy decision flow

*Five checks, in order, before a single document is read.*




**Where each decision is made**

| | | |
|---|---|---|
| Signature, expiry, audience | auth middleware | reject |
| token_kind = staff | tenant dependency | reject |
| laundry_id present in claims | tenant dependency | 403, fail closed |
| membership valid for that laundry | capability dependency | 403 |
| record belongs to the tenant | repository filter | 404, not 403 |

A refusal never reveals whether the record exists in another tenant: 404 is returned for both a foreign key and a missing one.

There is no default tenant and no system mode that skips this chain.

FIGURE 6 — THE TENANCY DECISION CHAIN. EACH FAILURE RETURNS BEFORE THE NEXT CHECK RUNS.

Chapter 3 sets out the enforcement rules in full from page 34. Tenancy is not a filter applied at the end of a query; it is a sequence of decisions that must all succeed before a query is built. The order matters: identity before token kind, token kind before tenant, tenant before membership, membership before the record itself.

The practical consequence is that a handler never asks which laundry is calling. It receives a resolved tenant context or it is never invoked, which removes the class of defect in which a developer forgets a filter.

> RULE
>
> A refusal never distinguishes a foreign record from a missing one. Both return not-found, so the endpoint cannot be used to enumerate another tenant's documents.

2.13 · FAILURE DOMAINS

# Deployment topology and failure domains

*What fails together, and why the backup must live outside the blast radius.*

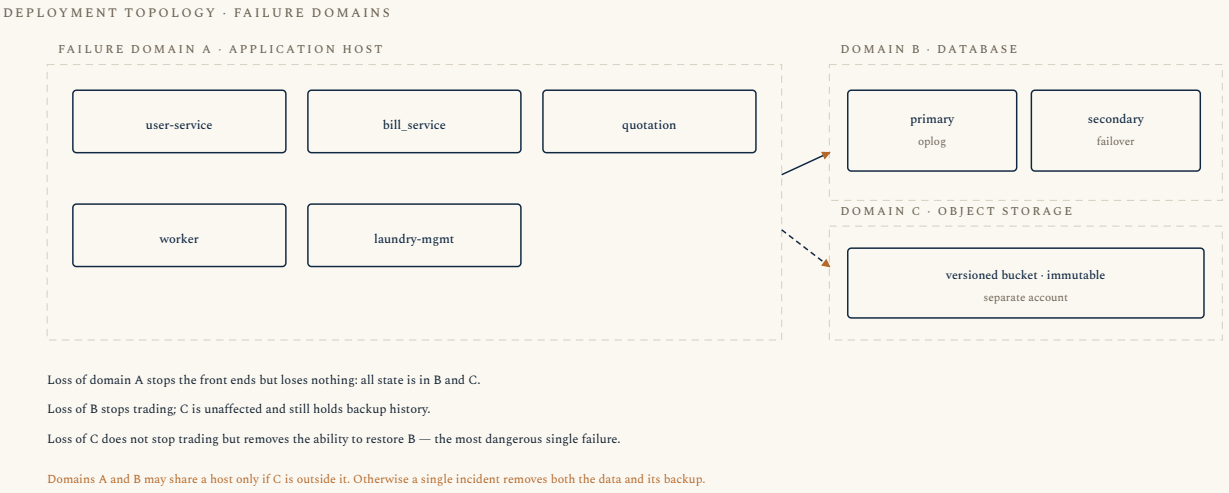



FIGURE 7 — FAILURE DOMAINS IN THE REFERENCE DEPLOYMENT: APPLICATION HOST, DATABASE REPLICA SET, OBJECT STORAGE.

The topology is drawn as three domains rather than three tiers, because the question that matters in a disaster is not what talks to what but what disappears together. The application host is disposable: it holds no state and can be rebuilt from the deployment steps.

The database is the domain that stops trade, and object storage is the domain that makes recovery possible. Keeping them in separate accounts is the control that prevents one compromised credential from removing both the data and its history.

> RULE
>
> If the database and its backups share an account, ransomware has one target, not two. Separation of accounts is a recovery control, not an administrative preference.

3

# Security Model

*Who can sign in, what they may do, and why one laundry can never read another's records.*

## 3.1 Authentication

The platform recognises three kinds of principal, and each has its own sign-in path. Staff — owners, managers, supervisors and counter staff — sign in against user-service with an email and password and receive a 24-hour staff token. Linen-room staff use the same path with linen-room roles attached to their membership. Hotel clients sign in against a separate endpoint with a separate collection and receive a 12-hour client token with a different claim shape. There is no anonymous access to any business endpoint.

TABLE 14 — AUTHENTICATION PATHS

| PRINCIPAL | ENDPOINT | LIFETIME | TOKEN CHARACTERISTICS |
|---|---|---|---|
| **Staff (laundry app, shop)** | `POST /auth/login` | 24 hours | `token_kind: "staff"`, `laundry_id`, `laundries` snapshot, role and capabilities |
| **Linen-room staff** | `POST /auth/login` | 24 hours | Same as staff, with `LINEN_SUPERVISOR` or `HOUSEKEEPER` role and linen capabilities |
| **Hotel client** | `POST /client/auth/login` | 12 hours | `token_kind: "client"`, `client_scope`, no tenant claim, no capabilities |

### Credential handling

- **Passwords are never stored.** Only a salted, slow hash is persisted. The plaintext exists for the duration of the request and is never logged, echoed or written to an audit record.
- **Sign-in responses are deliberately uniform.** A wrong password and an unknown email produce the same message and the same shape of failure, so that the endpoint cannot be used to enumerate accounts.
- **Status is checked at sign-in and at use.** A disabled user or client account cannot obtain a token; a token already issued for an account that is later disabled is rejected once the account status cache is refreshed, and immediately on the next sign-in attempt.
- **Tenant is chosen at sign-in.** If a user holds memberships in more than one laundry, the session is bound to one. Switching requires re-authentication — there is no endpoint that changes `laundry_id` inside a live token.
- **Client accounts cannot be promoted.** There is no code path that converts a `client_accounts` record into a membership, and no endpoint that issues a staff token for a client account.

### Session handling in the front ends

Each application keeps its token under its own key: `ll_shop_session` for the shop, `ll_hotel_session` for the hotel desk and `ll_linen_session` for the linen room. Separate keys mean that a supervisor who opens the shop app and the linen app in the same browser holds two independent sessions and cannot accidentally act in the wrong one. On a 401 the application clears its own key and returns to sign-in; it does not retry with the same token and it does not fall back to another stored session.

> WARNING
>
> Token lifetime is a security control, not a convenience setting. Extending the staff lifetime to “avoid signing in every morning” lengthens the window in which a stolen token is usable, because there is no server-side session to revoke. Disable the account or rotate the signing key instead.

### What authentication does not do

Authentication establishes who is calling. It does not establish what they may do — that is authorisation, covered next — and it does not by itself confine them to a tenant, which is why the tenant dependency runs as a separate stage and fails closed. A valid token with no resolvable tenant is useless: it authenticates a caller the platform then refuses to serve.

## 3.2 Roles and capabilities

Authorisation is capability-based. A role is a named bundle of capabilities, and every protected endpoint declares the capability it requires. Adding a role means adding a bundle; it never means adding an `if role == …` branch inside a handler. The complete matrix is in Appendix B; this section explains the roles and the reasoning behind the bundles.

TABLE 15 — ROLES, SCOPE AND INTENT

| ROLE | APPLIES TO | INTENT |
|---|---|---|
| **PLATFORM_ADMIN** | Platform | Superset of ADMIN. Creates laundries, provisions the first administrator of each tenant, and can act across tenants for support purposes. Held by the platform operator, not by laundry staff. |
| **ADMIN** | One laundry | Full control of a single tenant: users and memberships, hotel client accounts, configuration, day close, all operational and financial modules. |
| **MANAGER** | One laundry | Runs the operation: gate passes, deliveries, bills, payments, quotations, reports and day close. May not create users or change tenant configuration. |
| **STAFF** | One laundry | Day-to-day operational entry: gate passes, deliveries, bills and read-only dashboards. No payment posting, no reports, no administration. |
| **HOTEL_ADMIN** | One client account | Hotel-side administrator: sees the property's full portal, including invoices and statements. Cannot see any other client of the laundry. |
| **HOTEL_MANAGER** | One client account | Hotel-side operational reader: gate passes, deliveries and invoices for the property. Statements are available; account settings are not. |
| **LINEN_SUPERVISOR** | One laundry | Owns the linen room: scanning, status transitions, inventory, discrepancies, tag printing and the retirement of items. |
| **HOUSEKEEPER** | One laundry | Scans collection and receipt, and moves items through wash and press. Cannot retire, mark missing or print tags. |

## Capabilities

Capabilities are strings of the form `resource:action`. They are carried in the token, checked by a dependency on each route, and used by the front ends to decide which controls to render. The set in use is:

TABLE 16 — CAPABILITY REFERENCE

| CAPABILITY | GRANTS | NOTES |
|---|---|---|
| `gatepass:read` | List and view gate passes | Scoped to the session tenant and, for hotel tokens, to `client_name` |
| `gatepass:write` | Create, edit, cancel gate passes | Cancel is a status change, never a delete |
| `delivery:read` | List and view deliveries | Includes partial deliveries |
| `delivery:write` | Record and amend deliveries | Amendments require a reason and are audited |
| `bill:read` | View bills and balances | Hotel portal uses scope, not this capability |
| `bill:write` | Raise bills and adjustments | Issued bills are immutable |
| `payment:read` | View payments and receipts | |
| `payment:write` | Post payments and reversals | Never granted to STAFF |
| `linen:read` | View tags, inventory and discrepancies | |
| `linen:write` | Change tag status, retire items | Retirement is supervisor-only |
| `linen:scan` | Submit scan events | Separate from write so scanners can be restricted |
| `dashboard:read` | View operational dashboards | Totals respect tenant scope only |
| `report:read` | View reports and the monthly matrix | Manager and above |

### Why capabilities rather than roles in code

Role names are a poor thing to check inside a handler, because the business will always want one more variation — a senior housekeeper who may retire items, a counter supervisor who may post payments. With capabilities, that variation is a bundle change in one place, and the audit answer is a capability the user held rather than a job title interpreted somewhere in code.

## 3.3 Tenant isolation rules

Tenant isolation is the single most important property of the platform, and it is enforced by construction rather than by discipline. The rules below are stated as invariants: each one is either true of every request or the request fails.

TABLE 17 — TENANCY INVARIANTS

| # | INVARIANT | HOW IT IS ENFORCED |
|---|---|---|
| 1 | The tenant is read only from verified JWT claims. | Tenant dependency reads `laundry_id` from the decoded token. No request field is consulted for tenancy. |
| 2 | Every query carries the tenant filter. | Repositories take the tenant as a required argument; there is no repository method that omits it. |
| 3 | A missing tenant fails closed. | No claim → 403 before any handler runs. There is no default tenant and no unscoped fallback. |
| 4 | Cross-tenant references are rejected. | Creating a delivery against a gate pass from another laundry returns 404 or 422 — the referenced document is treated as non-existent. |
| 5 | Identifier lookups are tenant-scoped. | A valid identifier from another tenant behaves exactly like a fabricated one; the query never runs without the filter. |
| 6 | Aggregations and reports are scoped. | Dashboard and matrix pipelines begin with a `$match` on `laundry_id`, never with a global scan. |
| 7 | Hotel reads require both keys. | `/client-portal` queries filter on `laundry_id` *and* `client_name` from `client_scope`. |
| 8 | Staff routes reject client tokens. | Client tokens carry no `laundry_id` and no `laundries` claim, so the tenant dependency cannot resolve and the request fails. |

### How a violation surfaces

The platform deliberately does not distinguish “this record belongs to someone else” from “this record does not exist”. Both produce the same 404 or 422. The reason is that a different response would confirm the existence of another tenant's data, which is itself a leak: an attacker could enumerate identifiers and learn how many gate passes a competitor processes.

At the query level the filter is not applied as a post-processing step. A request for `/gate-passes/{id}` compiles to a lookup on `{ _id: id, laundry_id: <claim> }`. If the identifier exists in another tenant, the query returns nothing, the handler raises not-found, and no document is ever loaded into memory.

### Where tenancy is checked, end to end

- **At sign-in:** user-service resolves memberships and refuses to issue a token for a laundry the user does not belong to.
- **On every request:** the tenant dependency extracts and validates the claim, then exposes a tenant context object to the handler.
- **In every repository call:** the tenant is part of the query signature, so a handler cannot call a data access method without supplying it.
- **In every write:** new documents have `laundry_id` stamped from the context, never from the payload. A payload that includes `laundry_id` is rejected rather than trusted.
- **In cross-domain calls:** the caller's token is forwarded, so the callee applies the same claim and cannot be tricked into a wider scope.

### Testing the invariant

Tenancy is covered by explicit tests in both service suites: a request without a tenant claim must fail, a request with a foreign identifier must return not-found, a payload carrying `laundry_id` must be rejected, and a client token must be refused by staff routes. These are regression tests, not exploratory ones — they exist because the invariant is the product's central promise, and a refactor that quietly weakens it would otherwise pass every functional test.

> WARNING
>
> Never add a “system” or “service” flag that skips the tenant dependency. If an internal job needs cross-tenant access, it must iterate tenants explicitly, one scoped operation at a time, so that each unit of work has a named tenant in its logs.

## 3.4 Encryption at rest

Sensitive fields are encrypted before they reach MongoDB, using envelope encryption with AES-256-GCM. The database stores ciphertext, an initialisation vector and an authentication tag; it never stores the key that could decrypt them. The design has three parts: a per-document data key, a key-encryption key that wraps it, and a deterministic search token that allows equality lookups without decryption.

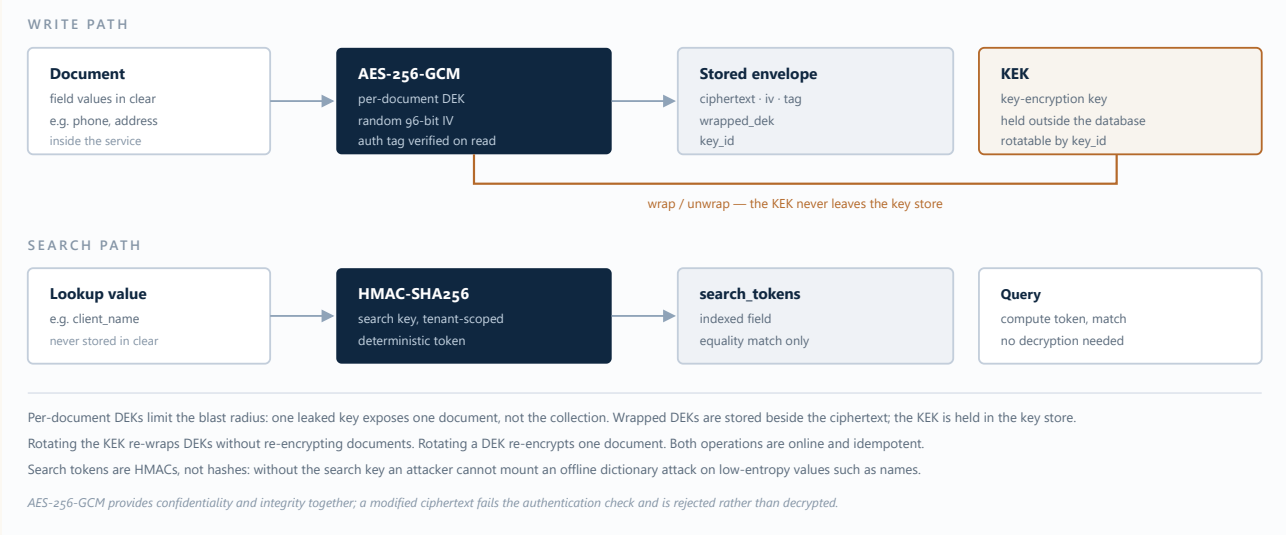



FIGURE 8 — ENVELOPE ENCRYPTION AT REST, WITH A SEPARATE DETERMINISTIC PATH FOR EQUALITY SEARCH.

## The three parts

**Per-document data key.** Each document that contains sensitive fields gets its own 256-bit data encryption key. The key is used once to encrypt the sensitive values with AES-256-GCM, a fresh 96-bit initialisation vector per encryption, and the resulting authentication tag is stored with the ciphertext. Reading the document requires the tag to verify; a tampered field fails rather than returning plausible garbage.

**Key-encryption key.** The data keys are themselves encrypted — wrapped — by a key-encryption key that is held outside the database, in the environment's key store or secret manager. The database therefore contains wrapped keys that are useless without the KEK. A stolen database backup yields ciphertext and wrapped keys, not plaintext.

**Search tokens.** Encrypted values cannot be matched with a `$regex` or a range query, and the platform does not pretend otherwise. Where equality lookup is required — a client name, an account reference — the service also stores an HMAC-SHA256 token computed with a tenant-scoped search key. Queries match the token, never the ciphertext. Because the input space of names is small, a plain hash would be reversible by dictionary attack; an HMAC with a secret key is not.

## What is encrypted, and what is not

Encryption is applied to personal and commercial identifiers: client contact details, addresses, account references and free-text notes that may contain personal data. Operational keys — identifiers, dates, statuses, quantities and amounts — remain in clear because they are required for indexing, sorting, aggregation and reconciliation, and because encrypting them would make the monthly matrix impossible without decrypting every document in memory. The boundary is deliberate and documented so that a future reviewer does not assume more protection than exists.

> KEY CUSTODY
>
> Losing the KEK loses the encrypted fields permanently; there is no recovery path by design. The KEK is therefore backed up separately from the database, under a different access policy, and its escrow is tested in the restore drill described in Chapter 10.7.

## 3.5 Transport, audit and rate limiting

### Transport

All traffic between clients and services, and between services and MongoDB, is encrypted in transit. The reverse proxy terminates TLS 1.2 or 1.3 with modern cipher suites, redirects plain HTTP to HTTPS, and sets HTTP Strict Transport Security so that a browser refuses to downgrade on subsequent visits. Certificates are issued for the platform's hostnames and renewed automatically; expiry is monitored as an alert.

Internal calls stay inside the private network and are authenticated with the caller's token. MongoDB connections use TLS with a per-service credential, so a service can read only the database it owns. The only public listener is the proxy on port 443.

### Audit

Security-relevant events are recorded with the actor, the tenant, the action, the target and the outcome. Audit records are append-only and are written even when the action fails, because a rejected attempt is often more interesting than a successful one.

TABLE 18 — AUDITED EVENTS

| EVENT CLASS | RECORDED DETAIL |
|---|---|
| **Sign-in success and failure** | Email, tenant where resolvable, source address, user agent, outcome. Failures never include the submitted password. |
| **Token rejection** | Reason class only — expired, bad signature, wrong token kind, unresolvable tenant — with the route and request id. |
| **Authorisation denial** | Required capability, the caller's role, the route, the tenant. |
| **Privileged writes** | User creation, role change, membership change, client account creation or status change, day close, bill issue, payment post and reversal. |
| **Data correction** | Amendment of a delivery, cancellation of a gate pass, adjustment of a bill — with reason and before/after summary. |
| **Administrative access** | Backup restore, key rotation, environment change, tenant creation. |

Audit entries are structured JSON written to the log pipeline with a retention period longer than the operational data they describe. They are not editable through any application route: no endpoint updates or deletes an audit record, and the application's storage credential is append-only.

### Rate limiting

Rate limits protect the sign-in endpoints from credential stuffing and the write endpoints from runaway clients — a scanner stuck in a retry loop, or a page that resubmits a bill. Limits are applied per source address and per account at the proxy and again in the service, so that a misconfigured proxy does not remove the protection.

TABLE 19 — RATE-LIMIT CLASSES

| CLASS | APPLIES TO | BEHAVIOUR ON BREACH |
|---|---|---|
| **Authentication** | `/auth/login`, `/client/auth/login` | 429 with a retry hint; repeated breaches escalate to a temporary block and an alert |
| **Read** | List and detail routes | 429; clients back off with jitter |
| **Write** | Creates, updates, payments | 429 before the handler; no partial write occurs |
| **Scan** | Linen scan submissions | Burst allowance for rapid scanning, then 429 with the remaining budget reported |
| **Export and report** | Monthly matrix, statements, PDF generation | Queued rather than rejected where the operation is long-running |

Thresholds are configuration values rather than code constants, so that a busy Saturday can be accommodated without a deployment. They are recorded with the rest of the environment configuration in Chapter 8.4, and any change to them is an audited administrative action.

> WARNING
>
> Rate limiting is not a substitute for authorisation. A caller who is under the limit and holds the wrong capability must still be refused; the two controls answer different questions.

## 3.6 Secrets management

The platform depends on a few high-value secrets: the JWT signing key, the key-encryption key, the search key for HMAC tokens, database credentials for each service, and the object-storage credential used by backups.

### Rules

- **No secret in source control.** Configuration files in the repository contain placeholders only. A committed secret is treated as compromised and rotated, even if the commit is reverted.
- **No secret in a front-end bundle.** Vite exposes every `VITE_*` variable to the browser.
- **One credential per service.** Each service has its own database user, limited to its own database. bill_service cannot authenticate against the accounting database, which converts a service compromise into a bounded incident.
- **Secrets are injected at start-up.** They arrive from the environment or a secret manager, never from a file inside the deployment directory, and they are never written to logs.
- **Redaction is default.** The logging configuration redacts `authorization`, `password`, `token`, `secret` and `cookie` at any depth, so a careless `log.info(payload)` cannot leak a credential.
- **Rotation is scheduled and tested.** Signing keys and database credentials rotate on a defined calendar (Chapter 13.4); the rotation procedure is exercised before it is needed.

### Rotating the JWT signing key

Because tokens are stateless, rotation must be staged so that valid sessions are not all terminated at once — unless termination is the intent.

1. Generate the new key and add it to the key set as the *next* signing key, keeping the current key for verification.
2. Deploy the updated key set to every service. Verification now accepts both keys; signing still uses the old key.
3. Confirm that all services report the new key set version — a service that missed the deployment will reject newly signed tokens.
4. Switch signing to the new key. Existing sessions continue to verify against the old key until they expire.
5. After the longest token lifetime has elapsed (24 hours for staff), remove the old key from the set.
6. Record the rotation in the audit log with the operator, the time and the key identifier.

> EMERGENCY ROTATION
>
> If a signing key is believed compromised, skip the staged overlap: remove the old key immediately, accepting that every session is invalidated and every user must sign in again. A forced sign-in is a minor inconvenience; an attacker minting staff tokens is a Severity 1 incident.

### Rotating the KEK and search key

The KEK is rotated by introducing a new key identifier and re-wrapping each document's data key with it. Documents are not re-encrypted, so the operation is fast and can run in the background; each document records the `key_id` of the KEK that wrapped its DEK, which makes the migration resumable and safe to interrupt. The old KEK is retained until every document reports the new identifier, then retired.

The search key is harder to rotate because its output is stored: changing the key invalidates every existing token. Rotation therefore requires a backfill that recomputes tokens for all affected documents, performed during a maintenance window with writes paused for the affected collection. Chapter 13.4 schedules this annually rather than continuously, because the cost is real and the benefit is marginal unless the key is suspected compromised.

### Response to a suspected secret leak

1. Identify which secret is exposed and what it grants: a signing key forges sessions; a database credential reads a database; a storage credential reads or deletes backups.
2. Revoke or rotate the secret first, then investigate. Containment precedes understanding.
3. Preserve evidence: snapshot logs and audit records before rotating, because rotation may invalidate the trail's context.
4. Determine the exposure window and the records touched, using audit entries and database access logs.
5. Notify according to Chapter 9.6 if personal data may have been accessed.
6. Record the incident and the rotation in the post-incident review, and add a control that would have prevented it.

3.7 · TOKEN SHAPES COMPARED

## Token shapes compared

*Two kinds of session, deliberately unequal.*

TOKEN SHAPES COMPARED

| STAFF TOKEN | | CLIENT TOKEN | |
|---|---|---|---|
| **sub** | user id | **sub** | client account id |
| **token_kind** | "staff" | **token_kind** | "client" |
| **laundry_id** | active tenant | ~~**laundry_id**~~ | *absent by design* |
| **laundries**[] | membership snapshot | ~~**laundries**[]~~ | *absent by design* |
| **role** | ADMIN \| MANAGER \| STAFF | ~~**role**~~ | *none* |
| **capabilities**[] | gatepass:write … | ~~**capabilities**[]~~ | *none* |
| ~~**client_scope**~~ | *absent* | **client_scope** | { laundry_id, client_name } |
| **exp** | 24 hours | **exp** | 12 hours |

A staff token satisfies the tenant dependency. A client token cannot: it carries no top-level tenant claim.

The absence is the control, not a gap to be filled. Adding laundry_id to a client token would let it pass staff checks.

**FIGURE 9** — CLAIM-BY-CLAIM COMPARISON OF STAFF AND HOTEL CLIENT TOKENS.

The two tokens are not variants of one design; they are different instruments. A staff token asserts a person, a tenant and a set of capabilities. A client token asserts a property and a scope, and nothing else — no tenant claim, no role, no capability list.

The asymmetry is what makes the isolation automatic. Staff endpoints resolve a tenant from the token, so a token without one cannot pass, no matter which route it is sent to. Hotel endpoints read the scope object, so a staff token cannot satisfy them either.

> RULE
>
> Adding a tenant claim to a client token to simplify a query would collapse the distinction and let a hotel satisfy a staff authorisation check. The absence is the control.

II

PART TWO

# The Applications


4 User Manual — Laundry App

5 User Manual — Shop Counter App

6 User Manual — Hotel Desk App

7 User Manual — Linen Room App

4

# User Manual — Laundry App

*The full-surface application for the laundry business: passes, deliveries, bills, quotations, workers, reports and day close.*

## 4.1 Sign-in

The laundry app is the quotations-ui application, extended for the operational modules. It is the only surface that exposes the complete commercial picture, and therefore the only one where a manager can move from a gate pass to the bill it produced and the payment that settled it.

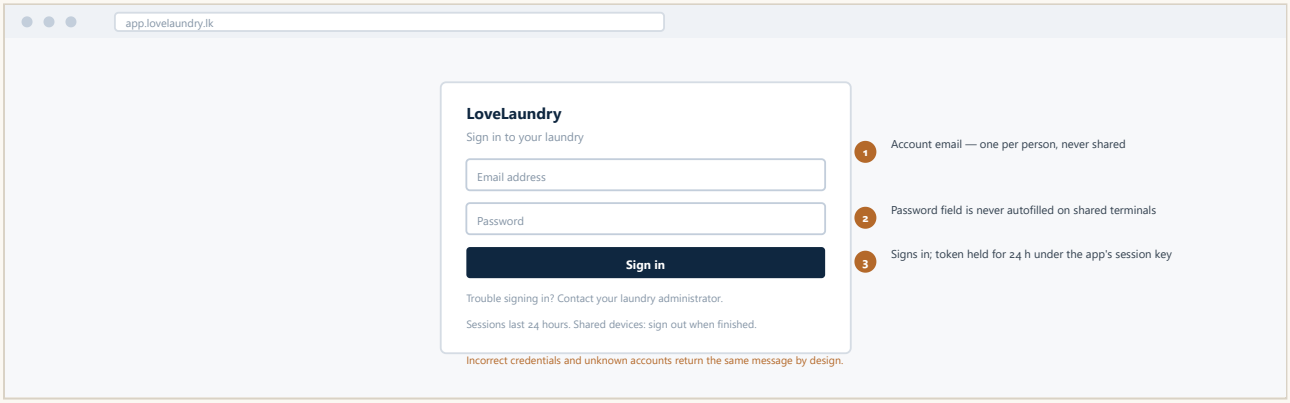



**FIGURE 10** — LAUNDRY APP SIGN-IN WITH FIELD ANNOTATIONS.

### Procedure — signing in

1. Open the laundry app in a supported browser on a device you control.
2. Enter your account email. Use your own account: the audit trail, and therefore every correction you make, is attributed to it.
3. Enter your password and select **Sign in**.
4. If you hold memberships in more than one laundry, choose the laundry you are working in for this session. The choice is bound to the session and cannot be changed without signing in again.
5. Confirm the laundry name shown in the top bar matches the business you are working for before entering any data.

> NOTE
> Sign-in failures do not distinguish between a wrong password and an unknown account. This is deliberate: a different message would let an outsider discover which email addresses have accounts.

## 4.2 Dashboards

The dashboard answers one question: what needs attention today? It is not a reporting surface — reports live under **Reports** — and it is not a workflow queue. Every tile is either a count that should be driven to zero, or a total that should match a physical count.

TABLE 20 — DASHBOARD TILES AND WHAT EACH ONE MEANS

| TILE | MEANING AND EXPECTED ACTION |
|---|---|
| **Open gate passes** | Passes received but not yet fully delivered. A number that only grows means deliveries are not being recorded, not that work is piling up. |
| **Due today** | Passes whose promised return date is today. Work these first; the tile turns amber when any item is overdue. |
| **Ready for delivery** | Items washed, pressed and marked ready in the linen room. These are physically on the rack and should leave today. |
| **Unbilled deliveries** | Deliveries recorded but not yet included in a bill. The figure should be zero at day close; anything left is revenue at risk. |
| **Outstanding balance** | Sum of unpaid balances across clients for the current period. Drill through to the client list, then to the bill. |
| **Linen discrepancies** | Open missing or damaged records. These need a supervisor decision, not a silent write-off. |
| **Today's collections** | Payments posted today, split by method. Compare with the cash drawer and the bank slip before day close. |

### Reading the dashboard honestly

Tiles are computed from the operational database on request; they are not maintained counters. That means they are always consistent with the documents behind them, and it also means a tile can change while you are looking at it if a colleague records a delivery. There is no stale cache to reason about, and no nightly job that could leave a number wrong until morning.

Each tile is a link. A count without a route to the underlying records is a number nobody can act on, so every tile opens a filtered list — open passes, due-today passes, unbilled deliveries — with the filter already applied and visible in the toolbar.

### Daily rhythm

TABLE 21 — TYPICAL USE OF THE DASHBOARD THROUGH THE DAY

| WHEN | LOOK AT | WHY |
|---|---|---|
| **Opening** | Due today, ready for delivery | Plan the routes and the wash order before the first van leaves. |
| **Midday** | Open passes, linen discrepancies | Catch missing items while the shift that handled them is still on site. |
| **Before dispatch** | Ready for delivery, due today | Confirm that what is on the rack matches what the client expects. |
| **Evening** | Unbilled deliveries, outstanding balance | Raise missing bills and chase balances before day close. |
| **Day close** | Today's collections | Reconcile cash and card totals against the drawer. |

> WARNING
>
> A dashboard that shows zero unbilled deliveries at the end of the day is only meaningful if every delivery was recorded as it happened. Recording a week of deliveries retrospectively makes the figures look correct while destroying their value for decisions.

## 4.3 Gate passes

The gate-pass register is the operational spine of the laundry. Every collection is recorded here, every delivery refers back to a pass, and every bill is built from delivered quantities. A pass records who sent what, when it arrived and when it is promised back.

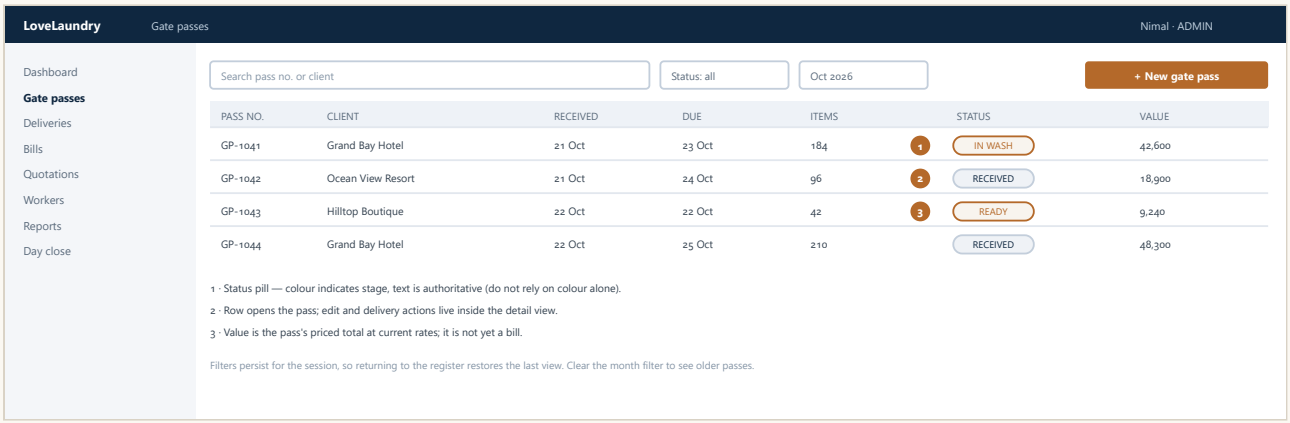



FIGURE 11 — GATE-PASS REGISTER WITH ANNOTATIONS.

### Columns and statuses

The register shows the pass number, the client, the date received, the promised return date, the item count and the current status. Statuses advance only through recorded actions: *received* when the pass is created, *in wash* when the linen room starts processing, *ready* when finishing is complete, *delivered* when a delivery is recorded against it, and *closed* when nothing further is outstanding. A pass is never deleted; a mistake is corrected by cancelling the pass with a reason, which leaves it visible in the register.

## 4.4 Creating a gate pass

A gate pass is created at the moment goods are handed over. Doing it later, from memory, is the single largest source of billing disputes, because the quantities recorded then are guesses rather than counts.

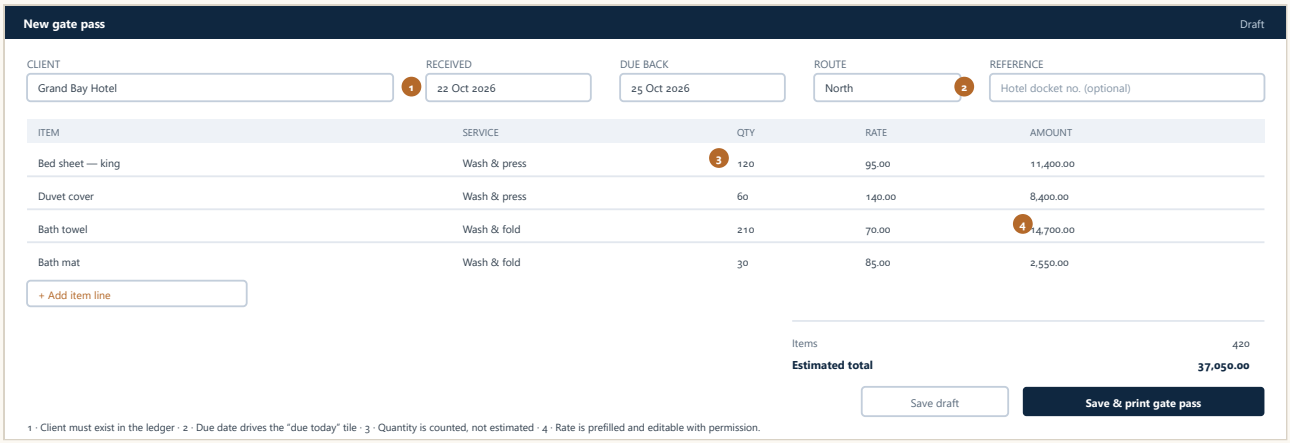



FIGURE 12 — GATE-PASS EDITOR, SHOWING HEADER FIELDS AND ITEM LINES.

### Procedure — creating a gate pass

1. From the register, select + **New gate pass**.
2. Choose the client. If the client is not listed, stop and ask an administrator to create the client record; do not type a variant of an existing name, which splits the client's history.
3. Set the date received to the actual handover date. Back-dating a pass distorts the monthly matrix and the day-close totals for both dates.
4. Set the promised return date. This is the date the client has been told; it drives the due-today worklist.
5. Add each item line: item, service, quantity and rate. Count the goods as you add them — the pass is a receipt as well as a work order.
6. Check the item count and estimated total against the physical count one more time. A typo here propagates into the delivery and the bill.
7. Select **Save & print gate pass** and hand the printed copy to the client's driver. The pass is now locked for quantity edits except by a manager, and every such edit is audited.

> WARNING
>
> Never create a pass for goods that have not arrived. A pass is evidence of possession; creating one early turns a missing delivery into a billing dispute.

## 4.5 Deliveries

A delivery records the return of finished goods against a gate pass. It is the bridge between operations and money: quantities delivered are what the client is billed for, and the delivery date determines the billing period.

### Partial and split deliveries

Clients rarely receive a whole pass at once. A hotel may take back its sheets today and its towels tomorrow, and the platform expects this. Each delivery carries its own quantities per line, and the pass remains open until the delivered quantity equals the received quantity on every line, or until a supervisor closes the balance as short-delivered with a reason.

TABLE 22 — DELIVERY RULES

| SITUATION | WHAT THE SYSTEM DOES |
|---|---|
| **Delivered < received** | Pass stays open; remaining quantity is shown as outstanding on the pass and in the due-today tile. |
| **Delivered = received** | Pass status becomes delivered; the lines become billable. |
| **Delivered > received** | Rejected. An over-delivery means either the pass quantity or the delivery count is wrong; correct the record rather than forcing the number. |
| **Short delivery closed** | Requires a reason and a supervisor. The undelivered quantity is written off explicitly and appears in the discrepancy report. |
| **Delivery corrected after billing** | Requires a credit note; the original delivery is not edited once a bill has consumed it. |

### Procedure — recording a delivery

1. Open **Deliveries** and select **Record delivery**, or open the pass and choose **Deliver** from its detail view.
2. Scan or select the gate pass. The form loads every line with the outstanding quantity prefilled.
3. Reduce quantities to what is actually being handed over. Leave a line at zero if nothing on it is going out.
4. Confirm the receiver's name — the person signing at the client — and the delivery date and time.
5. Select **Save delivery**. The pass status recalculates immediately and the dashboard tiles update.
6. Print or share the delivery note if the client requires one. The note shows quantities only, never rates.

## What a delivery is not

- **Not a bill.** Delivering goods does not raise an invoice. Bills are raised deliberately, in a batch or per client, so that the billing period is a business decision rather than a side effect of a van leaving.
- **Not a payment.** A delivery can be made on credit. The balance appears on the bill, and payment is a separate record with its own audit trail.
- **Not reversible by deletion.** A delivery recorded in error is reversed with a reason, which leaves both records visible. Deleting one would hide the fact that goods may have left the premises.

> NOTE
>
> The receiver's name matters more than it appears to. When a hotel later disputes a delivery, the name and timestamp on the note are the evidence that resolves it — which is why the field is required and is not defaulted to the signed-in user.

## 4.6 Bills

A bill is raised against a client for deliveries in a period. It states quantities, rates, charges, discounts and tax, and it carries the balance outstanding. Bills are the platform's commercial record: once issued, a bill is not edited, because an edited invoice cannot be reconciled against the copy the client holds.

### Raising a bill

1. Open **Bills** and select **Raise bill**.
2. Choose the client and the period. The form lists every unbilled delivery for that client in the period, with its pass reference.
3. Select the deliveries to include. Deliveries already on another bill are not offered, which prevents double billing at the source.
4. Review the line rates. Rates come from the client's agreed price list where one exists; a manual override requires `bill:write` and is visible on the bill as an edited rate.
5. Apply any discount or adjustment with a reason. A discount without a reason is a control failure, not a courtesy.
6. Check the totals: subtotal, discount, tax and amount due. Tax is computed from the configured rate, not typed.
7. Select **Issue bill**. The bill receives its number, becomes immutable, and the client's balance increases.
8. Print or share the bill. The print layout is produced by the guest-billing view, which shows the same figures as the screen.

TABLE 23 — BILL NUMBERING AND IDENTITY

| ATTRIBUTE | RULE |
|---|---|
| **Number** | Assigned on issue, sequential within the laundry and the financial year. Numbers are never reused, including for cancelled bills. |
| **Status** | *draft* before issue, *issued* after, *part-paid* or *paid* as payments arrive, *void* if cancelled with a reason. |
| **Immutability** | Issued bills do not change. Corrections are credit notes or adjustment records that reference the original. |
| **Currency** | One currency per laundry, set in configuration; bills do not convert. |

## Draft, issue, and the point of no return

A draft bill can be edited freely — lines added, rates corrected, the period changed. Issuing it is the point at which it becomes a financial document: the number is consumed, the client can see it in the hotel portal if they have an account, and any correction must be made by a visible adjustment rather than an edit. Staff who are unsure whether a bill is right should leave it in draft; a draft is invisible to the client, and nothing is lost by waiting.

## What goes on a bill

- Only delivered quantities. Work in progress is not billable, however far advanced it is.
- Rates that were agreed for that client, at the version in force on the delivery date.
- Adjustments from returns or short deliveries, shown as separate lines rather than netted silently into quantities.
- Tax computed by the platform from configuration, with the rate stated on the face of the bill.

> WARNING
>
> If a bill total looks wrong after issue, do not edit the bill. Raise a credit note or an additional charge and reference the original. Editing an issued bill breaks the client's copy, the statement and the audit trail at once.

4.7 · BILL DETAIL AND PAYMENTS

# Bill detail and payments

A bill's detail view is the client's account in miniature: what was billed, what has been paid, what remains, and every document that produced those figures. Payments recorded here are the only way a balance falls.

TABLE 24 — PAYMENT METHODS AND THEIR EFFECT

| METHOD | RECORDED AS | EFFECT ON BALANCE |
|---|---|---|
| **Cash** | Receipt against the bill, with drawer reference | Reduces balance immediately |
| **Card** | Receipt with terminal reference | Reduces balance on posting; settlement is a banking matter |
| **Bank transfer** | Receipt with transfer reference | Reduces balance when posted, not when the bank clears |
| **Cheque** | Receipt with cheque number and bank | Reduces balance on posting; bounced cheques are reversed |
| **Credit adjustment** | Adjustment record with reason | Reduces balance without cash; audited separately |

### Procedure — recording a payment

1. Open the bill from **Bills** or from the client's account view.
2. Select **Record payment**. The amount due is prefilled; overwrite it for a part payment.
3. Choose the method and enter the reference — drawer slip, terminal slip or transfer reference. A payment without a reference cannot be reconciled at day close.
4. Set the payment date to the date the money was received, not the date it was entered.
5. Select **Post payment**. The balance recalculates and the bill status moves to part-paid or paid.
6. Print the receipt if the client is present, and file the slip with the day's takings.

### Reversals, not deletions

A payment posted in error is reversed, never deleted. The reversal creates a negative record that references the original, so the client's statement shows both the mistake and its correction. Deleting a payment would make the day-close total disagree with the bank without explanation — the classic symptom that turns a five-minute correction into an audit.

> WARNING
>
> Applying a payment to the wrong bill is corrected by reversing it and re-posting to the correct bill. Never edit the original amount: the receipt the client holds would no longer match the record.

4.8 · QUOTATIONS

# Quotations

A quotation is a priced offer. It carries line items, rates, validity and terms, and it can be converted into operational work — but it never becomes a bill by itself. The separation matters: a quotation is a promise about price, while a bill is a statement of work actually delivered.

TABLE 25 — QUOTATION STATES

| STATE | MEANING | ALLOWED NEXT STEPS |
|---|---|---|
| **draft** | Being prepared; not visible to the client | Edit, send, discard |
| **sent** | Issued to the client with a validity date | Accept, decline, expire, revise |
| **accepted** | Client has agreed the price | Convert to gate pass or client rate card |
| **declined** | Client rejected the offer | Revise as a new draft |
| **expired** | Validity date passed without a decision | Re-issue with a new validity date |

### Procedure — preparing a quotation

1. Select **Quotations → New quotation** and choose the prospective or existing client.
2. Add line items: item, service, quantity and rate. Use the item catalogue where possible so that the wording matches the eventual bill.
3. Set the validity period. A quotation without an end date is a price the laundry may be held to indefinitely.
4. Review the totals and any terms, then select **Send** or **Mark as sent** once the client has received it.
5. When the client accepts, record the acceptance and convert the quotation. Conversion creates a draft gate pass or updates the client's rate card; it does not create a bill.

### AI assistance

The quotation module can draft line items and wording through the AI chat surface. Drafts are suggestions: every rate and quantity must be reviewed before the quotation is sent, and the platform records who issued it, not what suggested it. The assistant has no ability to send a quotation or to change a price without a user action.

> NOTE
> A converted quotation is linked to its source. When a client later disputes a rate, the chain from quotation to pass to bill is visible in one place.

4.9 · WORKERS

## Workers

The worker register records the people who do the work and, where the laundry pays by output, the work attributed to them. It is deliberately simple: identity, role, status and assignment. Payroll arithmetic belongs to the accounting service, not to the operational record.

TABLE 26 — WORKER RECORD

| FIELD | PURPOSE | NOTES |
|---|---|---|
| **Worker code** | Short identifier used on production sheets | Unique within the laundry |
| **Name** | Display name on assignment lists | Editable; historical records keep the name at the time |
| **Role** | Washer, presser, driver, checker, supervisor | Drives which worklists the worker appears in |
| **Status** | Active, on leave, inactive | Inactive workers cannot be assigned new work |
| **Contact** | Phone for shift coordination | Optional; treated as personal data |

### Procedure — adding a worker

1. Open **Workers** and select **Add worker**.
2. Enter the worker code exactly as it appears on the laundry's own records; the code is how production sheets are reconciled.
3. Set the role and status. A worker who has left is set to inactive rather than deleted, because historic assignments reference them.
4. Save. The worker appears in assignment lists for the roles selected.

### Assignment and accountability

Work can be attributed to a worker at the gate-pass level, the item-line level, or at the wash batch level in the linen room. Attribution is optional for operations and essential for piece-rate payroll: if a laundry pays by item, the assignment is the evidence of who processed what. Where attribution is used, it must be recorded at the time of work, not reconstructed at the end of the week.

> NOTE
>
> Attendance and salary records live in laundry-management, not here. The worker register supplies identity to both, so a name change in one place does not silently diverge from the other.

4.10 · REPORTS AND THE MONTHLY MATRIX

## Reports and the monthly matrix

Reports answer questions that outlive the day. They are computed from the same operational documents as the dashboards, so a report never disagrees with the screen it summarises — and a report is only as good as the discipline with which passes and deliveries were recorded.

TABLE 27 — AVAILABLE REPORTS

| REPORT | ANSWERS | TYPICAL USE |
|---|---|---|
| **Daily production** | What was received, washed and delivered on a date | Shift handover and capacity planning |
| **Client statement** | Every bill, payment and balance for one client | Month-end credit control |
| **Monthly matrix** | Quantities and values by client by day | Contract reconciliation and volume checks |
| **Unbilled deliveries** | Delivered work not yet on a bill | Revenue at risk before day close |
| **Linen discrepancy** | Missing, damaged and short-delivered items | Loss investigation and client conversations |
| **Worker productivity** | Items or batches attributed per worker | Piece-rate payroll and scheduling |

### The monthly matrix

The matrix is a grid: clients down the side, days of the month across the top, and a cell for each day holding the quantity and value processed for that client. It exists because a monthly total hides the pattern that matters — a client whose volume collapses for four days, or a day on which one client's items were entered against another's name.

Rows are clients, columns are calendar days, and the final columns total the month. Cells are derived from gate passes (received) and deliveries (returned), so the matrix is a view, not a ledger: nothing is typed into it, and it cannot be edited. A cell that looks wrong is a pass that was entered wrongly, and that is where the correction belongs.

> NOTE
>
> Export the matrix before a contract review. It is the document clients recognise, and it is generated from the same records that produced their bills.

4.11 · DAY CLOSE

## Day close

Day close is the point at which a business date stops moving. It freezes the day's totals, confirms that every delivery has been billed or consciously deferred, and records who closed the day and when. It is a control, not a formality: most reconciliation failures are discovered at close, which is exactly when they are cheapest to fix.

### Procedure — closing the day

1. Confirm that all gate passes for the date are recorded. Compare the register with the paper collection book.
2. Confirm that all deliveries made today are recorded and that no pass is left open without a reason.
3. Check **Unbilled deliveries**. Raise the missing bills, or record why a delivery is intentionally held into the next period.
4. Reconcile **Today's collections** against the cash drawer and the card terminal report. Record any difference with an explanation.
5. Review linen discrepancies raised during the day and confirm that each has an owner.
6. Select **Close day**. The system writes a day-close record with totals, the operator and a timestamp.
7. Print or save the close summary and attach it to the day's banking.

TABLE 28 — WHAT THE CLOSE FREEZES

| ITEM | EFFECT OF CLOSING |
|---|---|
| **Bill totals** | Figures for the date become the reported figures; later corrections appear on the date they are made |
| **Payment totals** | Collections for the date are fixed for reconciliation |
| **Delivery counts** | Volumes reported in the monthly matrix stop changing for that day |
| **Day-close record** | Immutable; a correction requires a reopening with a reason |

> WARNING
>
> Do not close a day to make a warning disappear. A closed day with an unresolved discrepancy is harder to correct than an open one, because reopening is itself an audited event.

4.12 · ADMINISTRATION, USERS AND COMMON ERRORS

# Administration, users and common errors

Administration in the laundry app is limited to the tenant you are signed in to: users, their memberships and roles, and hotel client accounts for your own clients. Creating a laundry is a platform operation, described in Chapter 8.1, and is not available from this application.

### Users and roles

A user is a person with a login. A membership is that person's link to your laundry, carrying their role. Removing someone's access means removing the membership, not deleting the user — they may belong to another laundry, and their history on your records must remain attributable.

TABLE 29 — COMMON ERRORS AND WHAT TO DO

| MESSAGE OR SYMPTOM | USUAL CAUSE | ACTION |
|---|---|---|
| **403 — tenant could not be resolved** | Token issued without a tenant, or a client token used on a staff route | Sign out and sign in again; hotel accounts must use the Hotel Desk app |
| **403 — capability required** | Role lacks the capability for this action | Ask an administrator to review the membership role |
| **409 — duplicate pass number** | Two passes created from the same paper docket | Open both, keep the correct one, cancel the other with a reason |
| **422 — quantity exceeds received** | Delivery quantity larger than the pass line | Recount; correct the pass quantity with a manager if the pass was wrong |
| **Bill total differs from the print** | Bill edited after issue, or printed from a stale tab | Reload the bill; issued bills cannot change, so a difference indicates a stale view |
| **Dashboard tile shows zero** | Filter applied, or the day is closed | Clear the date filter; check the day-close record |

> NOTE
>
> Errors that mention a capability or a tenant are authorisation outcomes, not faults. Report the exact message to an administrator; it names what was missing.

4.13 · DAY-CLOSE SEQUENCE

# Day-close sequence

*What has to be true before a date can be frozen, and what happens when it is not.*

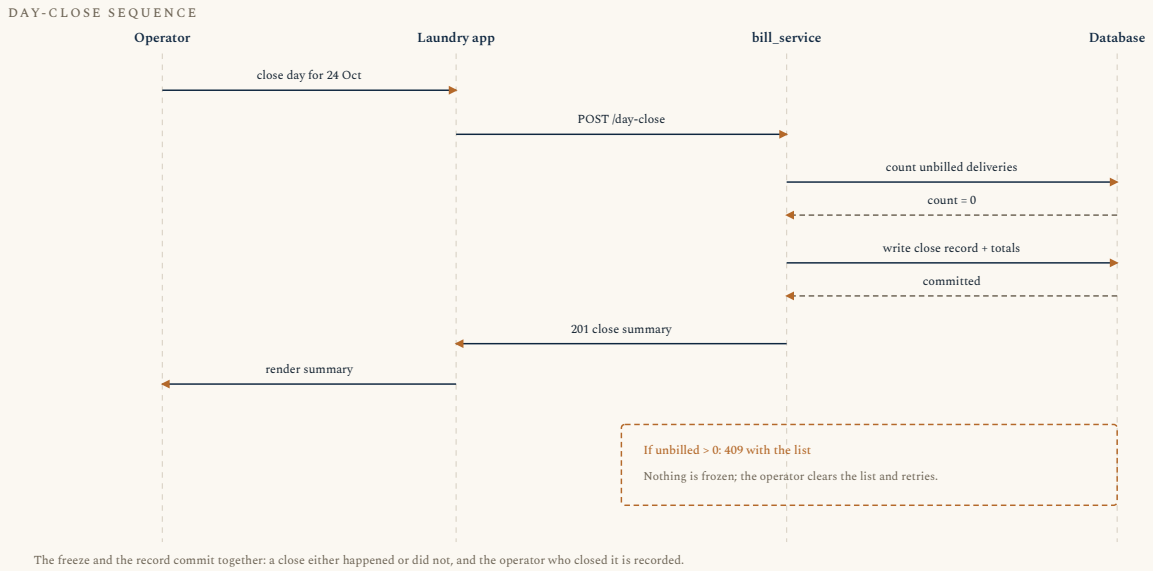



**FIGURE 13** — DAY-CLOSE SEQUENCE, INCLUDING THE BLOCKED BRANCH WHEN UNBILLED DELIVERIES REMAIN.

Closing a day is a transaction with a precondition, not a button that writes a total. The service counts unbilled deliveries first; if any exist, it refuses and returns the list. Nothing is frozen, so the operator can correct the cause and try again.

When the close succeeds, the totals and the close record commit together. That single unit is what makes the day's figures defensible: there is no state in which a date is marked closed but its totals were never written.

> RULE
>
> A close that fails is cheaper than a close that succeeds with an unresolved discrepancy. Reopening is itself an audited event and is never the first resort.

5

# User Manual — Shop Counter App

*A point-of-sale surface for walk-in customers: fast, narrow, and reconciled at the end of every shift.*

## 5.1 Overview and sign-in

The Shop Counter app is the shop-app front end. It serves a different transaction from the laundry: a customer arrives with a bag, not an account, and leaves with a receipt. Everything about the interface follows from that: few required fields, large targets, keyboard-first entry, and a day close that reconciles takings rather than work in progress.

TABLE 30 — SHOP COUNTER AT A GLANCE

| ATTRIBUTE | VALUE |
|---|---|
| **Application** | shop-app — React 19, TypeScript, Vite 8, Tailwind 4 |
| **Development port** | 5174 |
| **Sign-in endpoint** | POST /auth/login (user-service) |
| **Session key** | ll_shop_session |
| **Core records** | Shop bills, today's takings, day-close totals |
| **Typical user** | Counter staff, shop supervisor |

### Procedure — signing in at the counter

1. Open the Shop Counter app on the counter terminal.
2. Enter your own staff email and password, then select **Sign in**.
3. Confirm the laundry name in the header is your shop, and that the date shown is today.
4. Leave the session open for the shift. The token lasts 24 hours, but a shared terminal should be signed out at handover.

> WARNING
>
> Never sign in with a colleague's account to "save time". Every void, refund and discount is attributed to the signed-in user, and a shared login makes those actions unattributable.

5.2 · TODAY VIEW

## Today view

The today view is the shop's home screen and, for most staff, the only screen they need between customers. It answers three questions: what have I taken today, what is waiting to be finished, and what did the last few transactions look like?

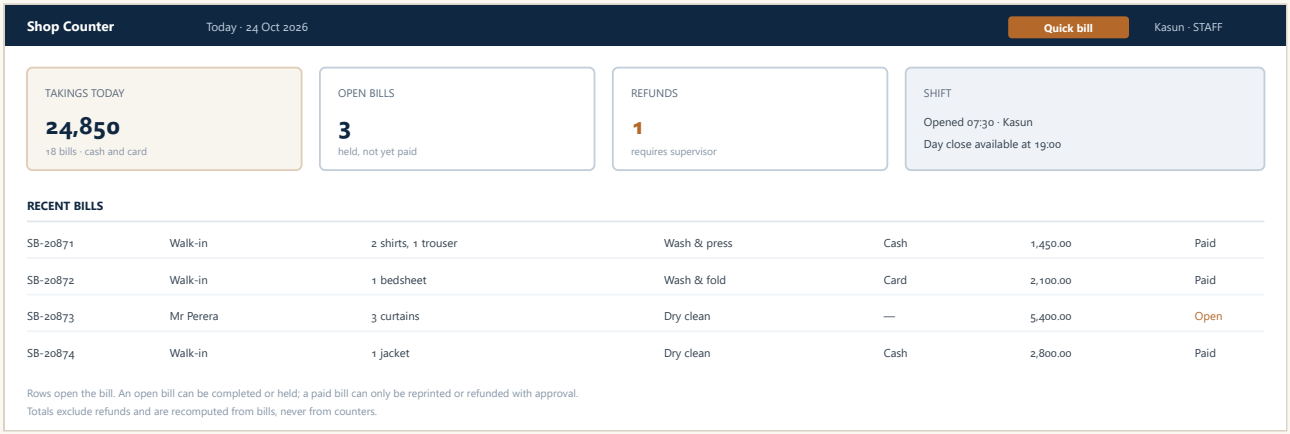



FIGURE 14 — SHOP COUNTER TODAY VIEW.

### What the tiles mean

**Takings today** is the sum of paid shop bills for the business date, split by tender in the detail. **Open bills** are transactions started but not settled — held garments awaiting collection, or a bill left open while the customer finds a card. **Refunds** counts refunds recorded today and flags that each requires supervisor approval. The **shift** panel names who opened the till and when day close becomes available.

> WARNING
>
> A large open-bill count at closing time usually means bills were abandoned mid-transaction rather than held deliberately. Resolve them before day close; an abandoned bill is a customer who left without paying and without a receipt.

5.3 · BILLS

# Shop bills

Shop bills are counter sales. They carry their own numbering, separate from laundry bills, because they are a different kind of document: a completed retail transaction rather than a periodic charge for work done. Keeping them separate means shop takings reconcile cleanly and laundry receivables are not diluted by walk-in cash.

TABLE 31 — SHOP BILL STATES

| STATE | MEANING | WHAT IS STILL POSSIBLE |
|---|---|---|
| **draft** | Items added, not yet taken to payment | Edit lines, apply discount, abandon |
| **open** | Payment not completed; customer may return | Complete payment, hold, add items |
| **paid** | Tendered and receipted | Reprint, refund with approval |
| **refunded** | Fully reversed after payment | None; a new bill is required for new goods |
| **void** | Cancelled before payment, with a reason | None; the number is retained |

### Procedure — completing a sale

1. Select **Quick bill**, or open an existing draft bill.
2. Add items: choose from the catalogue or type a free line where the catalogue has no entry.
3. Confirm quantities and rates aloud with the customer before taking payment.
4. Select the tender: cash or card. For cash, enter the amount tendered so the change is calculated and shown.
5. Select **Take payment**. The bill moves to paid and the receipt is offered for printing.
6. Hand over the receipt and the goods together. The receipt is the customer's evidence of the transaction and the shop's evidence of the tender.

### Discounts and their limits

A discount is a line-level or bill-level reduction with a reason. Counter staff may apply the configured standard discount; anything beyond it requires a supervisor, and the reason is recorded against the bill. The point is not to prevent generosity but to make it visible: a discount without a reason is indistinguishable from an error, and at month end it is indistinguishable from theft.

> NOTE
>
> Voided and refunded bills keep their numbers. Reconciling a day's takings against a bill sequence with gaps is far harder than reconciling one where every number is accounted for.

5.4 · QUICK BILL

# Quick bill

Quick bill is the counter's fast path: a numeric keypad, the most-used services as one-touch buttons, and a single total. It exists because the difference between a twelve-second and a forty-second transaction is the difference between a queue and a complaint.

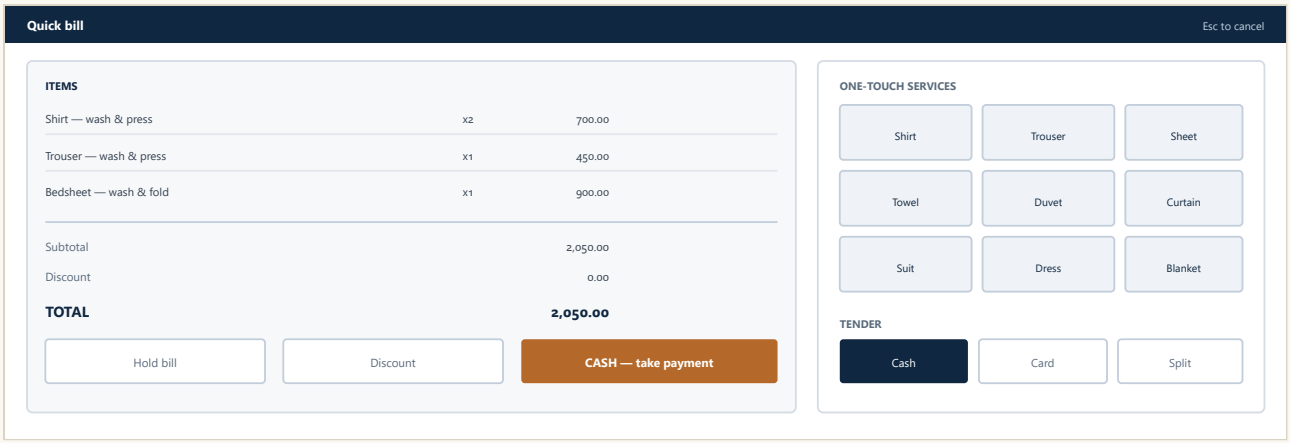



FIGURE 15 — QUICK BILL PAD: ITEM LIST LEFT, ONE-TOUCH SERVICES AND TENDER RIGHT.

### Procedure — a cash sale in under a minute

1. Select **Quick bill** from the header.
2. Tap the service buttons for each garment type; adjust quantity with the keypad if more than one.
3. Read the total to the customer and confirm the item count.
4. Select **Cash**, enter the amount tendered, and confirm the change shown.
5. Print the receipt. The bill is now paid and appears in today's takings.

Card sales follow the same path with the **Card** tender; the terminal is separate, and the app records the reference so the day close can be reconciled against the terminal report. Split tender is used when a customer pays part cash and part card, and the app requires both amounts to sum to the total before it will accept the payment.

> NOTE
> Quick bill never creates a laundry bill, a gate pass or a linen record. If a customer wants laundry service rather than a counter sale, use the laundry app.

5.5 · DAY CLOSE

## Day close at the counter

The shop's day close is shorter than the laundry's, and it has one job: prove that the money in the drawer matches the money recorded. Everything else — bills raised, refunds given, discounts applied — is detail that explains a difference, not a difference itself.

### Procedure — closing the shop day

1. Resolve every open bill: complete payment, or void it with a reason.
2. Count the drawer and record the counted amount in the close form.
3. Compare the counted amount with the system's cash total. If they differ, enter the difference and a short explanation before closing.
4. Check that card takings match the terminal's end-of-day report.
5. Review refunds and discounts recorded during the shift; each must have an approver.
6. Select **Close day**. The close record is written with totals, the operator and a timestamp.
7. Print the close summary, attach the terminal report and the bank slip, and hand both to the supervisor.

TABLE 32 — RECONCILIATION ITEMS

| ITEM | SOURCE OF TRUTH | TOLERANCE |
|---|---|---|
| **Cash takings** | Physical drawer count | Any difference is recorded, not adjusted away |
| **Card takings** | Terminal end-of-day report | Must match to the cent; a mismatch is investigated before close |
| **Refunds** | Refund records with approver | Each must be approved; unapproved refunds block close |
| **Open bills** | Bill list filtered to open | Must be zero, or explicitly held with a reason |

> WARNING
>
> Closing the day does not move money. If the drawer is short, record the shortage and escalate; do not take cash from a later sale to balance the day.

5.6 · ROLES AND PERMISSIONS

# Roles and permissions at the counter

The shop uses the same staff roles as the rest of the platform, with capabilities scoped to counter work. A shop user is a member of the laundry with a role; there is no separate shop-only account type. That keeps one identity per person across the business and one audit trail.

TABLE 33 — SHOP ROLES

| ROLE | CAN DO | CANNOT DO |
|---|---|---|
| **STAFF** | Create quick bills, complete payment, reprint receipts, hold bills | Give refunds, exceed standard discount, close day, void a paid bill |
| **MANAGER** | All counter actions, refunds, discounts beyond standard, day close, reprint any bill | Change user roles, alter shop configuration |
| **ADMIN** | Everything a manager can, plus user and membership management for the laundry | Create laundries or act outside the tenant |

### How the controls appear

The app hides actions the signed-in user cannot perform, so a counter assistant does not see a refund button they are not allowed to use. That is a courtesy, not the control: every action is re-checked on the server against the capability carried in the token. If a hidden action is somehow invoked — a stale tab, a crafted request — it fails with a permission error and an audit entry.

TABLE 34 — CAPABILITY MAPPING

| ACTION | CAPABILITY | HELD BY |
|---|---|---|
| **Create or pay a shop bill** | bill:write | STAFF, MANAGER, ADMIN |
| **View bills and takings** | bill:read | STAFF, MANAGER, ADMIN |
| **Post or reverse a payment** | payment:write | MANAGER, ADMIN |
| **View the day-close report** | report:read | MANAGER, ADMIN |
| **Open the dashboard** | dashboard:read | STAFF, MANAGER, ADMIN |

> NOTE
>
> A refund is a payment reversal in accounting terms, which is why it needs `payment:write` and therefore a manager. This is a deliberate restriction, not an oversight.

5.7 · PROCEDURES

# Counter procedures

The four procedures below cover almost every exception a counter encounters. Each is written to be followed under time pressure, with a customer waiting.

### Procedure — refunding a paid bill

1. Open the bill from **Today** or search its number.
2. Select **Refund**. The full amount is prefilled; reduce it for a partial refund.
3. Enter the reason and select the refund tender: cash, or reversal to the original card.
4. Call a supervisor to approve. The approval is recorded against the refund with the approver's name.
5. Hand over the cash or confirm the card reversal, and print the refund slip.

### Procedure — voiding a bill before payment

1. Open the open bill and select **Void**.
2. Enter the reason: wrong item, customer left, duplicate entry.
3. Confirm. The bill keeps its number and disappears from the open list.

### Procedure — holding a bill

Use **Hold bill** when a customer steps away or goes to fetch a card. A held bill stays in the open list with its items intact; it is not a credit sale and it does not reserve stock. At shift handover, resolve every held bill — complete it, void it, or write down why it remains open.

### Procedure — reprinting a receipt

Open the paid bill and select **Reprint**. The reprint carries the same number and a *duplicate* marker so that two copies cannot be presented as two transactions. Reprints are audited; a pattern of reprints on the same bill is worth a supervisor's attention.

WARNING

Never re-enter a sale to correct a receipt. That creates a second bill and inflates takings; void the original and re-enter once, or reprint it.

5.8 · SHIFT HANDOVER AND RECONCILIATION

# Shift handover and reconciliation

Most counter discrepancies are born at handover: a bill left open by one shift, a drawer counted by nobody, a refund approved verbally. A short, disciplined handover is cheaper than any reconciliation at month end.

TABLE 35 — HANDOVER CHECKLIST

| CHECK | DONE WHEN | EVIDENCE |
|---|---|---|
| **Open bills resolved** | No bill left open without a written reason | Open-bill list printed or photographed |
| **Drawer counted** | Count agreed by both shifts | Count slip signed by both |
| **Card report matched** | Terminal total equals recorded card takings | Terminal report attached |
| **Refunds accounted** | Each refund has an approver recorded | Refund list with approver column |
| **Petty cash** | Any shop float returned to the agreed amount | Float note |
| **Session closed** | Signed-out terminal, or explicit handover of the session | Sign-out recorded in the log |

### When the drawer does not match

1. Recount before concluding anything. Most differences are a miscount of notes of one denomination.
2. Check the card terminal report against recorded card takings; a card entry keyed as cash is the most common single cause.
3. Review the shift's bills for a total that was edited after payment, or a bill voided after cash was taken.
4. Record the remaining difference with an explanation in the day-close form. Do not adjust a bill to absorb it.
5. Escalate a difference above the shop's agreed threshold to the supervisor the same day.

NOTE

Differences are recorded, never absorbed. An absorbed difference hides a pattern, and a pattern is the only way to find a genuine problem.

5.9 · COMMON ERRORS

# Shop Counter — common errors

The messages below account for nearly all counter incidents. Each names what the system refused and why; the action column is written for the person at the till, not for an engineer.

TABLE 36 — COMMON ERRORS AT THE COUNTER

| MESSAGE | CAUSE | ACTION |
|---|---|---|
| **401 — session expired** | Token older than 24 hours, or the terminal's clock moved | Sign in again; if it recurs, check the terminal clock |
| **403 — payment:write required** | A refund or reversal attempted by STAFF | Call a supervisor to approve with their own account |
| **422 — tender does not equal total** | Split payment entered with a missing amount | Re-enter both amounts; the app requires them to sum exactly |
| **409 — bill already paid** | Two terminals completed the same bill | Open the bill, confirm the tender, refund the duplicate |
| **Bill not found** | Filtered to today, bill belongs to an earlier date | Clear the date filter and search by number |
| **Printer shows nothing** | Print dialog sent to a paused queue | Check the printer queue; the bill is already saved and can be reprinted |
| **Day close blocked** | Open bills or unapproved refunds remain | Resolve the listed items, then close again |

### When to stop and call for help

Stop and escalate if the app shows a total that disagrees with the items, if a paid bill cannot be found, or if the drawer difference exceeds the shop's threshold. In each case the correct action is to preserve the state — do not void, refund or re-enter — and to record what was on screen, because a second transaction taken to “fix” the first is what turns a reportable discrepancy into an unexplainable one.

> NOTE
>
> Report the exact message text. “It wouldn't let me” cannot be diagnosed; “403 payment:write required” identifies the cause immediately.

6

# User Manual — Hotel Desk App

*A read-only portal in which a hotel sees its own laundry records — and nothing else.*

## 6.1 Hotel client sign-in

The Hotel Desk app is the hotel-app front end. It is used by hotel front-office and housekeeping managers to check what they sent, what came back, and what they owe. It is not a staff application and it cannot change operational records: a hotel observes, it does not edit.

TABLE 37 — HOTEL DESK AT A GLANCE

| ATTRIBUTE | VALUE |
|---|---|
| **Application** | hotel-app — React 19, TypeScript, Vite 8, Tailwind 4 |
| **Development port** | 5175 |
| **Sign-in endpoint** | POST /client/auth/login (user-service) |
| **Session key** | ll_hotel_session |
| **Token lifetime** | 12 hours |
| **Scope** | One laundry, one client_name |

### Procedure — signing in as a hotel

1. Open the Hotel Desk app. Use the hotel's client account, not a staff account — staff credentials are refused by the client sign-in endpoint.
2. Enter the client account email and password and select **Sign in**.
3. Confirm the property name shown in the header matches your hotel. If it does not, stop: you are looking at another property's account and must report it immediately.
4. Leave the session to expire, or sign out at the end of the shift. Sessions last 12 hours.

> WARNING
>
> A hotel session carries no tenant claim and no staff role. It cannot be used in the laundry, shop or linen-room applications; those applications will reject it. This is by design, not a fault to work around.

> NOTE
>
> If the account is locked or the password is lost, contact the laundry that issued the account. The hotel cannot reset its own credentials from this application.

6.2 · OVERVIEW

# Overview

The overview is the hotel's single screen: what is currently with the laundry, what has been returned this month, what is outstanding financially. Every figure is filtered by the hotel's own `client_name` within the issuing laundry, so no other property's volume can appear in a total.

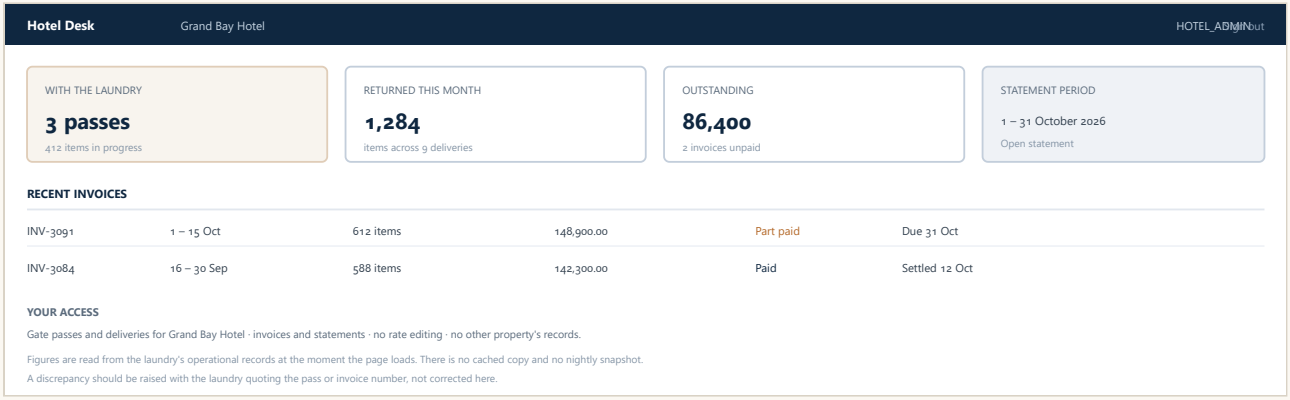



FIGURE 16 — HOTEL DESK OVERVIEW FOR A SINGLE PROPERTY.

### Reading the tiles

The four tiles are deliberately operational rather than analytical. **With the laundry** counts open gate passes and the items on them; **returned this month** counts delivered items in the current calendar month; **outstanding** is the unpaid balance across issued invoices; and the **statement period** links to the current statement. None of these figures includes another client's data, and none can be re-scoped by the hotel.

> NOTE
> A hotel cannot change the date range of a total on the overview. Period selection belongs to the statement, where the boundaries are explicit and the document is designed to be printed.

6.3 · GATE PASSES

# Gate passes (hotel view)

The hotel's gate-pass list shows every pass raised for its property: the date the goods were collected, the promised return date, the item count and the current status. It is the operational answer to "where is our linen?"

TABLE 38 — COLUMNS IN THE HOTEL GATE-PASS LIST

| COLUMN | MEANING |
|---|---|
| **Pass number** | The laundry's reference; quote it in any query |
| **Received** | Date the laundry took possession of the goods |
| **Due back** | Promised return date |
| **Items** | Total quantity on the pass |
| **Status** | Received, in wash, ready, delivered, closed |
| **Last update** | When the status last changed |

### What the hotel can see on a pass

The hotel sees the items, the quantities, the service type and the status history. It does not see rates on the pass view — pricing is a commercial matter between the laundry and the property, stated on the invoice — and it never sees the laundry's internal cost, worker assignment or route information.

### Procedure — finding a pass

1. Open **Gate passes**.
2. Filter by date range if you know when the goods were collected, or search by pass number from the laundry's collection note.
3. Open the pass to see its item lines and status history.
4. If the status has not moved when you expect it to, contact the laundry quoting the pass number rather than raising a new pass; there is no way to create a pass from this application.

> WARNING
> A hotel cannot create, edit or cancel a gate pass. A missing pass is a laundry-side data entry issue: report it with the collection date and the driver's name, and the laundry will raise or correct it.

6.4 · DELIVERIES

# Deliveries (hotel view)

Deliveries show what has come back, when, and in what quantity. For a hotel, this is the operational record that supports housekeeping's own counts — and the evidence used when an invoice quantity is questioned.

TABLE 39 — DELIVERY INFORMATION VISIBLE TO A HOTEL

| FIELD | WHY IT MATTERS |
|---|---|
| **Delivery date and time** | Matches the property's receiving log |
| **Pass reference** | Links the delivery to what was sent |
| **Items and quantities** | What was actually returned, line by line |
| **Received by** | The name recorded by the laundry at handover |
| **Partial indicator** | Shows that a pass is only partly returned |

### Partial deliveries

A pass may come back in several deliveries. The hotel sees each one separately, with the outstanding quantity on the pass. This is normal for large properties where sheets and towels are processed in different batches, and it is not a discrepancy.

### Procedure — reconciling a delivery

1. Open **Deliveries** and filter to the date of receipt.
2. Compare the quantities on each delivery with the property's receiving sheet, line by line.
3. Where a line differs, note the pass number, the item and both quantities before contacting the laundry.
4. Report the difference the same day. A difference found a week later cannot be checked against the laundry floor.
5. Do not sign for a delivery you have not counted; the recorded receiver is the evidence that the count was accepted.

> NOTE
> Deliveries are read-only here. If a count was signed for in error, the correction is made by the laundry and appears in the history, including the reason.

6.5 · INVOICES

## Invoices

An invoice is the hotel's bill: quantities delivered in a period, at the agreed rates, with tax and the balance outstanding. The hotel can open an invoice, read its lines and download it — but it cannot alter it, and it cannot raise one.

TABLE 40 — INVOICE STATES VISIBLE TO THE HOTEL

| STATE | WHAT IT MEANS FOR THE PROPERTY |
|---|---|
| **Issued** | Payable; the amount due is shown on the invoice |
| **Part paid** | Some payment has been recorded; the remainder is shown |
| **Paid** | Settled in full; no action required |
| **Void** | Cancelled by the laundry; a replacement may exist — check the statement |

### Procedure — checking an invoice

1. Open **Invoices** and select the period or invoice number.
2. Open the invoice and compare its lines with your own receiving records for the same period.
3. Download the PDF if you need it for internal approval. The download is the same document the laundry holds.
4. Where a line is disputed, quote the invoice number, the line and the pass reference in your query to the laundry.
5. Do not withhold payment for one disputed line without telling the laundry: a short payment without explanation appears as an aged balance and may suspend service.

### What an invoice does not show

Invoices state quantities and agreed rates. They do not show the laundry's internal costs, worker productivity, other clients' rates, or the composition of any discount beyond the line description. Those are outside the client relationship and are not exposed by any hotel endpoint.

> WARNING
>
> If an invoice includes a pass you do not recognise, do not pay it and do not ignore it. Report it with the pass number the same day; a misposted pass is easy to correct before the period closes and hard afterwards.

6.6 · STATEMENTS

## Statements

A statement lists every invoice, payment and adjustment for a period with a running balance. It is the document a hotel's finance team reconciles against its own ledger, and it is the reason the portal exists for many properties.

TABLE 41 — STATEMENT CONTENTS

| SECTION | CONTENT |
|---|---|
| **Opening balance** | Amount outstanding at the start of the period |
| **Invoices** | Each invoice with date, number and amount |
| **Payments** | Each payment received, with method and reference |
| **Adjustments** | Credit notes and corrections, each with a reason |
| **Closing balance** | Opening balance plus invoices, less payments and credits |

### Procedure — reconciling a statement

1. Open **Statements** and select the period. The default is the current month to date.
2. Print or download the statement for your finance file.
3. Match each invoice to your approved purchase record.
4. Match each payment to your bank or petty-cash record. A payment recorded by the laundry but not yet cleared in your bank will appear here before it appears there — that is timing, not error.
5. Query any unmatched line with its reference. The closing balance is the figure that must agree, not the individual lines alone.

> NOTE
>
> Statements are generated on request from the laundry's records. A statement downloaded twice in a day may differ if a payment was posted in between, so always keep the version you reconciled against.

6.7 · WHAT A HOTEL CAN AND CANNOT DO

## What a hotel can and cannot do

The hotel portal is read-only by design. The table below is the complete statement of a hotel account's authority; anything not listed as permitted is refused by the server, not merely hidden in the interface.

TABLE 42 — HOTEL PERMISSIONS

| ACTION | PERMITTED | NOTES |
|---|---|---|
| **View own gate passes** | Yes | Filtered by laundry_id and client_name |
| **View own deliveries** | Yes | Includes partial deliveries |
| **View and download own invoices** | Yes | Issued documents only; drafts are invisible |
| **View and download own statements** | Yes | Any period within retention |
| **Create or edit a gate pass** | No | Laundry staff only |
| **Record or amend a delivery** | No | Laundry staff only |
| **Raise, edit or void an invoice** | No | Issued invoices are immutable for everyone |
| **Post a payment** | No | Payments are recorded by the laundry |
| **View another property's records** | No | Different client_name; no endpoint accepts a name parameter |
| **View rates, costs or worker data** | No | Outside the client relationship |
| **Manage hotel user accounts** | No | HOTEL_ADMIN manages its own users only through the laundry |

### Why read-only

A hotel that could edit its own records could dispute an invoice by changing the evidence. Making the portal read-only removes that possibility entirely and makes the portal's data credible to both parties: what the hotel sees is exactly what the laundry recorded, unmediated.

> NOTE
>
> HOTEL_ADMIN and HOTEL_MANAGER differ in what they can see of the account's commercial detail, not in what they can change. Neither role can change operational data.

6.8 · PROCEDURES

# Hotel desk procedures

These are the five tasks a hotel performs in the portal. All of them end in reading or downloading; none of them changes a laundry record.

**Procedure — checking what is still with the laundry**

1. Open **Overview** and read the *with the laundry* tile.
2. Open **Gate passes** and filter to open statuses.
3. For each pass, check the due-back date against housekeeping's requirement for the coming days.
4. If a pass is overdue, contact the laundry with the pass number and the date it was due.

**Procedure — preparing a month-end reconciliation**

1. Open **Statements** and select the closed month.
2. Download the statement and every invoice referenced on it.
3. Match invoices to approved purchase records and payments to bank entries.
4. Record any unmatched item with its reference and raise a single consolidated query with the laundry.

**Procedure — reporting a discrepancy**

Collect the pass number or invoice number, the item description, the quantity you counted and the date. Send them in one message to the laundry's client contact. A query with references is resolved in minutes; a query without them requires the laundry to reconstruct the day.

> WARNING
>
> Do not open a second account to check another property, and do not share credentials between properties. Each client account is scoped to one property; sharing it is a breach of the terms of use and destroys the audit trail.

6.9 · COMMON ERRORS AND SUPPORT

# Hotel desk — common errors and support

TABLE 43 — ERRORS A HOTEL USER MAY SEE

| MESSAGE | CAUSE | ACTION |
|---|---|---|
| **401 — invalid credentials** | Wrong password, or a staff account used on the client endpoint | Use the hotel client account; ask the laundry to reset if forgotten |
| **403 — client scope not resolved** | Account exists but is not linked to a laundry and client_name | Ask the laundry to correct the client account |
| **404 — not found** | The record belongs to another property | Check the reference; do not retry with a different account |
| **No invoices for this period** | Invoices not yet issued for the period | Ask the laundry when the period will be billed |
| **Session ended** | 12-hour client token expired | Sign in again |
| **Statement shows an unknown invoice** | Pass posted to this property in error | Report the invoice and pass number the same day |

### Support path

The hotel's first contact is the laundry that issued the account: they own the records, the invoicing and the client account itself. The laundry escalates to the platform operator only for authentication faults that survive a password reset — for example, a client account whose scope is missing or a token that is refused by every endpoint.

### What to include in a support request

- The property name and the client account email (never the password).
- The exact message text and the time it occurred.
- The pass or invoice number involved.
- What you expected to see and what you saw instead.

> NOTE
>
> Hotel accounts are created and disabled by laundry administrators in the user-service client account screen. A hotel cannot create its own account, and there is no self-service registration.

6.10 · HOTEL PORTAL REQUEST PATH

## Hotel portal request path

*How one property's records are separated from every other property's, on every request.*

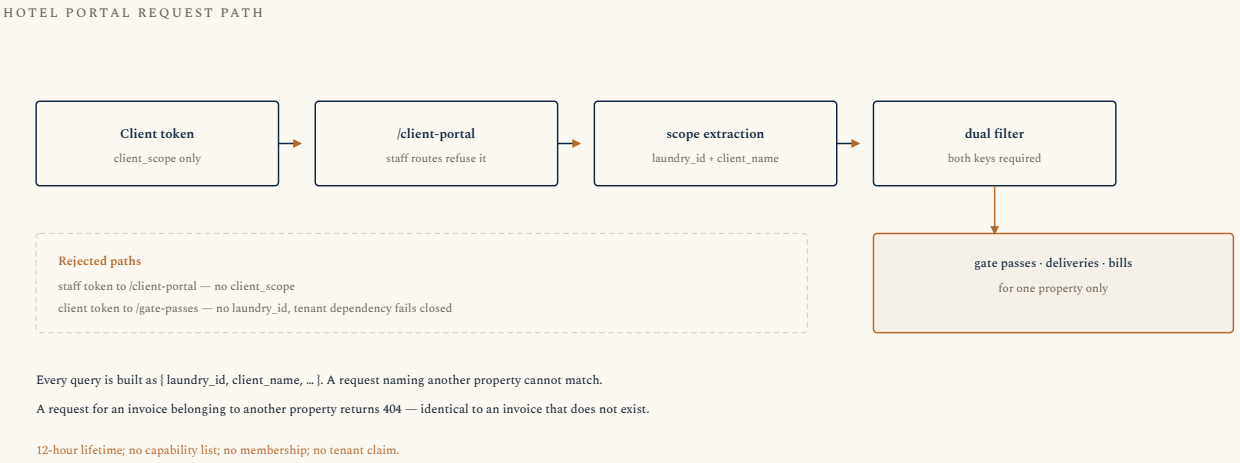



**FIGURE 17** — REQUEST PATH FROM A CLIENT TOKEN TO A SCOPED QUERY, WITH THE REJECTED PATHS SHOWN.

The portal has no endpoint that accepts a client name as a parameter. The name comes from the token's scope, so a hotel cannot ask for another property's records even by guessing an identifier.

The dual filter is the mechanism: every query carries both the laundry and the client name. Filtering on the name alone would be unsafe across laundries; filtering on the laundry alone would expose every hotel to every other hotel.

> RULE
>
> A request for another property's invoice returns not-found, identical to an invoice that does not exist. The portal never confirms that another property's records exist.

7

# User Manual — Linen Room App

*Scan-driven tracking of every tagged item, from collection to retirement.*

## 7.1 Overview and scanning

The Linen Room app is the linen-room-app front end, used on a tablet or handheld at the wash station. Its users work one-handed with a scanner, so the interface is a single scan field, a large current-item card, and a short list of valid next actions. Everything else is history.

TABLE 44 — LINEN ROOM AT A GLANCE

| ATTRIBUTE | VALUE |
|---|---|
| **Application** | linen-room-app — React 19, TypeScript, Vite 8, Tailwind 4 |
| **Development port** | 5176 |
| **Sign-in** | Staff sign-in against user-service |
| **Session key** | ll_linen_session |
| **Roles** | LINEN_SUPERVISOR, HOUSEKEEPER |
| **Capabilities** | linen:read, linen:scan, linen:write |

### Why scanning, not typing

A typed tag number is a guess with a keyboard. A scan is the identifier itself. The platform therefore treats the scan as the only way a lifecycle status advances: there is no “set status” dropdown, and no bulk action that moves items without reading them. If an item was not scanned, the record says it was not scanned — which is the truth the laundry needs when a client disputes a count.

### Procedure — starting a shift at the station

1. Sign in on the station device with your own linen-room account.
2. Confirm the station name and the current batch or pass shown at the top.
3. Test the scanner by scanning any tag already in hand; the item card should populate within a moment.
4. Keep the scan field focused. If a scan produces no response, tap the field before scanning again rather than retyping the tag.

> WARNING
>
> Do not share a station login between staff. Every scan is attributed to the signed-in user, and an unattributable scan is worthless when a discrepancy is investigated.

7.2 · LIFECYCLE STATUSES

## Lifecycle statuses

Each tag moves through a fixed sequence. Statuses advance only by recording the action that caused the move, and each action is attributed to a user and a time. The sequence is enforced server-side: a scan that would skip a stage is rejected.

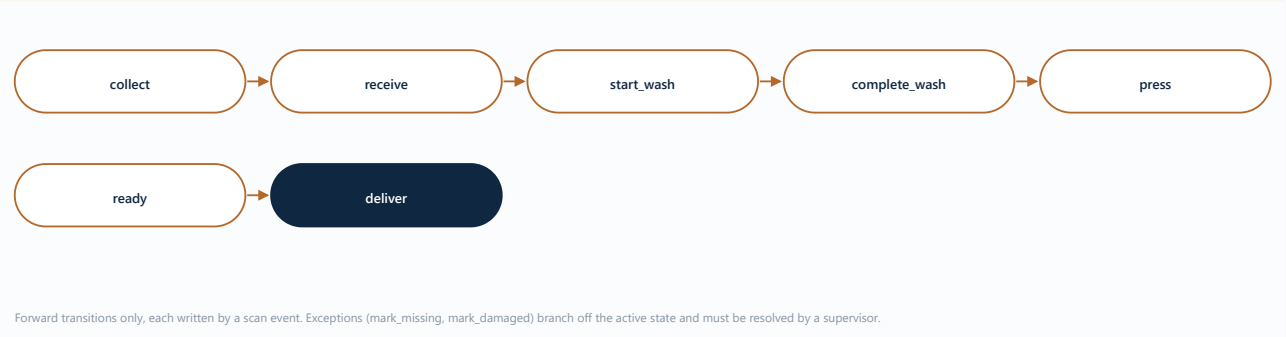



FIGURE 18 — LINEN STATE MACHINE. EXCEPTIONS ( `MARK_MISSING` , `MARK_DAMAGED` , `RETIRE` ) BRANCH FROM THE ACTIVE STATE.

TABLE 45 — ACTIONS AND THEIR MEANING

| ACTION | FROM → TO | WHEN IT IS USED |
|---|---|---|
| **collect** | new → collected | Goods picked up from the client or from the property's linen room |
| **receive** | collected → received | Counted in at the laundry; discrepancies are raised here |
| **start_wash** | received → in wash | Loaded into a wash batch |
| **complete_wash** | in wash → washed | Batch finished; items awaiting finishing |
| **press** | washed → pressed | Ironed or folded and inspected |
| **ready** | pressed → ready | Racked and available for delivery |
| **deliver** | ready → delivered | Handed to the client against a delivery |
| **mark_missing** | any active → missing | Item cannot be found; requires a supervisor |
| **mark_damaged** | any active → damaged | Item is damaged; requires a supervisor |
| **retire** | any → retired | Item withdrawn from service permanently |

> NOTE
>
> There is no reverse transition. An item scanned into the wrong batch is corrected by recording the exception and re-scanning it into the correct batch, which leaves both events visible.

7.3 · SCAN WORKFLOW

# Scan workflow

The station screen is built around one loop: scan, confirm, act. The current item card shows what the platform knows, and the action buttons show only what is legal next — which is how a fast operator avoids invalid transitions without memorising the rules.




FIGURE 19 — SCAN STATION: SCAN FIELD, CURRENT TAG CARD AND LEGAL NEXT ACTIONS.

### Procedure — scanning a batch through

1. Scan the tag. The current tag card confirms the item and its current status.
2. Check the item against the card — type, client and pass. A scan of the wrong item is worse than no scan.
3. Select the next action. Only legal transitions are offered. The list updates immediately.
4. Repeat for each item. The batch counter increments as items advance.
5. When the batch is complete, confirm the batch total on screen against the physical count before moving the load.

> NOTE
> Scanning is idempotent: scanning the same tag twice for the same action does not duplicate the event. It is safer to scan again than to assume a scan registered.

7.4 · INVENTORY

## Inventory

Inventory is the count of tags by status, item type and client. It is derived from scan history, so it is a live picture rather than a periodic stock take — and it is only as accurate as the scanning discipline behind it.

TABLE 46 — INVENTORY VIEWS

| VIEW | SHOWS | USE |
|---|---|---|
| **By status** | Tags in each lifecycle state | See what is in wash, ready or missing at a glance |
| **By item type** | Sheets, duvet covers, towels, mats and so on | Capacity planning and reorder decisions |
| **By client** | Tags attributable to each client_name | Client-level reconciliation and loss conversations |
| **By age** | Tags whose current status is older than expected | Find items stuck in a stage |
| **Retired** | Tags withdrawn from service | Asset history and replacement rate |

### What inventory is not

It is not a valuation. The platform counts tags and items, not their book value; asset accounting belongs to the accounting service and to the laundry's own fixed-asset register. It is also not a guarantee of physical presence: a tag can be scanned and then lost, which is exactly what the discrepancy process exists to catch.

### Procedure — daily inventory check

1. Open **Inventory** and select **By status**.
2. Compare the *in wash* count with the batches physically on the floor. A large difference means items were loaded without scanning.
3. Compare *ready* with the rack. Ready items that are not on the rack are either delivered without a scan or missing.
4. Check **By age** for items in one status for longer than the wash cycle allows.
5. Record any unexplained difference through the discrepancy process rather than adjusting the count.

> WARNING
>
> Never correct inventory by retiring items to make the numbers match. Retirement is permanent and removes the asset from service; a missing item may still be found.

7.5 · DISCREPANCIES

# Discrepancies

A discrepancy is a recorded mismatch: an item counted in short, found damaged, or unaccounted for at the end of a batch. The platform records discrepancies rather than smoothing them away, because a pattern of loss is only visible if each instance survives.

TABLE 47 — DISCREPANCY TYPES

| TYPE | RAISED WHEN | EFFECT |
|---|---|---|
| **Short receipt** | Fewer items received than the pass states | Pass line quantity is queried; client is notified |
| **Missing** | Item cannot be found after a search | Tag leaves the active batch; supervisor review |
| **Damaged** | Item is torn, stained beyond recovery or otherwise unusable | Tag marked damaged; client liability assessed |
| **Short delivery** | Fewer items delivered than the client expected | Pass remains open or is closed short with a reason |
| **Unscanned** | Item found physically but absent from the batch record | Item is scanned into the correct stage; cause noted |

### Procedure — recording a missing item

1. Search the station, the batch and the rack before recording anything. Most missing items are in the previous batch.
2. Scan or enter the tag and select **mark_missing**.
3. Enter where it was last seen and what search was carried out.
4. Confirm. The item leaves the active batch and appears in open discrepancies.
5. Notify the supervisor, who decides whether to inform the client and whether to replace the item.

### Closing a discrepancy

A discrepancy is closed when the item is found, when the client accepts the loss, or when it is written off with a reason. Closing is a supervisor action with an audit entry. An item found later is scanned back into the correct stage, which closes the missing record automatically and leaves both events in the tag's history.

> NOTE
>
> Discrepancy counts are part of the dashboard for a reason: a laundry that never records a discrepancy is usually not counting, not not losing.

7.6 · TAG PRINTING

# Tag printing

Tags are the physical identity of an item. Printing is therefore a controlled action: each printed tag is recorded, and a tag that is printed but never attached is a future discrepancy.

TABLE 48 — TAG PRINTING RULES

| RULE | REASON |
|---|---|
| **Tags are printed in batches linked to an item type** | Keeps numbering readable and simplifies replacement |
| **A printed tag is recorded as issued** | Allows an unattached tag to be traced |
| **Duplicate tags are never printed for an active item** | Two tags on one item double-counts inventory |
| **A replacement tag is issued only after the old tag is retired** | Prevents two live identities for one item |
| **Printing requires linen:write** | HOUSEKEEPER can scan but not print |

### Procedure — printing a tag batch

1. Open **Tags → Print batch**.
2. Choose the item type and the number of tags required.
3. Select the printer and confirm that labels are loaded — a half-printed batch wastes tag numbers.
4. Print, then apply each tag to an item immediately. Do not leave printed tags loose at the station.
5. Confirm the batch in the app so that the tags are recorded as issued and attached.

### Replacing a damaged tag

Where a tag is unreadable but the item is identifiable, retire the old tag and issue a replacement, recording the reason. Where the item is not identifiable, treat it as an unscanned item: count it, assign it to the client whose linen it matches, and record how the identification was made. Guessing the client is worse than leaving the item unassigned.

> WARNING
>
> Never reprint an existing tag number onto a new label while the original is still attached. The item will be counted twice and the duplicate will be found months later during a reconciliation nobody can explain.

7.7 · PROCEDURES

## Linen room procedures

The four procedures below cover a normal shift: receiving a pass, running a batch, delivering, and handing over at the end of the shift.

### Procedure — receiving a pass

1. Open the pass from the station's pass list.
2. Count the items physically, by type, before scanning anything.
3. Scan each tag as it is counted. Items without a tag are tagged first, then scanned.
4. Compare the scanned count with the pass quantity. Where it differs, record a short receipt with the count you found.
5. Confirm receipt. The pass moves to received and the items become available for batching.

### Procedure — running a wash batch

1. Create a batch for the machine and load it.
2. Scan each item into the batch and select **start_wash**.
3. At the end of the cycle, select **complete_wash** for the batch and scan out any item removed for separate treatment.
4. After pressing, scan each item and select **press**, then **ready** when it is racked.

### Procedure — delivering from the linen room

Delivery is recorded in the laundry app against the gate pass, not in the linen room. The linen room's part is to ensure that every item on the rack has been scanned to *ready*, so that the delivery quantity is supported by tag history. Where the linen room is also the dispatch point, scan the items out as **deliver** at handover so that the tag and the delivery agree.

> NOTE
> Shift handover: confirm the batch count on screen matches the physical count, note any open discrepancies, and sign out. An unsigned station is an unattributable one.

7.8 · TROUBLESHOOTING

## Linen room — troubleshooting

TABLE 49 — COMMON PROBLEMS AT THE STATION

| SYMPTOM | LIKELY CAUSE | ACTION |
|---|---|---|
| **Scan produces nothing** | Field lost focus, or scanner in keyboard-wedge mode disconnected | Tap the scan field and scan again; check the scanner cable or pairing |
| **"Invalid transition"** | Item is not in the state the action requires | Read the current status on the card; choose a legal action |
| **"Tag not found"** | Tag belongs to another laundry, or was never issued | Check the tag number against the print batch; do not create a duplicate |
| **Item scanned twice** | Double trigger or repeated scan | No action — scans are idempotent for the same action |
| **Batch count does not match the rack** | Item removed without scanning out | Scan the item to its actual stage; record an unscanned-item note |
| **Cannot print tags** | Role lacks linen:write | Ask a supervisor to print, or to adjust the role |

### When the network is down

The station cannot record scans without a connection to bill_service. Do not continue scanning into a local file or a paper list and enter it later as if it happened in sequence: the timestamps would be false and the batch history misleading. Record the physical count on paper, keep the items segregated, and enter the scans when the connection returns, noting the delay. Chapter 11.4 describes what remains usable while a service is unavailable.

### Escalation

Escalate to a supervisor when an item cannot be identified, when a discrepancy involves more than a few items, or when a tag appears to exist in two batches. Escalate to the platform operator only when the application itself fails — a blank screen, a repeated server error, or a sign-in that fails for every user.

> NOTE
>
> Report the tag number and the exact message. A screenshot of the station screen is the fastest possible evidence.

7.9 · LINEN LIFECYCLE STATE MACHINE

## Linen lifecycle state machine

*Seven forward states, three exception states, and no way back.*

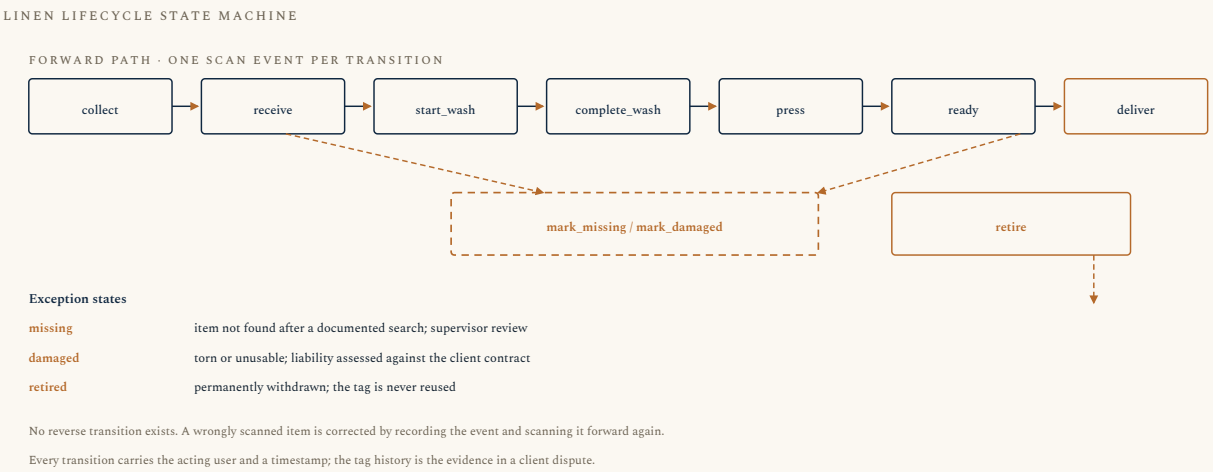



FIGURE 20 — THE LINEN STATE MACHINE WITH ITS EXCEPTION BRANCHES. TRANSITIONS ARE WRITTEN BY SCAN EVENTS ONLY.

The scan procedure that writes these transitions is on page 75. The forward path is linear because the physical process is linear: goods are collected, received, washed, finished, pressed, made ready and delivered. Modelling it as a chain rather than a set of flags means an item cannot be simultaneously ready and in wash.

The exception states branch off the active state rather than replacing the chain. An item marked missing leaves the batch but keeps its history, so when it is found it can be scanned back into the correct stage without inventing a new tag.

> RULE
>
> There is no reverse transition and no status dropdown. A wrongly scanned item is corrected by recording the event and scanning it forward again, which leaves both facts visible.

III

PART THREE

# Operating the Business


8 Administration and Configuration

9 Disaster Management

10 Backup and Recovery

11 Fail-Safes and Resilience

PART THREE · ADMINISTRATION AND CONFIGURATION

# 8 Administration and Configuration

*Creating tenants, granting access, and configuring an installation that can be rebuilt from documentation.*

## 8.1 Creating laundries

A laundry is a tenant. Creating one is a platform operation performed by a `PLATFORM_ADMIN`, because it establishes the boundary inside which every later record will live. Laundry administrators cannot create tenants; they administer the one they belong to.

TABLE 50 — WHAT A NEW LAUNDRY REQUIRES

| ELEMENT | PURPOSE |
|---|---|
| **Name and short code** | Display name and a stable short reference used on documents |
| **Status** | Active or suspended; a suspended tenant can sign in to read but not to write |
| **Currency and tax configuration** | Applied to every bill raised in the tenant |
| **First administrator** | An existing user granted ADMIN membership, or a new user created for the purpose |

### Procedure — creating a laundry

1. Sign in as a platform administrator and open the tenant administration screen.
2. Create the laundry: name, short code, currency and tax rate. Check the tax rate against the jurisdiction before saving — changing it later does not alter issued bills.
3. Create or select the first administrator's user account.
4. Add a membership linking that user to the new laundry with the ADMIN role.
5. Sign in as that administrator in a private window and confirm that the tenant name appears and that an empty dashboard loads.
6. Record the creation in the platform change log with the operator and the time.

> WARNING
>
> Do not reuse a short code, even for a closed laundry. Short codes appear on printed documents and in exports; a reused code makes two businesses' history indistinguishable.

8.2 · USERS AND MEMBERSHIPS

## Users and memberships

The platform separates a person from their access. A **user** is an identity — an email, a password hash, a display name. A **membership** is that identity's link to one laundry, carrying a role. One person may hold memberships in several laundries and one role in each.

TABLE 51 — ROLE ASSIGNMENT GUIDANCE

| ROLE | GIVE IT TO | DO NOT GIVE IT TO |
|---|---|---|
| ADMIN | The owner and at most one deputy | Every manager, by default |
| MANAGER | Supervisors who close days and handle money | Counter assistants |
| STAFF | Operational data entry | Anyone who approves refunds |
| LINEN_SUPERVISOR | The person accountable for linen loss | Seasonal staff without handover responsibility |
| HOUSEKEEPER | Scanning and finishing staff | Anyone who prints tags or retires items |

### Procedure — adding a user to a laundry

1. Open **Users** and select **Add user**.
2. Enter the person's own email address. Shared mailboxes and role-based addresses make the audit trail useless.
3. Set an initial password and require a change at first sign-in.
4. Add the membership: choose the laundry (fixed to your own tenant if you are not a platform administrator) and the role.
5. Ask the user to sign in while you are present, so that a mistyped address is discovered immediately.

### Removing access

Removing someone's access means removing the membership or disabling the user, never deleting history. A deleted user leaves records attributed to nobody, which is exactly the situation an audit cannot tolerate. Disabling the user prevents new sign-ins; existing tokens remain valid until they expire, so a departure that requires immediate certainty also requires a signing-key rotation or a documented acceptance of the residual window.

> NOTE
> Changing a role takes effect at the next sign-in, because capabilities are carried in the token. If a change must apply immediately, disable the user and re-enable them so that a fresh token is issued.

8.3 · HOTEL CLIENT ACCOUNTS AND ROLES

# Hotel client accounts and roles

Hotel accounts live in a separate collection from staff users, and they are created by the laundry that serves the hotel. The account binds an email and password to exactly one `client_name` within the laundry, which is the whole of its authority.

TABLE 52 — CLIENT ACCOUNT RECORD

| FIELD | MEANING |
|---|---|
| **Email** | Sign-in identity for the hotel user |
| **Password hash** | Salted hash; never displayed or exported |
| **laundry_id** | The tenant whose records the account may read |
| **client_name** | The one hotel whose records are visible |
| **Role** | HOTEL_ADMIN or HOTEL_MANAGER |
| **Status** | Active or disabled |

### Procedure — creating a hotel account

1. Confirm the exact `client_name` spelling used on the hotel's gate passes and bills. A variant spelling creates an account that sees nothing.
2. Create the client account with the hotel's contact email and a role.
3. Communicate the credentials through a channel the hotel controls; never send a password in the same message as the sign-in address.
4. Ask the hotel to sign in and confirm they can see their own passes and invoices.
5. Record the creation and the operator in the change log.

### Enabling and disabling accounts

A client account is disabled when a property changes management, when a member of staff leaves, or when credentials are suspected compromised. Disabling is immediate for new sign-ins and is recorded with the operator and reason. The endpoints are `GET /client/auth/status` for the account's own state, `GET|POST /clients` for listing and creating accounts, and `PATCH /clients/{id}/status` for changing an account's state.

> WARNING
>
> Never create a client account with a staff role, and never add a client account to a membership. The two identity systems are separate so that a hotel token can never satisfy a staff authorisation check.

8.4 · ENVIRONMENT VARIABLES

# Environment variables

Configuration is supplied through the environment. Nothing that differs between installations is compiled into the code, and nothing secret is committed to the repository. The table lists the variables an operator must set; names follow the service that reads them.

TABLE 53 — REQUIRED CONFIGURATION

| VARIABLE | READ BY | PURPOSE |
|---|---|---|
| MONGO_URI | All services | Database connection string, with TLS and the service's own credential |
| DB_NAME | Per service | The database the service owns (user_db, operations, accounting) |
| JWT_SIGNING_KEY | user-service | Signs staff tokens; rotated on the schedule in 13.4 |
| JWT_VERIFY_KEYS | All services | Key set used to verify tokens, including the next key during rotation |
| JWT_TTL_STAFF / JWT_TTL_CLIENT | user-service | Token lifetimes: 24 hours and 12 hours |
| KEK_ID / KEK_MATERIAL | bill_service, user-service | Key-encryption key reference and material for envelope encryption |
| SEARCH_KEY | bill_service | HMAC key for deterministic search tokens |
| CORS_ORIGINS | All services | Exact front-end origins permitted to call the API |
| LOG_LEVEL | All services | Verbosity; production defaults to structured informational logging |
| RATE_LIMIT_* | Proxy and services | Thresholds per class, as described in 3.5 |

### Front-end configuration

Each React application reads build-time variables prefixed `VITE_`. Only public values belong there — service base URLs, environment name, feature flags — because everything so prefixed is embedded in the bundle and readable by anyone with a browser. A secret placed in a `VITE_` variable is published, not configured.

```
VITE_API_BASE=/api
VITE_ENV=production
VITE_ENABLE_AI_ASSIST=true
```

> WARNING
>
> Changing a signing key or a KEK is not a routine configuration edit. Follow the rotation procedures in 3.6; an unplanned change invalidates every session or makes encrypted fields unreadable.

8.5 · DEPLOYMENT STEPS

# Deployment steps

A deployment is a build, a configuration check, a service restart and a smoke test — in that order. The steps below assume a single application host behind a reverse proxy, as described in 2.9.

### Procedure — deploying a release

1. Build the four front-end bundles from the tagged commit. Record the commit hash in the change log.
2. Confirm the environment file contains every required variable, and that no secret is present in a `VITE_` variable.
3. Apply any database index changes for the release. Index creation is online in MongoDB and must precede the code that depends on it.
4. Stop and restart each service in dependency order: user-service, bill_service, then the supporting services.
5. Verify health: sign in as a staff user, load the dashboard, open a gate pass, and confirm the hotel portal rejects a staff token.
6. Publish the new static bundles. Because filenames are hashed, the old bundles remain valid until the proxy switch.
7. Watch the error log for five minutes, then declare the release complete — or roll back by restoring the previous bundle and the previous service version.

TABLE 54 — ROLLBACK DECISION POINTS

| SYMPTOM | ROLLBACK? | NOTE |
|---|---|---|
| **Sign-in failing for everyone** | Yes, immediately | Usually a key or configuration mismatch |
| **One screen erroring** | No | Fix forward; rollback does not restore data written since |
| **Bills wrong after release** | Yes, and stop writes | Preserve the database state before any further action |
| **Slow but functional** | No | Investigate before rolling back a working system |

> WARNING
>
> Never deploy on a Friday afternoon or immediately before a day close. A release that fails at close leaves the business unable to bill, and the person who can fix it is least available.

# 9 Disaster Management

*What can go wrong, how bad it would be, and who does what in the first hour.*

## 9.1 Framework and risk method

Disaster management here means the decisions made before an incident, so that during one nobody has to invent a process. The framework has four parts: a risk register that names and ranks the threats, a severity matrix that determines the response, a runbook that sequences the first hour, and a review that turns each incident into a change.

Risks are scored on likelihood and impact, each on a three-point scale. Likelihood reflects the platform's actual exposure — a single application host, one replica set, a small team — not a hypothetical adversary. Impact reflects the business consequence: whether the laundry can trade, whether records are lost, and whether obligations to clients are breached.

TABLE 55 — SCORING SCALES

| SCORE | LIKELIHOOD | IMPACT |
|---|---|---|
| **High** | Expected within a year on current controls | Trading stops, or data is lost or exposed |
| **Medium** | Possible within one to three years | A function is unavailable for hours, or figures are unreliable |
| **Low** | Unlikely without a change in circumstances | Local inconvenience, recoverable within a shift |

## 9.2 Risk register

The register is reviewed quarterly and after every Severity 1 or 2 incident. Likelihood and impact are re-scored then; an entry that has not changed for a year is either genuinely stable or not being examined.

TABLE 56 — RISK REGISTER

| RISK | LIKELIHOOD | IMPACT | MITIGATION | OWNER |
|---|---|---|---|---|
| **Database corruption or loss** | Low | High | Nightly logical backups, oplog retention, quarterly restore drills | Platform operator |
| **Staff credential compromise** | Medium | High | Short token lifetime, capability checks, sign-in auditing, rapid disable and key rotation | Laundry administrator |
| **Ransomware on the application host** | Low | High | Immutable off-host backups, least-privilege accounts, no interactive browsing on the host | Platform operator |
| **Insider misuse of discounts or voids** | Medium | Medium | Reason codes, supervisor approval, reprint detection, day-close reconciliation | Laundry owner |
| **Cloud region outage** | Low | High | Documented rebuild in a second region, configuration held outside the region, DNS switch | Platform operator |
| **Payment or messaging supplier outage** | Medium | Medium | Manual fallback recorded in the day book, reconciliation on recovery | Laundry manager |
| **Disk exhaustion on the database host** | Medium | High | Capacity alerts, log rotation, backup pruning, retention policy | Platform operator |
| **Key loss (KEK or signing key)** | Low | High | Escrowed KEK, documented rotation, staged key overlap | Platform operator |
| **Silent billing error after a release** | Medium | High | Suite gates, no Friday deploys, day-close review of unbilled items | Developer |

> NOTE
>
> The last entry is the one most often missed. A release that produces subtly wrong bills is worse than an outage, because nobody notices until clients do.

9.3 · THREAT SCENARIOS

# Threat scenario — database loss

**Scenario.** The operations database is lost or corrupted: a failed migration, an accidental drop, or storage failure on the primary. Sign-in still works, because user_db is intact, which makes the incident look smaller than it is — staff can reach a dashboard that shows nothing.

### Immediate signs

- Every list view returns empty or errors while sign-in succeeds.
- Service logs show connection failures or authentication errors against the operations database.
- The MongoDB primary is unavailable and no election completes.

### First actions

1. Stop writes. Take bill_service out of the proxy rotation so that no partial records are created against a broken store.
2. Confirm the scope: is the database unreachable, corrupted, or empty? These have different recovery paths.
3. Check the replica set state and the most recent successful backup timestamp before attempting any repair.
4. Declare severity: database loss affecting trading is Severity 1, and the response clock starts now.
5. Begin the restore procedure in 10.4 in the fixed order: user_db, operations, accounting.
6. Notify the laundry's manager with a factual statement and an expected next update time, not an estimate of recovery.

### What not to do

Do not run a repair against the only copy. Do not restore over a database that might still be recoverable in place. Do not restart every service at once — the restart destroys the evidence in memory and makes the timeline harder to reconstruct. And do not accept a restored database without the verification steps in 10.5; a restore that loses yesterday's bills is a second incident.

> WARNING
>
> If the loss is caused by a delete or a bad migration, point-in-time recovery (10.6) is almost always better than restoring last night's backup. Restore the backup only when the oplog cannot cover the window.

9.3 · THREAT SCENARIOS (CONTINUED)

## Threat scenario — credential compromise

**Scenario.** A staff password is phished, or a token is copied from a shared terminal. Because tokens are stateless, the attacker can act as that user until the token expires — up to 24 hours — and every action will be attributed to the real person.

### Detection

- Sign-ins from unfamiliar addresses or at unusual hours, visible in the authentication log.
- A burst of record creation or payment posting by one account.
- A user reporting that they were signed out unexpectedly, or that records exist they did not create.

### Containment

1. Disable the affected user immediately. This prevents new tokens even though it does not revoke the existing one.
2. Identify the capability of the compromised account: a STAFF token can create passes; an ADMIN token can change users.
3. If the account held ADMIN or the token may have reached a privileged surface, rotate the signing key to invalidate every session (3.6).
4. Preserve logs and audit entries before rotating, so the attacker's actions remain reconstructible.
5. Review the account's actions during the exposure window and reverse anything fraudulent through the normal correction paths — never by editing records directly.
6. Notify affected clients if their data was accessed, following 9.6.

**Afterwards**

Force a password reset for the user, review whether the password was reused elsewhere, and check whether the compromise came from a shared terminal. The most common root cause is not a sophisticated attack but a password written beside a counter terminal, and the control that fixes it is a lock screen and individual accounts.

> WARNING
>
> Do not simply change the user's password. The existing token remains valid until expiry; disabling the account or rotating the key is the only way to stop it.

9.3 · THREAT SCENARIOS (CONTINUED)

# Threat scenario — ransomware and insider misuse

## Ransomware on the application host

**Scenario.** The host is encrypted by ransomware, either through a compromised remote-access account or through a downloaded file. The application stops; the database may be on the same host or reachable from it.

1. Isolate the host from the network immediately. Do not shut it down if evidence may be needed, but do isolate it.
2. Confirm that off-host backups exist and are immutable. This is the moment the design decision in 10.1 proves itself.
3. Do not pay, and do not attempt decryption on the production host.
4. Build a clean host from the documented deployment steps (8.5), then restore data (10.4).
5. Rotate every credential the host could read: database users, storage keys, signing keys, KEK if it was present on the host.
6. Report the incident, including any personal data exposure, per 9.6.

## Insider misuse

**Scenario.** A member of staff issues refunds to themselves, voids bills after taking cash, applies discounts for friends, or exports client data. The pattern is usually small and repeated rather than large and obvious.

- **Prevention:** reason codes, supervisor approval for refunds and discounts, capability separation between counter staff and managers, and no shared logins.
- **Detection:** review refund and void frequency per user, compare discounts against the standard, and reconcile the drawer daily rather than monthly.
- **Response:** preserve the audit records, involve the owner before confronting anyone, and correct the financial records through adjustment documents so that the client's statement stays true.

> NOTE
>
> Audit records are append-only and are the reason insider misuse is provable at all. Any proposal to allow editing or deleting audit entries should be refused on these grounds alone.

9.3 · THREAT SCENARIOS (CONTINUED)

# Threat scenario — region outage and supplier outage

## Cloud region outage

**Scenario.** The region hosting the application and database becomes unavailable: a provider incident, a network partition, or a sustained control-plane failure. Both the application and its primary database are unreachable, and the provider's own console may be degraded too.

1. Confirm the outage is the provider's, not yours: check status pages from outside the region and test connectivity from a second location.
2. Declare Severity 2 (or 1 if trading is impossible during business hours) and start the communication plan in 9.7.
3. Do not attempt to rebuild in a second region until the provider confirms the primary region's state; a split-brain restore is worse than an outage.
4. When confirmed unrecoverable within the business window, provision the second region from the documented steps, restore the latest off-host backup, and repoint DNS.
5. Accept and record the data loss window: work recorded after the last backup but before the outage cannot be reconstructed from the platform and must be recovered from paper.

## Supplier outage

**Scenario.** A payment terminal, SMS provider or email provider fails. The platform is healthy but one step of the business cannot complete.

TABLE 57 — SUPPLIER FAILURE FALLBACKS

| SUPPLIER | IMPACT | FALLBACK |
|---|---|---|
| **Card terminal** | Card payments cannot be taken | Record cash only, or record the card sale manually and reconcile when the terminal returns |
| **SMS or email** | Clients do not receive notifications | Phone the client; the record is unaffected |
| **Object storage** | Backups fail; document downloads may fail | Keep the local backup copy, alert immediately, and restore the storage path before the next scheduled backup |

NOTE

A supplier outage is not a reason to bypass a control. Recording a card payment as cash to "keep moving" corrupts the day-close reconciliation and takes longer to unwind than the original outage.

9.4 · INCIDENT SEVERITY MATRIX

# Incident severity matrix

Severity determines who is called, how fast, and what is communicated. It is assigned at declaration and may be raised once, never lowered during the incident — a lower severity mid-incident invites people to stand down too early.

TABLE 58 — SEVERITY LEVELS, EXAMPLES AND TARGETS

| SEVERITY | DEFINITION | EXAMPLES | RESPONSE TARGET |
|---|---|---|---|
| **SEV1** | Trading impossible, data loss or exposure | Database lost; sign-in broken for all; ransomware | Acknowledge 15 min, contain 1 h, update every 30 min |
| **SEV2** | A major function unavailable, workaround exists | Hotel portal down; billing blocked for one client; region degraded | Acknowledge 30 min, mitigate 4 h, update hourly |
| **SEV3** | A minor function degraded, business continues | One report failing; tag printing slow; single user blocked | Acknowledge 4 h, resolve 2 business days |
| **SEV4** | Cosmetic or single-user inconvenience | Label misaligned; a filter not remembered | Next planned release |

### Who decides

The first responder declares the severity using this table, and the platform operator confirms or raises it. When in doubt, declare higher: the cost of an unnecessary SEV1 is a wasted hour, and the cost of a late SEV1 is a day of unbilled work.

### Escalation

SEV1 involves the platform operator, the laundry owner and — where personal data may be involved — the person responsible for data protection. SEV2 involves the operator and the laundry manager. SEV3 and SEV4 are handled by the operator alone and reported in the weekly summary.

WARNING

Severity is about business impact, not about how interesting the technical cause is. A dull configuration error that stops billing is SEV1; an exotic bug in a report nobody uses is SEV3.

9.5 · RESPONSE RUNBOOK

# Response runbook

The runbook is written to be followed in the first hour, when information is incomplete and everyone is busy. Times are targets measured from declaration, not aspirations.

1. **T+0 — Declare.** Name the incident, assign a severity, open a timeline document with the time and the reporter. One person owns the timeline.
2. **T+5 min — Stop the harm.** Isolate the affected component: remove a service from rotation, disable an account, block an address. Containment before diagnosis.
3. **T+10 min — Assess data.** Determine whether data is lost, exposed, or merely unavailable. This single question sets the recovery path and the notification obligation.
4. **T+15 min — Notify.** SEV1: operator and owner by phone. SEV2: operator, then manager. State facts and the next update time; do not estimate recovery.
5. **T+30 min — Stabilise.** Restore service from backup, fail over, or apply the documented workaround. Record every command that changes state.
6. **T+60 min — Verify.** Run the smoke checks: sign-in, dashboard, one gate pass, one bill, one hotel portal read. Confirm with a real user, not only with a health endpoint.
7. **T+4 h — Reconcile.** Identify work performed on paper during the outage, enter it with the correct dates, and note the delay in the record.
8. **T+24 h — Review.** Hold the post-incident review within one business day, while memory is fresh; publish actions with owners and dates.

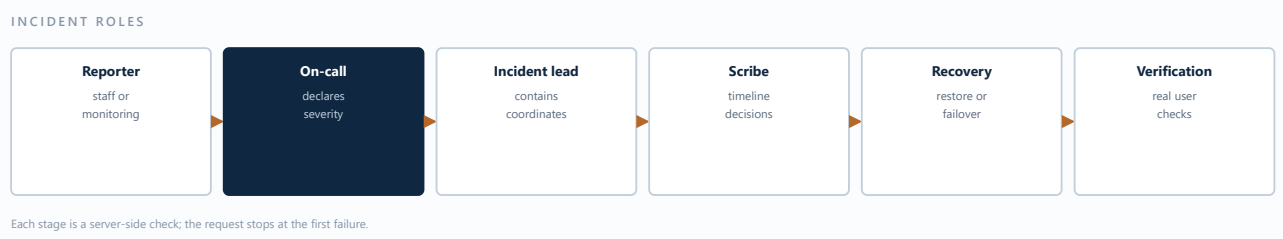



FIGURE 21 — INCIDENT ROLES. IN A SMALL TEAM ONE PERSON MAY HOLD TWO ROLES, BUT THE SCRIBE ROLE IS NEVER DROPPED.

> NOTE
>
> The scribe is the most valuable role in a long incident and the first one omitted. A timeline written during the event is worth more than a reconstruction afterwards.

9.6 · COMMUNICATION AND REVIEW

# Communication plan and post-incident review

## 9.6 Communication plan

TABLE 59 — WHO IS TOLD WHAT, AND WHEN

| AUDIENCE | TRIGGER | CHANNEL AND CONTENT |
|---|---|---|
| **Laundry owner** | SEV1, SEV2 | Phone within 15 minutes: what is affected, what is being done, next update time |
| **Laundry manager and supervisors** | Any severity affecting trading | Direct message: workaround instructions and what to record on paper |
| **Hotel clients** | Their records or invoices are affected | Written notice from the laundry: what happened, what was corrected, what they should check |
| **All staff** | SEV1, or any change to procedure | Notice at next sign-in or shift briefing |
| **Data protection contact** | Personal data may have been accessed | Immediate written notice with the facts known at the time |

Communication during an incident is factual and time-bound. Say what is known, what is not known, and when the next update will come. Never speculate about cause or recovery time, and never blame a named individual — the review establishes cause, not the incident channel.

### 9.7 Post-incident review

Every SEV1 and SEV2 incident gets a written review within one business day. The review is blameless in method but specific in outcome: it must produce changes with owners and dates, not a narrative of what everyone did wrong.

TABLE 60 — REVIEW AGENDA

| SECTION | CONTENT |
|---|---|
| **Timeline** | Detection, declaration, containment, recovery, verification — with times |
| **Impact** | What was unavailable, for how long, and which records were affected |
| **Cause** | The technical cause and the contributing conditions that let it matter |
| **What worked** | Controls that did their job, so they are not removed in the next redesign |
| **Actions** | Specific changes, each with an owner and a due date |
| **Verification** | How the fix will be tested, and when the review is closed |

> WARNING
>
> An action without an owner and a date is a wish. If a review produces more than five actions, it has produced none that will be done — choose the ones that change the outcome.

9.8 · INCIDENT SEVERITY MATRIX

## Incident severity matrix

*One table that decides who is called and how fast.*

INCIDENT SEVERITY MATRIX

| | Business impact | Response target | Who is called |
|---|---|---|---|
| **SEV1** severity | Trading stops; data lost or exposed assigned at declaration; may be raised, never lowered | Acknowledge 15 min · contain 1 h | Operator, owner, DPO |
| **SEV2** severity | Major function unavailable assigned at declaration; may be raised, never lowered | Acknowledge 30 min · mitigate 4 h | Operator, manager |
| **SEV3** severity | Minor function degraded assigned at declaration; may be raised, never lowered | Acknowledge 4 h · resolve 2 days | Operator |
| **SEV4** severity | Cosmetic or single user assigned at declaration; may be raised, never lowered | Next planned release | Backlog |

When in doubt, declare higher: a wasted SEV1 costs an hour, a late one costs a day of unbilled work.

**FIGURE 22** — SEVERITY BANDS WITH RESPONSE TARGETS AND THE PEOPLE THEY SUMMON.

Severity is assigned at declaration, from business impact rather than technical curiosity. A dull configuration error that stops billing is Sev1; an exotic defect in a report nobody reads is Sev3.

The matrix is read downwards, not upwards: start at the impact, then take the response target that belongs to it. The target is measured from declaration, not from the moment someone understands the cause.

> RULE
>
> Severity may be raised during an incident and is never lowered. Lowering it mid-incident invites people to stand down before the work is finished.

10

# Backup and Recovery

*What is protected, how quickly it must come back, and how the restore is proved rather than assumed.*

## 10.1 What is backed up

Backups exist to restore a business, not to archive a server. The inventory below therefore follows the three-database model: everything that cannot be rebuilt from source is backed up, and everything that can be rebuilt is not.

TABLE 61 — BACKUP INVENTORY

| ITEM | METHOD | WHY IT IS BACKED UP |
|---|---|---|
| **user_db** | Nightly logical dump plus continuous oplog | Identity and tenancy; without it nobody can sign in |
| **Operations database** | Nightly logical dump plus continuous oplog | Gate passes, deliveries, bills, linen history — the trading record |
| **Accounting database** | Nightly logical dump | Ledger and payroll; changes slowly, restored independently |
| **Uploaded documents and printed artefacts** | Object-storage versioning | Bill PDFs and attachments referenced by records |
| **Configuration** | Version-controlled environment templates, secrets in the key store | A host can be rebuilt only if its configuration is known |
| **KEK and search-key escrow** | Separate secure escrow, distinct access policy | Encrypted fields are unreadable without them |
| **Audit log** | Append-only log pipeline with extended retention | Needed for investigation after an incident |

### What is deliberately not backed up

Front-end bundles are rebuilt from the tagged commit, not backed up as files. Container images are rebuilt from the same commit. Temporary caches, session state and search indexes hold nothing that cannot be reconstructed. Backing these up would lengthen every restore without changing the outcome.

> WARNING
>
> A backup that has never been restored is a hypothesis. The drill in 10.7 exists because the most common backup failure is a backup that completes successfully and cannot be read.

10.2 · RPO AND RTO TARGETS

## Recovery objectives

Recovery point objective (RPO) is how much data the business can afford to lose. Recovery time objective (RTO) is how long it can afford to be without the function. Both are business statements that the technical design must meet — not measurements of what happens to be possible.

TABLE 62 — TARGETS PER COMPONENT

| COMPONENT | RPO | RTO | BASIS |
|---|---|---|---|
| **user_db** | 15 minutes | 1 hour | Oplog replay from the last dump; identity is a precondition for everything |
| **Operations database** | 15 minutes | 2 hours | Oplog replay; the trading record, largest restore |
| **Accounting database** | 24 hours | 4 hours | Daily dump is sufficient; restores independently |
| **Object storage** | 24 hours | 1 hour | Versioning; documents referenced by records |
| **Application host** | Not applicable | 2 hours | Rebuilt from configuration, no state |
| **Full estate** | 15 minutes | 4 hours | Sequential restore in dependency order |

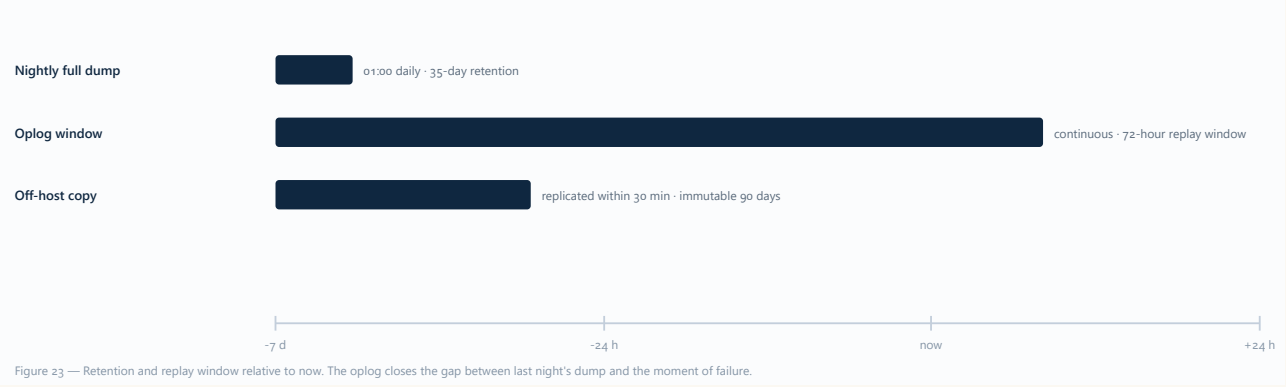



Figure 23 — Retention and replay window relative to now. The oplog closes the gap between last night's dump and the moment of failure.

FIGURE 23 — BACKUP, RETENTION AND POINT-IN-TIME RECOVERY TIMELINE.

> NOTE
>
> The 15-minute RPO depends on the oplog being intact. If the oplog is lost with the database, the achievable RPO becomes 24 hours, which is why the oplog is replicated off-host and monitored as a first-class backup artefact.

10.3 · BACKUP SCHEDULE

# Backup schedule

Backups run on a schedule, are copied off-host, and are verified by a job that reads them back. A backup job that reports success without a verification step is reporting that a file was written, not that it can be used.

TABLE 63 — SCHEDULE AND RETENTION

| JOB | FREQUENCY | RETENTION | DESTINATION |
|---|---|---|---|
| **user_db logical dump** | Daily 01:00 | 35 daily, 12 monthly | Object storage, separate account |
| **Operations dump** | Daily 01:15 | 35 daily, 12 monthly | Object storage, separate account |
| **Accounting dump** | Daily 01:30 | 35 daily, 24 monthly | Object storage, separate account |
| **Oplog archive** | Continuous | 72 hours | Object storage, separate account |
| **Configuration snapshot** | On every deployment | Indefinite (version control) | Repository |
| **Restore verification** | Weekly | Latest verified copy recorded | Isolated environment |
| **Full drill** | Quarterly | Report retained 2 years | Isolated environment |

### Ordering and overlap

The three dumps are staggered so that they do not compete for disk or network, and so that a failure in one does not delay the others. The operations dump is the longest and runs second; the accounting dump is small and runs last. Every job writes a manifest containing the start time, end time, byte count, database name and checksum, which is what the verification job reads.

### Monitoring the schedule

- A missed job raises an alert within 15 minutes of its expected completion.
- A job that completes with a size more than 20 percent below its recent average is treated as suspect and alerted, not accepted.
- Two consecutive verification failures are a Severity 2 incident, because the business is one failure away from having no usable backup.

> WARNING
> Never store the only backup copy on the host being backed up. Ransomware and disk failure both remove the backup and the data in one event.

10.4 · RESTORE PROCEDURE

## Restore procedure

Restore order is fixed: user_db first, operations second, accounting third. Operations documents reference users and memberships; accounting references operational documents. Restoring out of order produces a database that looks complete and is internally inconsistent.

**Procedure — restoring to a clean host**

1. **Freeze.** Take the application out of service so that no writes occur during the restore. Announce the maintenance window.
2. **Provision.** Build the host and install the database version recorded in the release notes. Do not restore into a different major version without a tested upgrade path.
3. **Fetch.** Retrieve the chosen dump and its manifest from object storage. Verify the checksum against the manifest before use.
4. **Restore user_db.** Load the dump, then apply oplog entries up to the target time if point-in-time recovery is required.
5. **Verify identity.** Confirm that users, laundries and memberships are present and that a known administrator can authenticate.
6. **Restore operations.** Load the dump into the operations database and replay the oplog to the same target time.
7. **Restore accounting.** Load the accounting dump, then reconcile its payment totals against the restored operations data.
8. **Re-encrypt check.** Confirm that a known encrypted field decrypts with the escrowed KEK. A restore with the wrong key is indistinguishable from data loss until someone opens a record.
9. **Start services.** Bring services up in dependency order and run the smoke checks in 10.5.
10. **Record.** Write the restore record: operator, dump identifiers, target time, duration, verification results.

> WARNING
> Do not restore over a live database in place. Restore into a fresh database, verify, then switch the connection string. An in-place restore that fails halfway leaves neither the old data nor the new.

10.5 · VERIFICATION AND INTEGRITY CHECKS

# Verification and integrity checks

A restore is complete when the data is proved usable, not when the load command exits. The checks below run in order; the first failure stops the process and the restore is repeated from the previous step.

TABLE 64 — POST-RESTORE CHECKS

| CHECK | METHOD | PASS CONDITION |
|---|---|---|
| **Row counts** | Count per collection against the manifest | Within the recorded tolerance; no empty core collection |
| **Identity** | Sign in as a known administrator | Sign-in succeeds and the tenant resolves |
| **Tenancy** | Request a record with a foreign tenant identifier | Returns not-found, never a record |
| **Financial continuity** | Sum bill totals, payments and balances | Balances equal totals less payments; no negative balance without a credit record |
| **Referential integrity** | Deliveries without a pass; bills without lines | Zero orphans |
| **Linen consistency** | Tags in a status with no corresponding event | Zero, or explained by the restore window |
| **Encryption** | Open a record with an encrypted field | Decrypts correctly; no key errors in the log |
| **Recent activity** | Open the last gate pass before the failure | Present with its deliveries intact |

### Integrity queries worth keeping

The financial check is the one that catches a partial restore most reliably, because a half-loaded set of payments still looks plausible in a list but cannot balance. The orphan check catches a restore that stopped between collections, which is exactly what happens when a restore is interrupted and resumed incorrectly.

```
// deliveries whose gate pass is absent
{"$lookup": {"from": "gate_passes", "localField": "gate_pass_id",
             "foreignField": "_id", "as": "gp"}},
{"$match": {"gp": []}}
```

> NOTE
>
> Keep the verification queries in version control alongside the restore script. A check that exists only in someone's memory is not a control.

10.6 · POINT-IN-TIME RECOVERY

# Point-in-time recovery

Point-in-time recovery (PITR) restores the database to a moment before a damaging change — a bad migration, a mistaken delete, a corrupting script. It combines the last full dump with oplog entries replayed up to, but not including, the harmful operation.

### Procedure — recovering to a chosen moment

1. Identify the exact time of the harmful change from the audit log or the application log. Precision matters: replaying one minute too far re-applies the damage.
2. Confirm that the required window is inside the oplog retention. Outside it, only a full restore is possible.
3. Restore the most recent dump that precedes the target time into a fresh database.
4. Replay oplog entries from the dump's end time to the target time, excluding the harmful operation.
5. Verify with the checks in 10.5, then switch the connection string and restart the services.
6. Reconcile any work performed after the target time from paper or from client records, and enter it with its true date and a note explaining the gap.

TABLE 65 — CHOOSING THE RECOVERY POINT

| SITUATION | TARGET TIME | RATIONALE |
|---|---|---|
| **Accidental delete of one collection** | Just before the delete | Minimises lost legitimate work |
| **Bad migration altering many records** | Just before the migration started | The migration's effects are the damage |
| **Corruption of unknown origin** | Last verified good state | Prefer a known-good point to a guessed one |
| **Ransomware** | Last backup before encryption began | Assume everything after is untrusted |

> WARNING
>
> Replaying to the exact second of a transaction boundary can leave a bill without its payment. Choose a recovery point at a quiet moment — end of day, or after a close — whenever the business can tolerate it.

10.7 · RESTORE DRILL CHECKLIST

## Restore drill checklist

The drill is performed quarterly against an isolated environment, using the same artefacts and the same procedure as a real recovery. It is timed, and the result is recorded — including the parts that failed.

TABLE 66 — DRILL CHECKLIST

| STEP | CHECK | EVIDENCE RECORDED |
|---|---|---|
| 1 | Retrieve the latest dump and verify its checksum | Dump identifier, checksum, retrieval time |
| 2 | Provision an isolated host with the recorded versions | Host identifier, versions |
| 3 | Restore user_db and authenticate as a known administrator | Time to first successful sign-in |
| 4 | Restore operations and accounting databases | Time per database, total elapsed |
| 5 | Run the integrity checks in 10.5 | Each check's result, including failures |
| 6 | Decrypt a known sensitive field with the escrowed KEK | Field identifier, result |
| 7 | Replay a five-minute oplog window | Target time, entries applied |
| 8 | Run the smoke tests: pass, delivery, bill, hotel read | Screenshots or ids of created records |
| 9 | Compare achieved RPO and RTO with the targets | Measured values against 10.2 |
| 10 | Destroy the isolated environment | Destruction time, operator |

### What the drill must produce

- A measured restore duration, compared with the RTO. A drill that is not timed does not test the objective.
- A list of every step that required improvisation, because each is a gap in this chapter.
- Confirmation that the KEK escrow works from a clean environment, not from the operator's workstation.
- An updated procedure if any step was wrong. The manual is corrected the same week, not at the next review.

> NOTE
>
> Record failures in the drill report. A clean report that everyone knows was helped along is worse than a failed drill that gets fixed.

10.8 · BACKUP AND RECOVERY TIMELINE

## Backup and recovery timeline

*Three overlapping protections, and the fifteen minutes that separates them.*

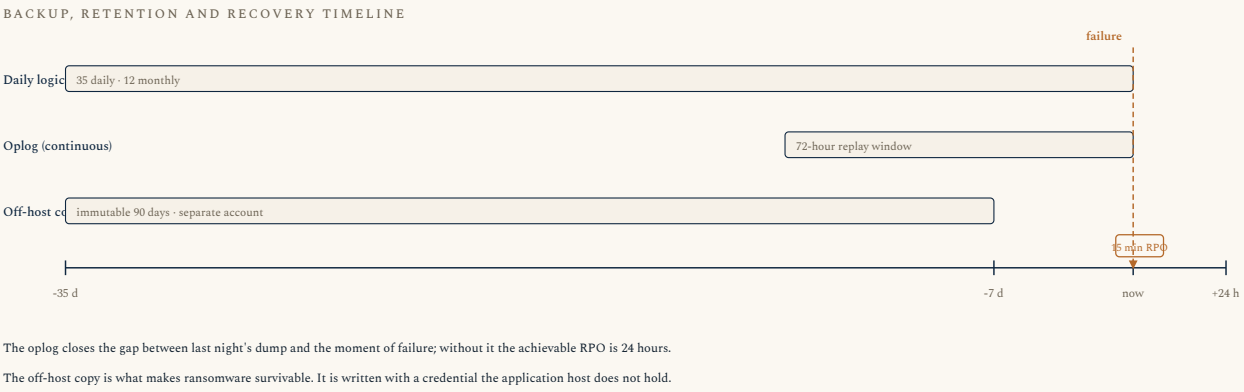



**FIGURE 24** — DAILY DUMPS, CONTINUOUS OPLOG AND THE OFF-HOST COPY, AGAINST THE RECOVERY-POINT TARGET.

The restore procedure itself begins on page 98, and the drill checklist on page 100. The timeline explains why the recovery-point target is fifteen minutes rather than a day. The nightly dump sets the floor, the oplog closes the gap to the moment of failure, and the off-host copy survives the loss of the host that wrote it.

Each layer answers a different failure. The dump answers corruption, the oplog answers a bad change made an hour ago, and the off-host copy answers the loss of the account in which everything else lives.

> RULE
>
> The oplog is a backup artefact, not a log to be pruned for space. If it is lost with the database, the achievable recovery point becomes the last dump.

11

# Fail-Safes and Resilience

*How the platform behaves when something is missing, slow, or called twice.*

## 11.1 Fail-closed tenancy

The platform's most important fail-safe is also its simplest: when the tenant cannot be established, nothing happens. There is no default tenant, no fallback to an unscoped query, and no "system" mode that skips the check. A request that cannot name its laundry is refused before it reaches any data.

TABLE 67 — FAILURE CONDITIONS AND OUTCOMES

| CONDITION | OUTCOME | WHY |
|---|---|---|
| **No token** | 401 | Identity is unknown |
| **Expired or badly signed token** | 401 | Identity cannot be trusted |
| **Client token on a staff route** | 403 | No tenant claim exists to resolve |
| **Staff token with no laundry_id** | 403 | Tenant context missing; fail closed |
| **Membership removed since issue** | 403 after the account is disabled | Capabilities are snapshot at sign-in |
| **Foreign identifier in a path** | 404 | Indistinguishable from a non-existent record |

### Why fail closed rather than fail open

A fail-open design answers a missing tenant with "all tenants", which turns a configuration mistake into a data breach. Fail-closed turns the same mistake into an outage — visible, recoverable, and confined to the person who caused it. For a multi-tenant system holding other businesses' commercial records, an outage is the cheaper failure every time.

> NOTE
>
> The same principle applies to capabilities: an unknown or missing capability is treated as absent, never as permitted. New routes must declare a capability or they cannot be reached at all.

11.2 · IDEMPOTENCY AND CONCURRENCY

# Idempotency and optimistic concurrency

Two failures cause most duplicate records in operational systems: a retried request that actually succeeded, and two users editing the same document. The platform addresses both explicitly rather than hoping they are rare.

## Idempotency keys

Every write that creates money or possession — bill, payment, delivery, gate pass — accepts an idempotency key generated by the client. The server records the key with the result. A retry with the same key returns the original result instead of creating a second record, whether the first attempt succeeded, failed, or is still in progress.




FIGURE 25 — IDEMPOTENT WRITE PATH. THE KEY AND THE RECORD ARE COMMITTED TOGETHER, SO A CRASH CANNOT LEAVE ONE WITHOUT THE OTHER.

## Optimistic concurrency

Documents that can be edited carry a version. An update must state the version it read; if the stored version has moved, the write is rejected with a conflict instead of silently overwriting a colleague's change. The user is shown both versions and chooses — which is slower by one interaction and correct by construction.

TABLE 68 — CONCURRENCY OUTCOMES

| SITUATION | RESPONSE | USER EXPERIENCE |
|---|---|---|
| **Same document, sequential edits** | Accepted; version increments | Normal editing |
| **Same document, simultaneous edits** | Second write rejected with 409 | Message shows the other change; reload and retry |
| **Retry after timeout** | Idempotency key returns the original result | The action appears to have succeeded, because it did |
| **Duplicate key without a stored result** | Write rejected as in-progress | The user is asked to wait rather than double-submit |

NOTE

Idempotency keys are generated per form submission, not per user action. Reopening a bill and paying again is a new intent and gets a new key, which is why the confirmation step matters.

11.3 · ATOMIC BILL AND PAYMENT TRANSITIONS

# Atomic bill and payment transitions

Money movements are the one place where a partial write is unacceptable. A payment that reduces a balance without being recorded, or a bill that is issued without its lines, is worse than a failed request because it is invisible until reconciliation.

### What must happen together

TABLE 69 — ATOMIC UNITS

| OPERATION | WRITES THAT MUST COMMIT TOGETHER |
|---|---|
| **Issue bill** | Bill document with lines and totals; sequence number; client balance |
| **Post payment** | Payment record; bill balance; receipt reference; audit entry |
| **Reverse payment** | Reversal record referencing the original; restored balance; audit entry |
| **Record delivery** | Delivery document; pass line delivered quantities; pass status |
| **Close day** | Day-close record with totals; frozen date marker; audit entry |

MongoDB multi-document transactions make each of these a single unit. The platform uses them only where the invariant spans documents — a transaction has a cost, and paying it on a read is waste. Where a single document holds the whole invariant, an atomic update on that document is sufficient and cheaper.

### Why the bill is not edited after issue

An issued bill is a document the client may already hold. Editing it would change the meaning of a record that exists outside the platform, and no transaction can undo that. Corrections are therefore new documents — credit notes, adjustments, reversals — each referencing the original, so the history remains a sequence of facts rather than a current opinion.

WARNING

Never “fix” a balance by writing the field directly. A balance is derived from records; writing it creates a number that no document supports, and the next reconciliation will fail with no explanation.

11.4 · OFFLINE CACHE, RETRIES AND CIRCUIT BREAKING

# Offline cache, retries and circuit breaking

A laundry is a physical place with unreliable connectivity: a basement linen room, a counter by a steel door, a driver's phone. The front ends are built to stay useful for reading when the network is poor, and to refuse to pretend when it is absent.

### Offline cache behaviour

- **Cached reads, never cached writes.** Lists and detail views may be served from the last successful response, clearly marked as cached with the time it was fetched.
- **No queued writes.** The platform does not store a write locally and replay it later with an invented timestamp. A scan recorded at 14:05 must not enter the system as 09:12.
- **Explicit staleness.** A cached view says when it was last updated; a user who sees "as of 09:12" knows to phone rather than assume.
- **Sign-out clears the cache.** Cached business data does not survive a session on a shared terminal.

### Retries and queues

Reads retry automatically with exponential backoff and jitter, up to a small limit. Writes retry only when the request is idempotent, and always with the same key, so that a retry cannot create a second bill. Where a write triggers a cross-service side effect — mirroring a payment into accounting — the platform writes a durable outbox record in the same transaction as the business write and processes it asynchronously. If the accounting service is down, the payment still posts; the outbox retries until it succeeds, and the discrepancy is visible rather than lost.

### Circuit breaking

When a dependency fails repeatedly, the caller stops calling it for a short period and fails fast instead of queueing requests that will time out. This protects the healthy services from thread and connection exhaustion caused by one sick dependency. The breaker closes automatically after a successful probe, and its state is exposed as a health detail so that an operator can see "degraded because accounting is down" rather than a generic error.

> NOTE
>
> Fail fast is a feature. A request that waits thirty seconds and then fails is worse for the user and for the system than one that fails in fifty milliseconds and says why.

11.5 · DEGRADED-MODE MATRIX

## Degraded-mode matrix

The matrix states what still works when a component is unavailable. It is written from the operator's point of view: during an incident, the question is never "which service is down?" but "can we still take in laundry and bill for it?"

TABLE 70 — CAPABILITY BY FAILED COMPONENT

| IF THIS IS DOWN… | STILL WORKS | STOPS | WORKAROUND |
|---|---|---|---|
| **user-service** | Nothing authenticated | Everything | Restore identity first — it is the precondition for all else |
| **bill_service** | Sign-in; quotations; accounting reads | Passes, deliveries, bills, linen, shop, hotel portal | Paper records with times; enter after recovery |
| **laundry-management** | All operational work | Accounting posting, payroll, expense entry | Outbox queues payments; reconcile after recovery |
| **quotation-service** | All operational and financial work | Quotation drafting and AI chat | Price manually and record once service returns |
| **worker-service** | All operational work | Worker lists and attendance | Record attendance on paper for the shift |
| **MongoDB primary** | Reads served by secondary, briefly | All writes | Wait for election; do not force a primary |
| **Object storage** | All application functions | Backups, document download | Alert immediately; local backup copy retained |
| **Reverse proxy** | Nothing from outside | All client access | Restore proxy config; services remain healthy |

### Reading the matrix in an incident

The first column that is empty tells you the severity. If sign-in is down, everything is down and the incident is SEV1 regardless of the technical cause. If only accounting is down, the laundry can trade all day and the incident is SEV2 with an outbox reconciliation afterwards.

> WARNING
>
> A workaround that bypasses a control is not a workaround. Paper records are acceptable; entering a bill twice "to be safe", or marking a card payment as cash, is not.

11.6 · GUARD RAILS AGAINST DOUBLE BILLING

## Guard rails against double billing

Double billing is the failure clients notice fastest and forgive slowest. The platform defends against it at four layers, so that no single mistake — human or technical — can produce a duplicate charge.

TABLE 71 — FOUR LAYERS OF DEFENCE

| LAYER | CONTROL | WHAT IT PREVENTS |
|---|---|---|
| **Data** | A delivery line can belong to only one bill | The same delivered quantity being billed twice |
| **Transaction** | Bill issue and delivery marking commit together | A bill issued without consuming its deliveries |
| **Request** | Idempotency key on bill creation | A retried issue creating a second bill |
| **Process** | Day-close review of unbilled deliveries and duplicate totals | Anything the first three layers missed |

### How a duplicate would still be caught

1. The bill list shows consecutive bills for the same client with identical totals and periods — flagged in the day-close review.
2. A delivery already consumed by a bill is not offered when raising the next bill, so the second attempt produces an empty selection rather than a duplicate.
3. A client statement shows both invoices with their numbers; the hotel portal makes the duplicate visible to the client the same day.
4. The audit trail records who issued each bill and when, which distinguishes a duplicate from a legitimate re-issue.

### Correcting a duplicate

Void the duplicate with a reason and re-check the client's balance. Do not delete it: a voided bill keeps its number and remains visible, which is what allows the client's copy and the platform's record to agree. If the duplicate has been paid, reverse the payment and void the bill, then issue a credit note if the client's money must be returned through accounting.

> NOTE
>
> Every guard rail here is also a test in the bill_service suite. The controls and the tests are maintained together, because a guard rail that is not tested is a comment.

11.7 · GUARD LAYERS AGAINST DOUBLE BILLING

## Guard layers against double billing

*Four independent defences, so that no single mistake produces a duplicate charge.*

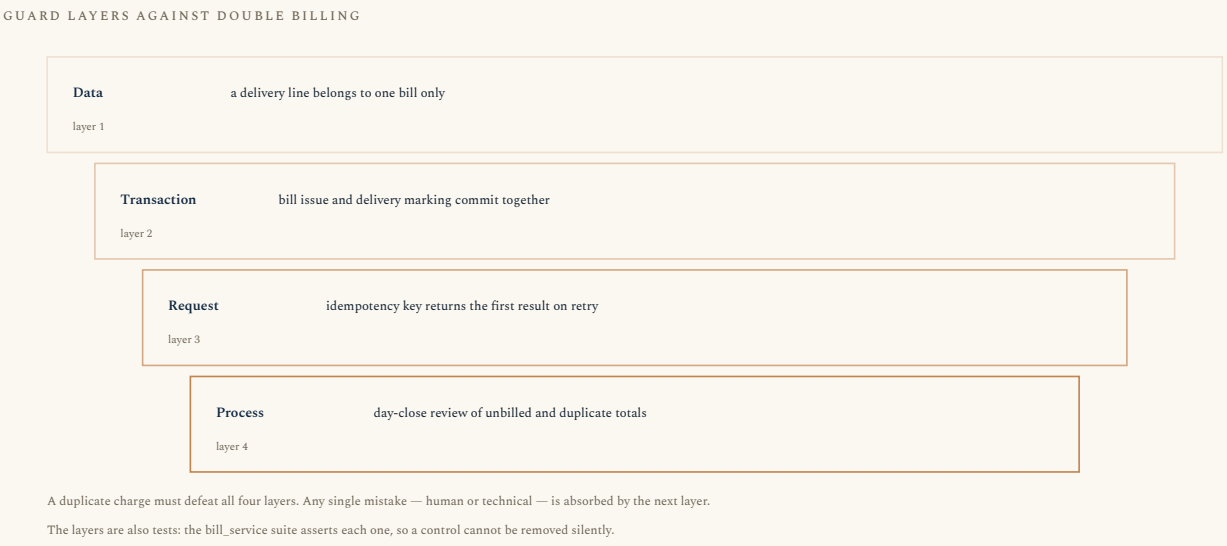



FIGURE 26 — LAYERED DEFENCES AGAINST DOUBLE BILLING, FROM THE DATA MODEL TO THE DAY-CLOSE REVIEW.

A duplicate charge is the failure clients notice fastest and forgive slowest, so the platform does not rely on one control. The data model prevents a delivered line from belonging to two bills; the transaction prevents a bill from existing without consuming its lines; the idempotency key prevents a retry from creating a second document.

The fourth layer is human and deliberately boring: the day-close review looks for consecutive bills with identical totals. It exists to catch the combinations the first three layers cannot foresee.

> RULE
>
> Each layer is also a test in the bill_service suite. A control that is not asserted is a comment, and comments do not survive refactoring.

# IV

PART FOUR

# Assurance and Reference


12 Testing and Quality Assurance

13 Operations Runbook and Monitoring

A Appendices

12

# Testing and Quality Assurance

*What is tested, what the suites actually report, and what remains unproven.*

## 12.1 Test strategy

The strategy is a pyramid, and it is deliberately bottom-heavy. Most behaviour is verified by fast service-level tests that exercise real handlers against a test database; a smaller number of integration tests cover cross-service contracts; and the top of the pyramid — where confidence is most expensive — is thin. That is a deliberate trade: a browser-driven suite for four applications would be slow, brittle, and rarely run.

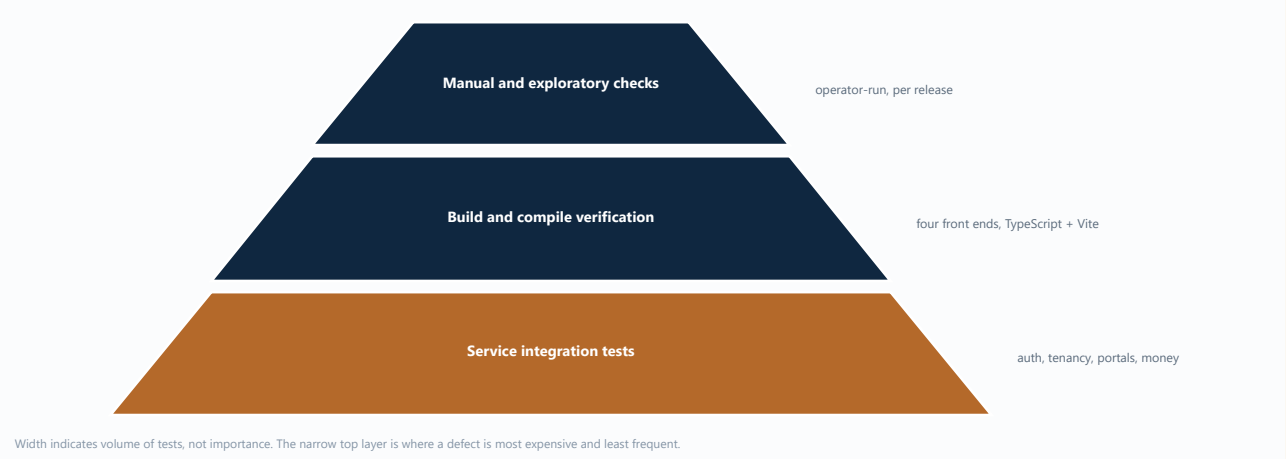



FIGURE 27 — TEST STRATEGY PYRAMID: BROAD SERVICE-LEVEL COVERAGE, THIN TOP LAYER.

TABLE 72 — TEST LAYERS AND THEIR SCOPE

| LAYER | EXERCISES | RUNS IN |
|---|---|---|
| **Unit** | Pure functions: totals, tax, status transitions, token parsing | Seconds, no database |
| **Service** | HTTP handlers with authentication and a test database | The service's own suite |
| **Integration** | Cross-service flows: login → pass → delivery → bill; client portal scope | bill_service and user-service suites |
| **Build** | TypeScript compilation and Vite production build for all four apps | CI or local release check |
| **Manual** | Exploratory checks of new screens and print layouts | Before each release |

> NOTE
>
> Tests run against a real MongoDB instance, not a mock. Tenancy and transaction behaviour are precisely the things a mock cannot reproduce, and they are the properties most worth testing.

12.2 · MEASURED RESULTS

## Measured results

The figures below are measured from the suites as they stand. They are reported as counts of passing tests, not as coverage percentages, because a coverage number would say more about the size of the codebase than about whether the important behaviour is verified.

TABLE 73 — MEASURED TEST RESULTS

| SUITE | RESULT | NOTES |
|---|---|---|
| **user-service** | 68 tests passing | Authentication, users, laundries, memberships, client accounts, token claims |
| **bill_service** | 380 passing, 2 skipped | Operational core: passes, deliveries, bills, shop bills, linen lifecycle, returns, matrix, dispatch, day close, dashboard, client portal |
| **— of which client portal** | 13 tests | Included in the 380; covers scope filtering, summary, passes, deliveries, bills |
| **shop-app** | Compiles and builds | TypeScript compilation and Vite production build succeed |
| **hotel-app** | Compiles and builds | TypeScript compilation and Vite production build succeed |
| **linen-room-app** | Compiles and builds | TypeScript compilation and Vite production build succeed |
| **quotations-ui** | Builds | Existing application; build verified as part of the release check |

### How to read these numbers

A passing suite means the behaviour it asserts still holds. It does not mean the product is defect-free, and the two skipped tests are skipped deliberately rather than hidden: they cover paths that require an external dependency not configured in the current environment. Chapter 12.3 states the gaps plainly.

### What is deliberately not claimed

- No coverage percentage is quoted, because line coverage would be misleading for a codebase with a large amount of declarative schema.
- No performance figures are quoted, because no load test has been run.
- No browser-level pass rate is quoted, because there is no browser-driven end-to-end suite.

> NOTE
> Reporting a measured 380 rather than a rounded “over 400” is itself part of the quality claim. The manual should not be more confident than the suites.

12.3 · COVERAGE AND GAPS

## What is covered and what is not

The table below is the honest boundary. It should be read before any release decision, and it should be updated whenever a gap is closed.

TABLE 74 — COVERAGE BY AREA

| AREA | COVERED | GAP |
|---|---|---|
| **Authentication and tokens** | Sign-in, wrong password, expired token, token kind, claims | — |
| **Tenancy isolation** | Missing tenant, foreign identifier, client token on staff route | Cross-tenant aggregation edge cases are covered by a single test |
| **Gate passes and deliveries** | Creation, status, partial delivery, over-delivery rejection | Bulk import paths are not exercised |
| **Bills and payments** | Issue, immutability, payment posting, reversal, balances | Concurrent payment posting is tested at the unit level only |
| **Shop bills** | Creation, tender, day close totals | Refund approval workflow is lightly covered |
| **Linen lifecycle** | All ten actions, invalid transitions, discrepancy records | Scanner hardware behaviour is simulated, not real |
| **Hotel client portal** | Scope filtering, summary, passes, deliveries, bills | PDF rendering of invoices is not asserted |
| **Front-end applications** | TypeScript compilation and production build | No rendered-DOM or interaction tests |




FIGURE 28 — MEASURED COVERAGE BY LAYER.

> WARNING
>
> There is no browser-driven end-to-end suite and no load testing. A regression that only appears when a real browser renders a real form will not be caught by these suites; it must be caught by the manual release check.

12.4 · RUNNING THE SUITES

# Running the suites

Each service has its own virtual environment and its own suite. Running them is a local operation and requires a reachable MongoDB instance; the tests create and drop their own database.

### Service suites

```
cd user-service
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
python -m pytest -q

cd ..\bill_service
python -m pytest -q
```

The expected results are the ones stated in 12.2: 68 passing for user-service, and 380 passing with 2 skipped for bill_service. A different count means the environment differs from the one those figures were measured in — check the Python version and the installed dependencies before treating it as a regression.

### Front-end builds

```
cd shop-app
npm ci
npm run build

# repeat for hotel-app, linen-room-app, quotations-ui
```

A successful build is the release gate for the front ends: TypeScript must compile with no errors and Vite must emit a bundle. Warnings are reviewed but do not block a release unless they indicate an unused secret or a missing environment variable.

TABLE 75 — TROUBLESHOOTING A SUITE RUN

| SYMPTOM | CAUSE | ACTION |
|---|---|---|
| **Collection error naming a missing module** | Dependency absent from the local virtualenv | Install the dependency, or accept the documented gap in 12.5 |
| **Database connection refused** | No local MongoDB, or wrong URI | Start MongoDB and set the test URI |
| **Tests pass individually but fail together** | Shared state between tests | Report as a defect; do not reorder tests to hide it |
| **Build fails on a missing VITE_ variable** | Environment file absent | Copy the example env file and fill the values |

12.5 · KNOWN ENVIRONMENT LIMITATIONS

## Known environment limitations

One suite cannot be collected in the local virtual environment. `tests/test_security_middleware.py` imports `httpx2`, which is not installed there. This is an environment gap, not a product defect: the middleware it exercises is covered indirectly by the authentication and tenancy tests that do run, and the module is expected to collect once the dependency is present.

TABLE 76 — KNOWN LIMITATIONS

| LIMITATION | EFFECT | STATUS |
|---|---|---|
| **`httpx2` missing in the local virtualenv** | `tests/test_security_middleware.py` cannot be collected | Environment gap; install the dependency to include it |
| **No browser-driven end-to-end suite** | Rendered behaviour is verified manually | Accepted gap; manual release check required |
| **No load or soak testing** | Throughput and behaviour under sustained load are unknown | Accepted gap; monitor in production |
| **2 skipped bill_service tests** | Paths requiring an unconfigured external dependency are unexercised | Deliberate skip; documented here |
| **Scanner hardware not simulated** | Scan timing and wedge-mode behaviour are not asserted | Accepted gap; station test at deployment |

### Why these are stated rather than hidden

A manual that lists only what works is a sales document. The value of this chapter is that a new maintainer can see exactly where confidence ends — and can decide, with the owner, whether a gap matters for the next release. The gaps above are known, bounded and owned; none of them is a surprise waiting to be discovered during an incident.

> NOTE
>
> Closing the `httpx2` gap requires only adding the dependency to the test requirements. It is listed as a limitation rather than fixed here because this manual reports the environment as measured, not as it could be.

13

# Operations Runbook and Monitoring

*The routine that keeps the platform observable, and the first five minutes of the incidents that interrupt it.*

## 13.1 Health checks

Each service exposes a health endpoint that answers two questions separately: is the process alive, and can it reach what it depends on? A single combined check is worse than useless, because it cannot distinguish a dead service from a healthy service with a dead database.

TABLE 77 — HEALTH PROBES

| PROBE | CHECKS | USED BY |
|---|---|---|
| **Liveness** | Process responds; no dependency access | Supervisor restart policy |
| **Readiness** | Database reachable; key material loaded | Reverse proxy rotation |
| **Deep health** | Database round-trip, key version, outbox backlog | Monitoring, every minute |
| **Auth probe** | A known test account can sign in (staging only) | Release verification |




FIGURE 29 — HEALTH DEPENDENCY CHAIN. THE PROXY ROUTES ONLY TO SERVICES WHOSE READINESS PROBE PASSES.

Health output is JSON with a status per dependency, so that a monitoring system can alert on the specific dependency rather than on "service unhealthy". The outbox backlog is included because a service that is healthy but cannot drain its queue is failing slowly and silently.

> NOTE
>
> Never use a deep health check as a liveness probe. A database blip would then restart healthy services, turning a short dependency failure into a restart storm.

13.2 · LOGS

## Logs

Logs are structured JSON, one event per line, written to stdout and collected by the log pipeline. They are designed to answer "who did what, to which record, in which tenant, and did it work?" without a debugger.

TABLE 78 — LOG FIELDS

| FIELD | CONTENT |
|---|---|
| **time** | ISO-8601 timestamp with offset |
| **level** | info, warning, error |
| **service** | user-service, bill_service, and so on |
| **request_id** | Correlates every log line from one request, including cross-service calls |
| **user_id / role** | Actor, when authenticated |
| **laundry_id** | Tenant, when resolved — never a tenant guessed from the payload |
| **action** | Route or operation name |
| **target** | Record identifier, when applicable |
| **outcome** | Status code or result class |
| **duration_ms** | Handler duration |

### What is never logged

- Passwords, in any form, including on failed sign-in.
- Bearer tokens or signing keys.
- Full request bodies for authentication and payment routes.
- Personal data beyond what an audit entry requires.

### Retention and access

Application logs are retained for 30 days at full detail and 12 months at reduced detail. Audit entries are retained for seven years and are append-only. Access to logs is restricted to the platform operator; a laundry administrator may be given a filtered extract for their own tenant when investigating a dispute, but never a cross-tenant view.

> WARNING
>
> A stack trace that includes a connection string is a credential leak. The logging configuration redacts known secret patterns, but new code must not log configuration objects wholesale.

13.3 · ALERTS AND FIRST RESPONSE

## Alerts and first response

An alert is only useful if the first action is obvious. Each alert below names the likely cause and the first thing to check, so that the person woken at night does not begin with an investigation.

TABLE 79 — ALERT CATALOGUE

| ALERT | LIKELY CAUSE | FIRST RESPONSE |
|---|---|---|
| **Sign-in failure rate above baseline** | Credential stuffing, or a broken key rotation | Check the failure reason distribution; block the source if it is external |
| **5xx rate above baseline on bill_service** | Database unreachable, or a bad release | Check readiness and the most recent deployment |
| **Backup job missed** | Scheduler stopped, or storage credential expired | Run the job manually; verify the credential |
| **Backup size drop > 20%** | Partial dump or empty database | Treat as failed; do not rotate retention until explained |
| **Disk usage above 80%** | Log growth or oplog retention | Prune logs, verify retention policy, plan capacity |
| **Oplog window below 24 hours** | Write volume outpacing oplog size | Increase oplog size before the RPO degrades |
| **Outbox backlog growing** | Accounting service unavailable | Check the dependency; the queue drains on recovery |
| **Certificate expiring in 14 days** | Renewal failed | Renew manually; verify the proxy reload |
| **Repeated 403 on one account** | Role removed, or a client token used on a staff route | Confirm the account type and membership |

### Escalation discipline

Acknowledge, then act. An alert acknowledged and forgotten is worse than one ignored, because it hides the problem from everyone else. If the first response does not resolve the alert within its target, escalate the severity rather than extending the window quietly.

> NOTE
>
> Alert fatigue is a design defect. If an alert fires routinely without action, either the threshold is wrong or the underlying problem should be fixed; leaving it in place trains people to ignore the one that matters.

13.4 · ROUTINE MAINTENANCE CALENDAR

# Routine maintenance calendar

Maintenance is scheduled so that it never competes with a day close or a Monday collection. The calendar is deliberately boring: the same tasks, at the same cadence, with the same owner.

TABLE 80 — MAINTENANCE SCHEDULE

| CADENCE | TASK | OWNER |
|---|---|---|
| **Daily** | Verify the overnight backups completed and their sizes are normal | Platform operator |
| **Daily** | Review the error log for new patterns | Platform operator |
| **Weekly** | Run the restore verification job against the latest dump | Platform operator |
| **Weekly** | Review refunds, voids and discounts by user | Laundry manager |
| **Monthly** | Apply operating-system and dependency security updates | Platform operator |
| **Monthly** | Review disk, oplog window and database growth | Platform operator |
| **Quarterly** | Full restore drill with timing (10.7) | Platform operator |
| **Quarterly** | Review the risk register and incident actions | Owner and operator |
| **Annually** | Rotate the search key with a planned backfill | Platform operator |
| **Annually** | Review roles, memberships and dormant accounts | Laundry administrator |




Figure 30 — Maintenance cadence. Daily tasks are verification; the heavier work is scheduled away from trading peaks.

FIGURE 30 — MAINTENANCE CALENDAR BY CADENCE.

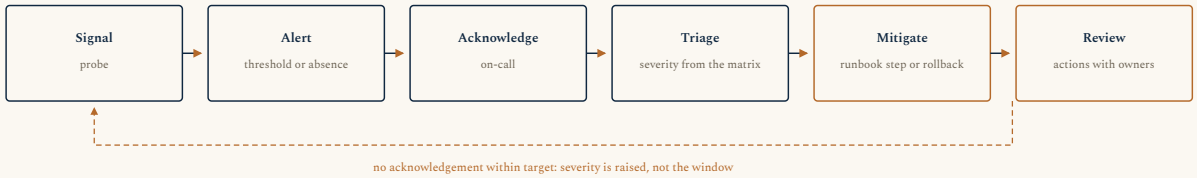
> WARNING
>
> Never apply security updates to the database host during trading hours without a tested rollback. A patched database that will not start is an outage with a maintenance window's worth of warning wasted.

13.5 · ALERT AND ESCALATION FLOW

# Alert and escalation flow

*From signal to review, with the loop that raises severity when nobody answers.*

ALERT AND ESCALATION FLOW




Every alert names the likely cause and the first check, so the person woken at night does not begin with an investigation.

An alert that fires routinely without action is a defect: fix the threshold or the cause.

**FIGURE 31** — ALERT LIFECYCLE AND THE ESCALATION LOOP WHEN ACKNOWLEDGEMENT DOES NOT HAPPEN WITHIN TARGET.

An alert is only useful if the first action is obvious, so every alert in the catalogue names a likely cause and the first thing to check. The flow above is the path from a signal to a written action, and it is deliberately short.

The loop at the top is the part that is usually missing: if nobody acknowledges within the target, severity is raised rather than the window quietly extended.

> RULE
>
> An alert that fires routinely without action is a design defect. Either the threshold is wrong or the cause should be fixed; leaving it trains people to ignore the one that matters.

PART FOUR · APPENDICES

# Appendices

*Reference tables, matrices and enumerations: endpoints, roles, statuses, vocabulary and revision history.*

Paths are the resource roots exposed by each FastAPI service. Method and path combinations below are the ones the front ends depend on; the authoritative schema for any deployment is served at `/openapi.json` by each service.

TABLE 81 — USER-SERVICE ENDPOINTS (PORT 8002)

| METHOD | PATH | PURPOSE | AUTH |
|---|---|---|---|
| **POST** | /auth/login | Staff sign-in; returns a 24-hour staff token | None |
| **POST** | /auth/refresh | Issue a fresh token for a valid session | Staff token |
| **GET** | /users | List users in the tenant | ADMIN |
| **POST** | /users | Create a user | ADMIN |
| **PATCH** | /users/{id} | Update display name or status | ADMIN |
| **GET** | /laundries | List laundries | PLATFORM_ADMIN |
| **POST** | /laundries | Create a laundry (tenant) | PLATFORM_ADMIN |
| **GET** | /memberships | List memberships for the tenant | ADMIN |
| **POST** | /memberships | Grant a role to a user | ADMIN |
| **DELETE** | /memberships/{id} | Remove a user's access | ADMIN |

TABLE 82 — CLIENT AND HOTEL ACCOUNT ENDPOINTS

| METHOD | PATH | PURPOSE | AUTH |
|---|---|---|---|
| **POST** | /client/auth/login | Hotel client sign-in; 12-hour client token | None |
| **GET** | /client/auth/status | Current client account state | Client token |
| **GET** | /clients | List client accounts for the tenant | ADMIN, MANAGER |
| **POST** | /clients | Create a client account bound to one client_name | ADMIN |
| **PATCH** | /clients/{id}/status | Enable or disable a client account | ADMIN |

> NOTE
>
> Staff routes reject client tokens because the tenant dependency cannot resolve `laundry_id` from a client token. Client routes reject staff tokens for the mirror-image reason: there is no `client_scope`.

APPENDIX A · API REFERENCE

# API reference — bill_service

TABLE 83 — OPERATIONAL ENDPOINTS (PORT 8001)

| METHOD | PATH | PURPOSE |
|---|---|---|
| **GET** | /gate-passes | List passes for the tenant, filtered by status and date |
| **POST** | /gate-passes | Create a pass with item lines |
| **GET** | /gate-passes/{id} | Pass detail with lines and delivery history |
| **PATCH** | /gate-passes/{id} | Amend a pass (manager or above) |
| **POST** | /gate-passes/{id}/cancel | Cancel with a reason; the number is retained |
| **GET** | /deliveries | List deliveries |
| **POST** | /deliveries | Record a delivery against pass lines |
| **GET** | /deliveries/{id} | Delivery detail and receiver |
| **POST** | /deliveries/{id}/reverse | Reverse a delivery with a reason |
| **GET** | /bills | List bills with status and balance |
| **POST** | /bills | Raise a bill from unbilled deliveries |
| **GET** | /bills/{id} | Bill detail with lines and payments |
| **POST** | /bills/{id}/payments | Post a payment against the bill |
| **POST** | /payments/{id}/reverse | Reverse a posted payment |

TABLE 84 — SHOP AND REPORTING ENDPOINTS

| METHOD | PATH | PURPOSE |
|---|---|---|
| **GET** | /shop-bills | List counter sales for a business date |
| **POST** | /shop-bills | Create a shop bill |
| **POST** | /shop-bills/{id}/pay | Complete payment and issue the receipt |
| **POST** | /shop-bills/{id}/void | Void before payment, with a reason |
| **POST** | /shop-bills/{id}/refund | Refund a paid bill; requires approval |
| **GET** | /reports/monthly-matrix | Client-by-day quantities and values |
| **GET** | /reports/unbilled | Deliveries not yet on a bill |
| **POST** | /day-close | Close the business date |
| **GET** | /day-close/{date} | Close record for a date |
| **GET** | /dashboard | Operational tiles for the tenant |

APPENDIX A · API REFERENCE

# API reference — linen, portal and supporting services

TABLE 85 — LINEN ENDPOINTS

| METHOD | PATH | PURPOSE |
|---|---|---|
| **GET** | /linens | List tags by status, client or item type |
| **POST** | /linens/scan | Submit a scan event for a tag |
| **POST** | /linens/{tag}/action | Apply a lifecycle action to a tag |
| **GET** | /linens/{tag}/history | Full event history for a tag |
| **POST** | /linens/{tag}/retire | Retire an item (supervisor only) |
| **GET** | /linens/inventory | Counts by status, type and client |
| **GET** | /linens/discrepancies | Open and closed discrepancies |

TABLE 86 — CLIENT PORTAL ENDPOINTS (BILL_SERVICE)

| METHOD | PATH | PURPOSE |
|---|---|---|
| **GET** | /client-portal/summary | Passes in progress, items returned, outstanding balance |
| **GET** | /client-portal/gate-passes | Passes scoped to the client_name in client_scope |
| **GET** | /client-portal/deliveries | Deliveries scoped to the client_name |
| **GET** | /client-portal/bills | Invoices for this property |
| **GET** | /client-portal/bills/{id} | Invoice detail, scoped by both keys |

TABLE 87 — SUPPORTING SERVICES

| SERVICE | RESPONSIBILITY | FRONT END |
|---|---|---|
| **quotation-service** | Quotations, line pricing, AI chat | quotations-ui |
| **worker-service** | Worker records and attendance | quotations-ui |
| **laundry-management** | Customers, transactions, payments, expenses, salary, employees, items | quotations-ui |
| **love-ai** | Drafting and summarised answers over permitted data | quotations-ui |

> NOTE
>
> Every client-portal query filters on both `laundry_id` and `client_name`. A request that supplies only an invoice identifier returns the invoice only if both keys match; otherwise it is indistinguishable from a non-existent record.

APPENDIX B

# Role and capability matrix

The matrix is the authoritative statement of which role holds which capability. [•] means granted; a blank cell means refused, and a refused capability is refused by the server regardless of what the interface shows.

TABLE 88 — CAPABILITIES BY ROLE

| CAPABILITY | ADMIN | PLATFORM_ADMIN | MANAGER | STAFF | LINEN_SUP. | HOUSEKEEPER | HOTEL_ADMIN | HOTEL_MGR |
|---|---|---|---|---|---|---|---|---|
| gatepass:read | ● | ● | ● | ● | ● | | ●[1] | ●[1] |
| gatepass:write | ● | ● | ● | ● | | | | |
| delivery:read | ● | ● | ● | ● | ● | | ●[1] | ●[1] |
| delivery:write | ● | ● | ● | ● | | | | |
| bill:read | ● | ● | ● | ● | | | ●[1] | ●[1] |
| bill:write | ● | ● | ● | ● | | | | |
| payment:read | ● | ● | ● | | | | | |
| payment:write | ● | ● | ● | | | | | |
| linen:read | ● | ● | ● | ● | ● | ● | | |
| linen:scan | ● | ● | ● | | ● | ● | | |
| linen:write | ● | ● | ● | | ● | | | |
| dashboard:read | ● | ● | ● | ● | ● | ● | | |
| report:read | ● | ● | ● | | | | | |
| user management | ● | ● | | | | | | |
| tenant creation | | ● | | | | | | |

[1] Hotel roles do not hold staff capabilities. Their access to gate passes, deliveries and bills is granted by `/client-portal` endpoints that check `client_scope`, not by the capability list. The marks are shown here to make the read-only equivalence visible; the mechanism is different, and deliberately so.

> NOTE
>
> PLATFORM_ADMIN is a superset of ADMIN, but only across tenants. It still cannot read a hotel client's records without a client session, because the portal scope is a separate dimension.

APPENDIX C

## Status enumerations

Statuses are stored as fixed strings. They are listed here because they appear in exports, reports and hotel-facing documents, and parsed by external tools that must not invent new values.

TABLE 89 — STATUS VALUES BY DOCUMENT

| DOCUMENT | VALUES | NOTES |
|---|---|---|
| **Gate pass** | received, in_wash, ready, delivered, closed, cancelled | Cancelled passes retain their number |
| **Delivery** | recorded, reversed | Reversal is a new record referencing the original |
| **Bill** | draft, issued, part_paid, paid, void | Issued bills are immutable |
| **Payment** | posted, reversed | Reversal never deletes the original |
| **Shop bill** | draft, open, paid, refunded, void | Separate numbering from laundry bills |
| **Quotation** | draft, sent, accepted, declined, expired | Conversion creates work, not a bill |
| **Linen tag** | collected, received, in_wash, washed, pressed, ready, delivered, missing, damaged, retired | Ten lifecycle states, forward transitions only |
| **Discrepancy** | open, resolved, written_off | Resolution requires a supervisor |
| **Client account** | active, disabled | Disabled blocks new sign-ins |
| **User** | active, disabled | Membership removal is separate from disabling |

### Why fixed strings

Free-text statuses make reports unreliable, because every spelling becomes a category. Fixed enumerations mean the monthly matrix, the dashboard and the hotel portal all count the same thing. When a new status is genuinely required, it is added to the enumeration, to the transition rules and to the tests in the same change.

> WARNING
>
> Never write a status that is not in this table. A value the platform does not recognise will disappear from every filtered view while remaining in the database, which is the hardest kind of missing record to find.

APPENDIX D

# Glossary

Terms as they are used in this manual and in the interface. Where a term has a narrower meaning here than in general use, the narrower meaning applies.

TABLE 90 — GLOSSARY OF TERMS

| TERM | DEFINITION |
|---|---|
| **Audit entry** | An append-only record of a security-relevant action: actor, tenant, action, target, outcome. |
| **Capability** | A named permission such as bill:write, carried in the token and checked per route. |
| **Client account** | A hotel login stored in client_accounts; scoped to one laundry and one client_name. |
| **client_name** | The string identifying a hotel on operational documents. Not an entity, not a tenant. |
| **client_scope** | The claim object on a client token containing laundry_id and client_name. |
| **Day close** | The act of freezing a business date's totals and recording who closed it. |
| **DEK** | Data encryption key: a per-document key used with AES-256-GCM. |
| **Delivery** | The record of returning finished goods against a gate pass. |
| **Discrepancy** | A recorded mismatch in quantity, condition or status. |
| **Gate pass** | The document recording goods received from a client and the promise to return them. |
| **Idempotency key** | A client-generated key that makes a retried write return the original result. |
| **KEK** | Key-encryption key: wraps DEKs and is held outside the database. |
| **Membership** | The link between a user and a laundry, carrying a role. |
| **Monthly matrix** | A client-by-day grid of quantities and values derived from passes and deliveries. |
| **Outbox** | A durable record of a side effect to be retried until it succeeds. |
| **RPO / RTO** | Recovery point objective (acceptable data loss) and recovery time objective (acceptable downtime). |
| **Search token** | An HMAC-SHA256 value enabling equality search over encrypted fields. |
| **Tenant** | A laundry; identified by laundry_id on every business document. |
| **token_kind** | A claim distinguishing staff tokens from client tokens. |

APPENDIX E

# Revision history and conventions

TABLE 91 — REVISION HISTORY

| VERSION | DATE | AUTHOR | CHANGE |
|---|---|---|---|
| **0.1** | Sep 2026 | R. D. Siriwardhana | Outline and architecture notes; not issued |
| **0.9** | Oct 2026 | R. D. Siriwardhana | Draft for internal review: chapters 1–8 |
| **1.0** | 24 Oct 2026 | R. D. Siriwardhana | First complete issue: 105 pages, all chapters and appendices |

## Document conventions

TABLE 92 — TYPOGRAPHIC AND STRUCTURAL CONVENTIONS

| CONVENTION | MEANING |
|---|---|
| **Numbered procedure** | Steps to be followed in order; each step is an action on the interface |
| **Bold interface label** | A control, field or screen name as it appears in the application |
| **Monospace** | Field name, endpoint, environment variable, status value or code |
| **Note callout (navy)** | Behaviour that is correct but easy to misread |
| **Warning callout (accent)** | An action that can lose data, expose it, or break a control |
| **Figure n.m / Table n.m** | Numbered by chapter; referenced in text by number |
| **Page footer** | Document title and page number on every page, including the cover |

### Maintenance of this manual

The manual is reviewed at every minor release and after every Severity 1 or 2 incident. A change to the platform that alters a procedure, an endpoint, a status value or a control requires a corresponding change here in the same release — a manual that lags the software is worse than no manual, because it is trusted.

> NOTE
>
> Corrections are recorded as new revisions rather than silent edits. The revision table is the record of what changed and when, and it is the first thing a new maintainer should read.

APPENDIX F

# Index

Page references are to the printed page numbers in the footer.

TABLE 93 — A – L


| TERM | PAGE |
|---|---|
| Access removal | 64 |
| Account lockout | 19 |
| API reference | 98 |
| Audit log | 23, 95 |
| Authentication | 19 |
| Backup inventory | 76 |
| Backup schedule | 78 |
| Bill immutability | 30, 85 |
| Capabilities | 20, 101 |
| Circuit breaking | 86 |
| Client accounts | 16, 65 |
| client_name | 16 |
| client_scope | 16 |
| Day close | 35, 41 |
| Degraded mode | 87 |
| Deliveries | 29 |
| Discrepancies | 59 |
| Disaster recovery | 68 |
| Encryption at rest | 22 |
| Envelope encryption | 22 |
| Environment variables | 66 |
| Fail-closed tenancy | 83 |
| Gate passes | 27 |
| Glossary | 103 |
| Health checks | 94 |
| Hotel portal | 46 |
| Idempotency | 84 |
| Incident severity | 73 |
| Inventory (linen) | 58 |
| JWT claims | 15 |
| KEK | 22 |
| Linen lifecycle | 56 |
| Linen room app | 55 |


TABLE 94 — M – Z


| TERM | PAGE |
|---|---|
| Memberships | 64 |
| Monthly matrix | 34 |
| Offline behaviour | 86 |
| Optimistic concurrency | 84 |
| Payments | 31 |
| Point-in-time recovery | 81 |
| Quotations | 32 |
| Rate limiting | 23 |
| Recovery objectives | 77 |
| Refunds | 43 |
| Reports | 34 |
| Restore drill | 82 |
| Restore procedure | 79 |
| Retirement (linen) | 60 |
| Risk register | 69 |
| Roles | 20, 101 |
| RPO / RTO | 77 |
| Scan workflow | 57 |
| Search tokens | 22 |
| Secrets management | 24 |
| Service catalogue | 10 |
| Shop counter | 37 |
| Statements | 51 |
| Status enums | 102 |
| Tag printing | 60 |
| Tenant isolation | 21 |
| Testing | 89 |
| Token lifetimes | 19 |
| Travel: none | — |
| Users | 64 |
| Verification checks | 80 |
| Workers | 33 |


NOTE

Bold page numbers in a printed index conventionally denote the principal treatment; this index gives a single page per term to keep the reference unambiguous.